# Embedded Systems 12th Week

## – SPI and I2C communication –

Jaeseok Yun

Soonchunhyang University

# Agenda

- SPI hardware and spec
- SPI communication applications
- $I^2C$ hardware and spec
- $I^2C$ communication applications

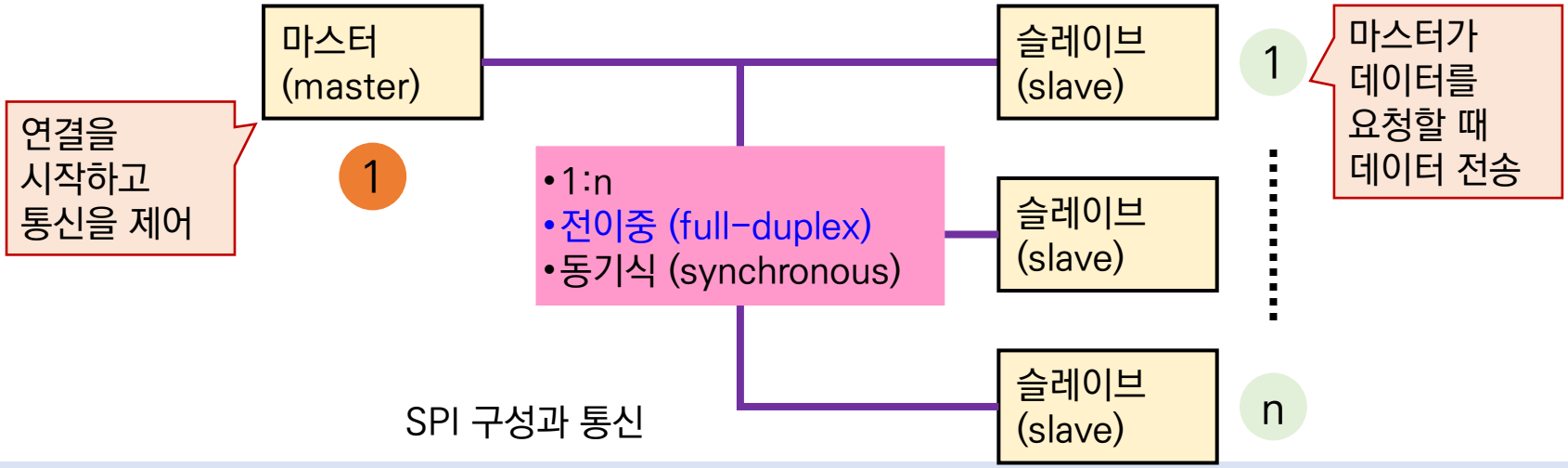
# SPI 통신

Serial Peripheral Interface

# SPI Serial Peripheral Interface

## ■ Serial Peripheral Interface[1,2]

- 마스터 (master) 장치와 하나 이상의 슬레이브 (slave) 장치로 구성
  - ✓ 1 (마스터) : n (슬레이브) 통신이 가능
- 데이터 송신과 수신이 동시에 진행될 수 있는 전이중 (full-duplex) 통신 표준
- 특정 슬레이브 선택 (Slave Select) 선을 이용해 1:n 통신 구현 ($I^2C$와 비교)
- 마이크로프로세서와 주변장치 (peripheral) 간 '고속, 근거리' 통신 (버스, bus[4])에서 많이 활용
  - ✓ 예, 센서, SD 카드, 레지스터 등




SPI 구성과 통신

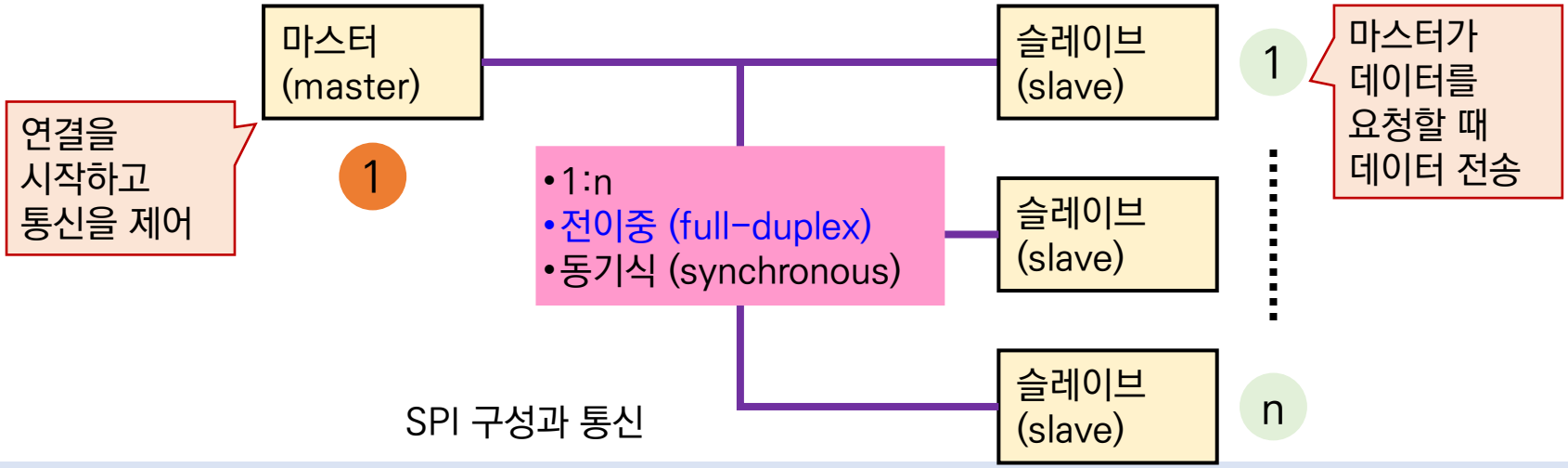
[1] https://en.wikipedia.org/wiki/Serial_Peripheral_Interface
[2] https://learn.sparkfun.com/tutorials/serial-peripheral-interface-spi/all
[3] de facto 표준이란 대중성과 시장 장악력을 바탕으로 어느 회사나 단체의 특정 방식 (예, 제조, 통신 등)이 표준 처럼 인정되는 것, 예로서 HDMI 포트
[4] 컴퓨터 구조에서 '버스'란 컴퓨터 내부에서 컴포넌트 사이에 또는 컴퓨터 사이에 데이터를 주고 받는 통신 시스템을 통칭하는 말

# SPI Serial Peripheral Interface

- 모토롤라에서 개발한 산업 (de facto) 표준[3]
- SPI 프로토콜은 공식적인 표준을 따르지 않기 때문에 장치마다 약간씩 다르게 동작
- 장치마다 정확한 사용을 위해 데이터 시트 확인!

## ■ Serial Peripheral Interface[1,2]

- 마스터 (master) 장치와 하나 이상의 슬레이브 (slave) 장치로 구성
  - ✓ 1 (마스터) : n (슬레이브) 통신이 가능
- 데이터 송신과 수신이 동시에 진행될 수 있는 전이중 (full-duplex) 통신 표준
- 특정 슬레이브 선택 (Slave Select) 선을 이용해 1:n 통신 구현 ($I^2C$와 비교)
- 마이크로프로세서와 주변장치 (peripheral) 간 '고속, 근거리' 통신 (버스, bus[4])에서 많이 활용
  - ✓ 예, 센서, SD 카드, 레지스터 등




SPI 구성과 통신

[1] https://en.wikipedia.org/wiki/Serial_Peripheral_Interface
[2] https://learn.sparkfun.com/tutorials/serial-peripheral-interface-spi/all
[3] de facto 표준이란 대중성과 시장 장악력을 바탕으로 어느 회사나 단체의 특정 방식 (예, 제조, 통신 등)이 표준 처럼 인정되는 것, 예로서 HDMI 포트
[4] 컴퓨터 구조에서 '버스'란 컴퓨터 내부에서 컴포넌트 사이에 또는 컴퓨터 사이에 데이터를 주고 받는 통신 시스템을 통칭하는 말

# SPI Serial Peripheral Interface

## ■ SPI 연결선 (wire)과 연결 방법

- MOSI (Master Out Slave In): 마스터에서 슬레이브 장치로 데이터 전송
- MISO (Master In Slave Out): 슬레이브에서 마스터 장치로 데이터 전송
- SCLK (Serial CLocK): 마스터-슬레이브 장치 간 동기화를 위한 클럭 전송
- SS (Slave Select): 마스터가 데이터를 주고 받을 슬레이브 장치를 선택

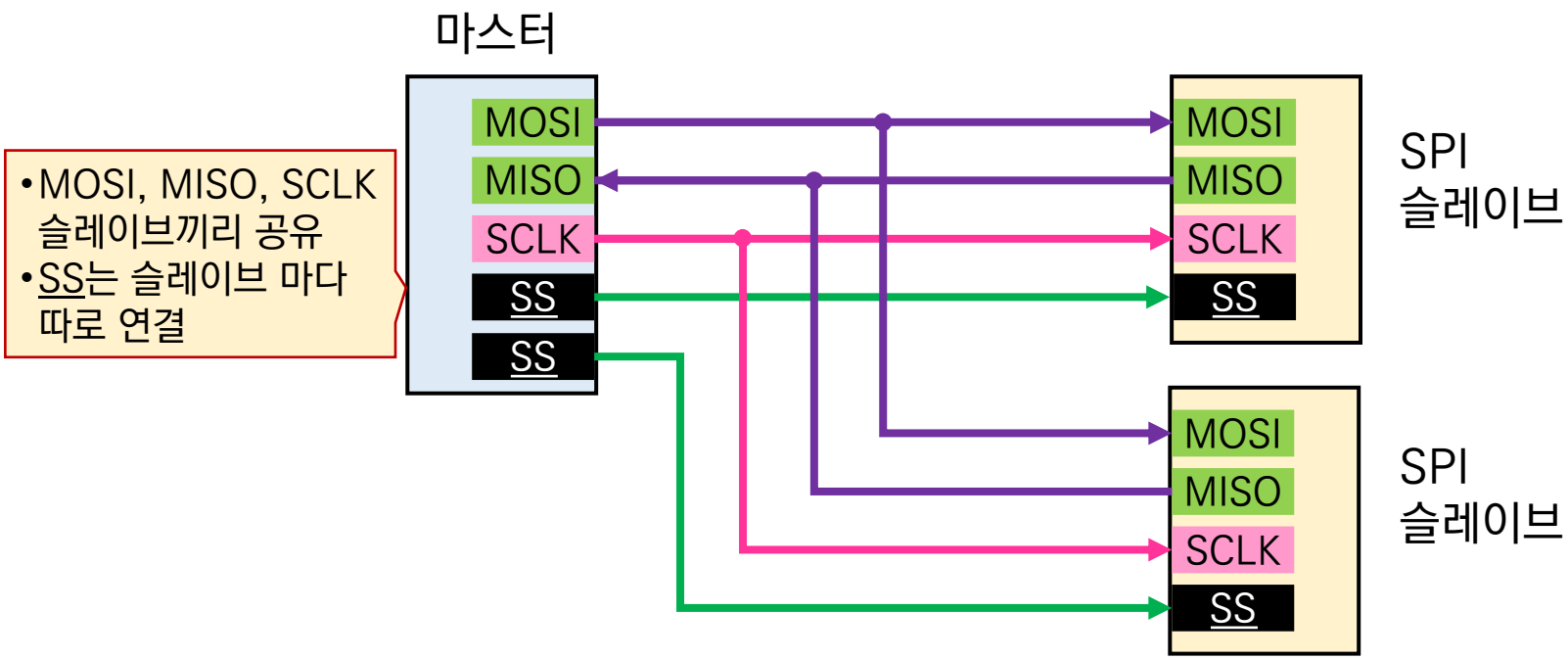
SPI 통신 연결 방법 (1:1)

- SS 연결선의 '_' 는 부논리 동작
- 평상시에는 (선택 받지 않았을 때는) HIGH를 유지, 특정 슬레이브를 선택할 때 LOW

# SPI Serial Peripheral Interface

## ■ SPI 연결선 (wire)과 연결 방법

- MOSI (Master Out Slave In): 마스터에서 슬레이브 장치로 데이터 전송
- MISO (Master In Slave Out): 슬레이브에서 마스터 장치로 데이터 전송
- SCLK (Serial CLocK): 마스터-슬레이브 장치 간 동기화를 위한 클럭 전송
- SS (Slave Select): 마스터가 데이터를 주고 받을 슬레이브 장치를 선택

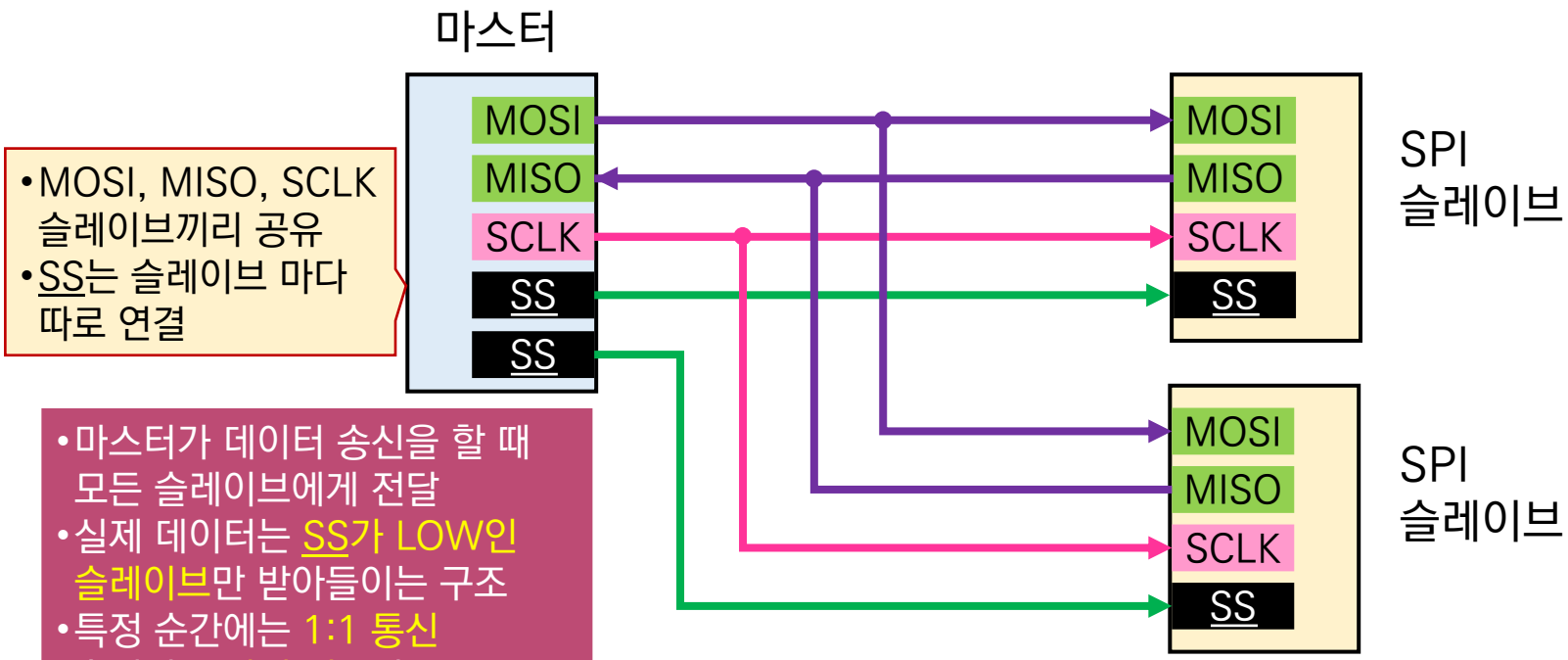



SPI 통신 연결 방법 (1:2)

# SPI Serial Peripheral Interface

## ■ SPI 연결선 (wire)과 연결 방법

- MOSI (Master Out Slave In): 마스터에서 슬레이브 장치로 데이터 전송
- MISO (Master In Slave Out): 슬레이브에서 마스터 장치로 데이터 전송
- SCLK (Serial CLocK): 마스터-슬레이브 장치 간 동기화를 위한 클럭 전송
- SS (Slave Select): 마스터가 데이터를 주고 받을 슬레이브 장치를 선택




SPI 통신 연결 방법 (1:2)

# SPI 데이터 전송 방식[1]

## ■ 마스터-슬레이브 간 동시 송수신

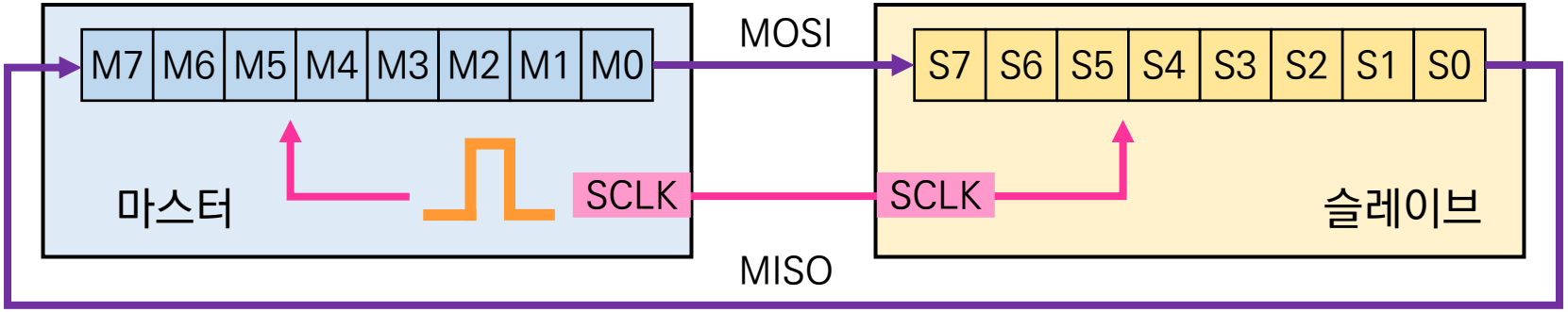
- 항상 송신과 수신이 '동시에' 이루어짐 (UART는 송수신이 별개로 동작)
- SPI는 마스터에서 전달하는 '시리얼 클럭'을 기준으로 데이터를 전송
  - ✓ 마스터에서 슬레이브로 전달할 때 마스터 클럭 기준으로 진행
  - ✓ 슬레이브로 마스터에서 전달할 때 마스터 클럭 기준으로 진행

[1] 허경용, 사물인터넷을 품은 아두이노, p54

# SPI 데이터 전송 방식[1]

## ■ 마스터-슬레이브 간 동시 송수신

- 항상 송신과 수신이 '동시에' 이루어짐 (UART는 송수신이 별개로 동작)
- SPI는 마스터에서 전달하는 '시리얼 클럭'을 기준으로 데이터를 전송
  - ✓ 마스터에서 슬레이브로 전달할 때 마스터 클럭 기준으로 진행
  - ✓ 슬레이브로 마스터에서 전달할 때 마스터 클럭 기준으로 진행
- 마스터와 슬레이브의 데이터 버퍼는 '원형 큐 (circular queue)'를 이룸
  - ✓ 마스터와 슬레이브가 데이터를 주고 받을 때 동시에 이루어짐



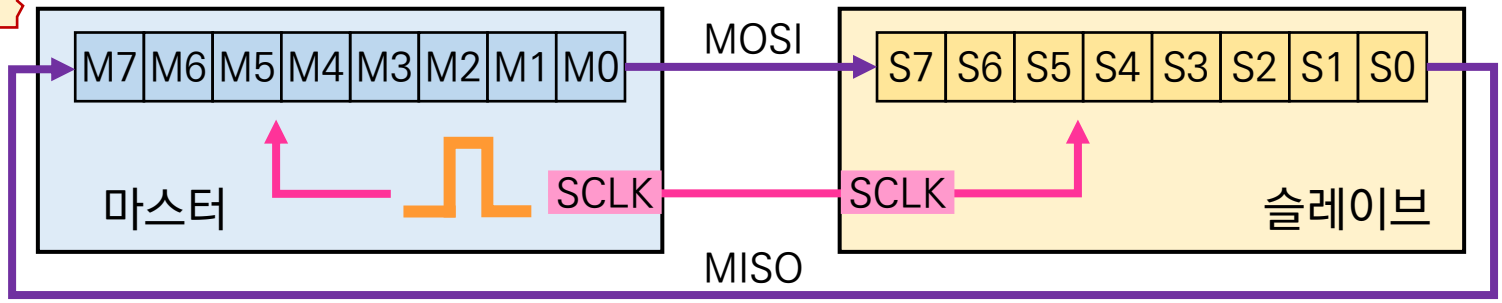

SPI 마스터-슬레이브 간 데이터 전송 구조

[1] 허경용, 사물인터넷을 품은 아두이노, p54

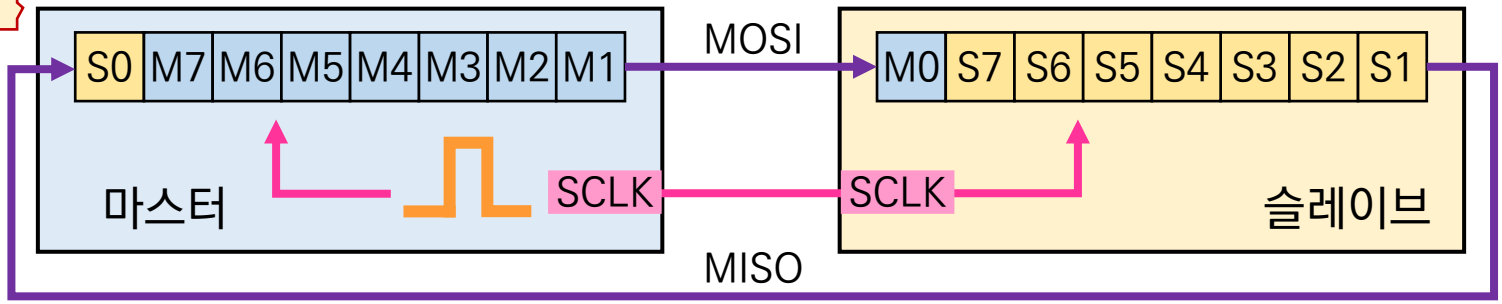
# SPI 데이터 전송 방식[1]

- 시리얼 클럭이 하나 발생하면 마스터 데이터 버퍼의 1 bit 데이터 M0가 슬레이브로 전달되며 동시에,
- 슬레이브 데이터 버퍼의 1 bit 데이터 S0가 마스터로 전달
- 즉 두 버퍼 간 원형 큐 데이터 이동과 동일한 결과
- 8번의 시리얼 클럭으로 1 byte 데이터 '교환' 가능

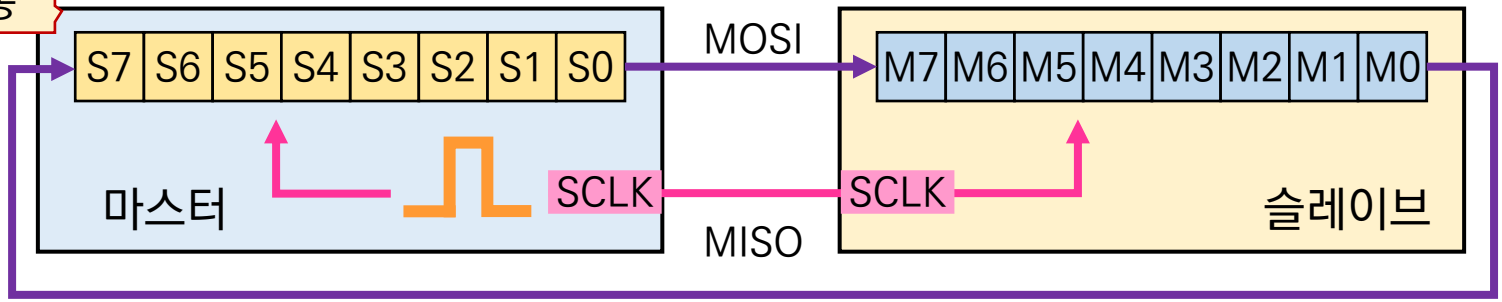
- 대기상태




- 1bit 전송




- 1byte 전송

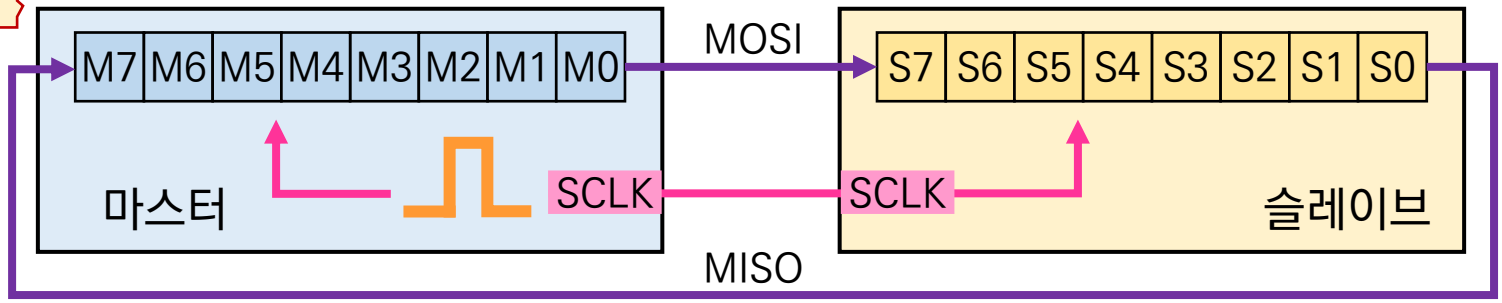



[1] 허경용, 사물인터넷을 품은 아두이노, p54

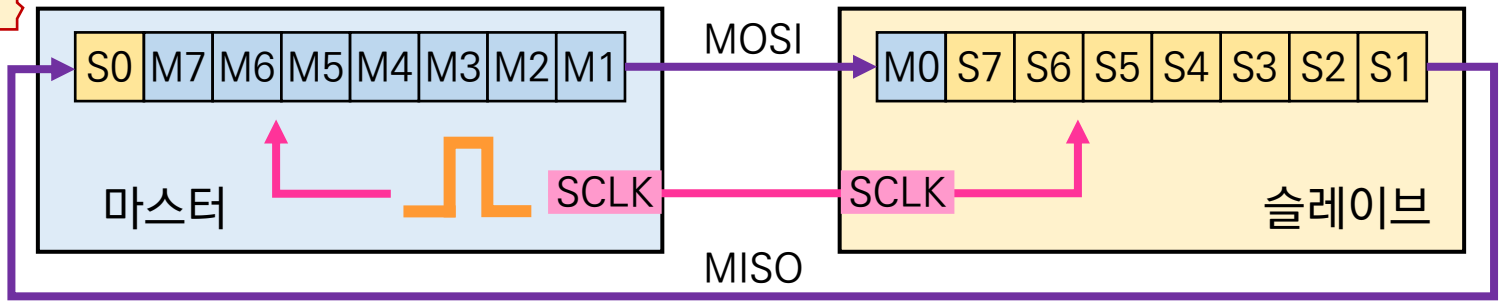
# SPI 데이터 전송 방식[1]

- 시리얼 클럭이 하나 발생하면 마스터 데이터 버퍼의 1 bit 데이터 M0가 슬레이브로 전달되며 동시에,
- 슬레이브 데이터 버퍼의 1 bit 데이터 S0가 마스터로 전달
- 즉 두 버퍼 간 원형 큐 데이터 이동과 동일한 결과
- 8번의 시리얼 클럭으로 1 byte 데이터 '교환' 가능

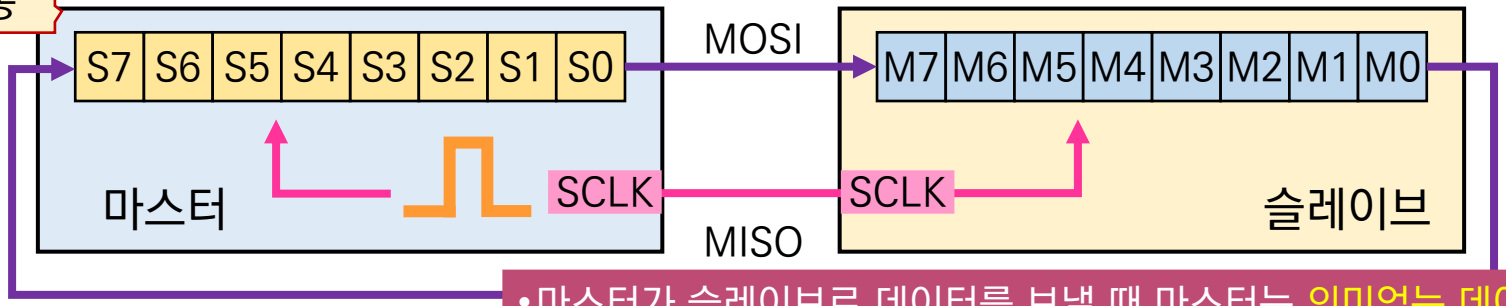
- 대기상태




- 1bit 전송



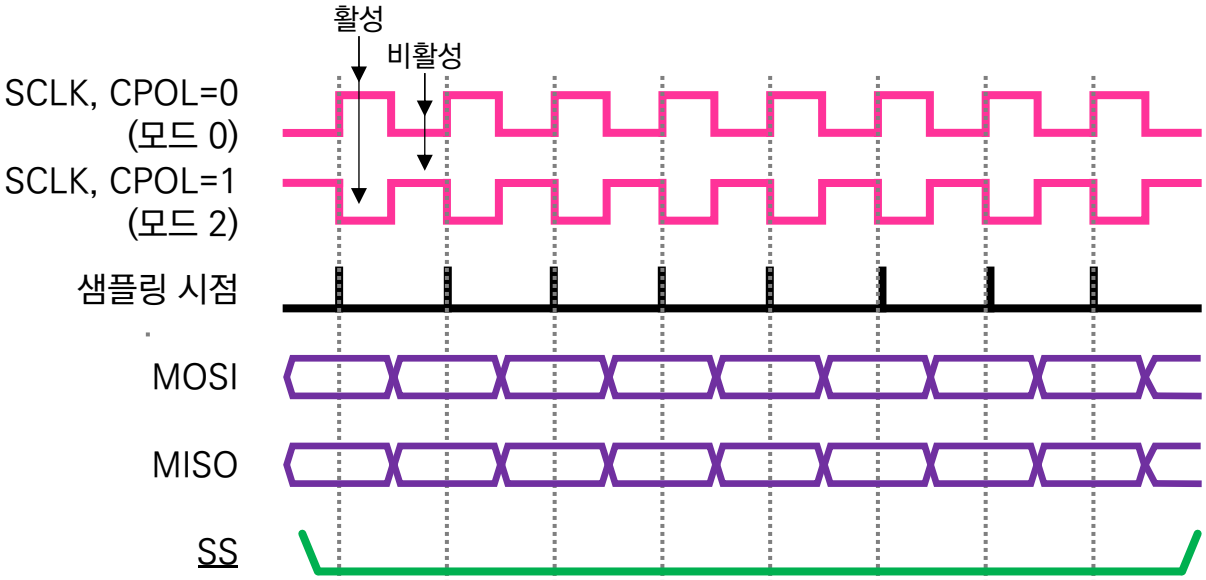

- 1byte 전송

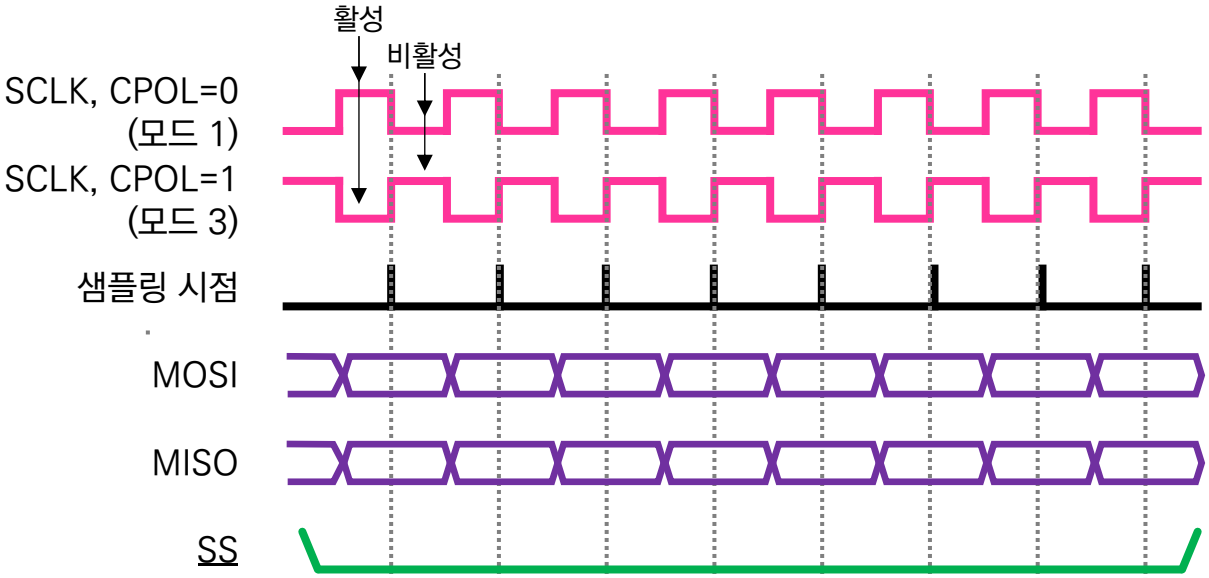



- 마스터가 슬레이브로 데이터를 보낼 때 마스터는 의미없는 데이터를 슬레이브로부터 받음, 반대의 경우도 동일한 결과

[1] 허경용, 사물인터넷을 품은 아두이노, p54

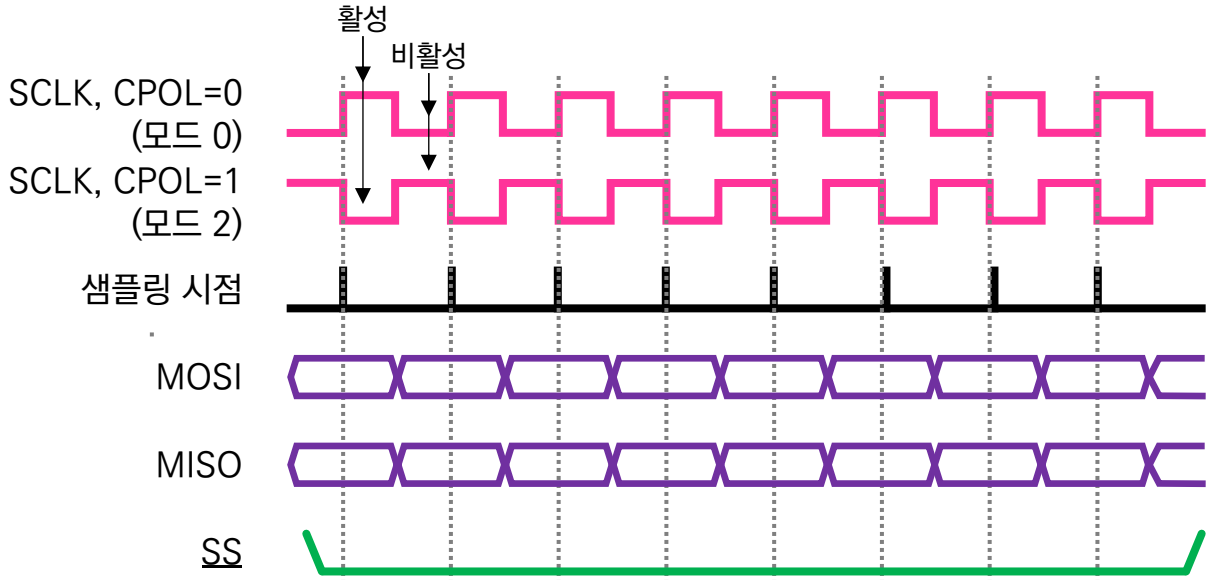
# SPI 데이터 전송 방식[1] 데이터 전송 다이어그램



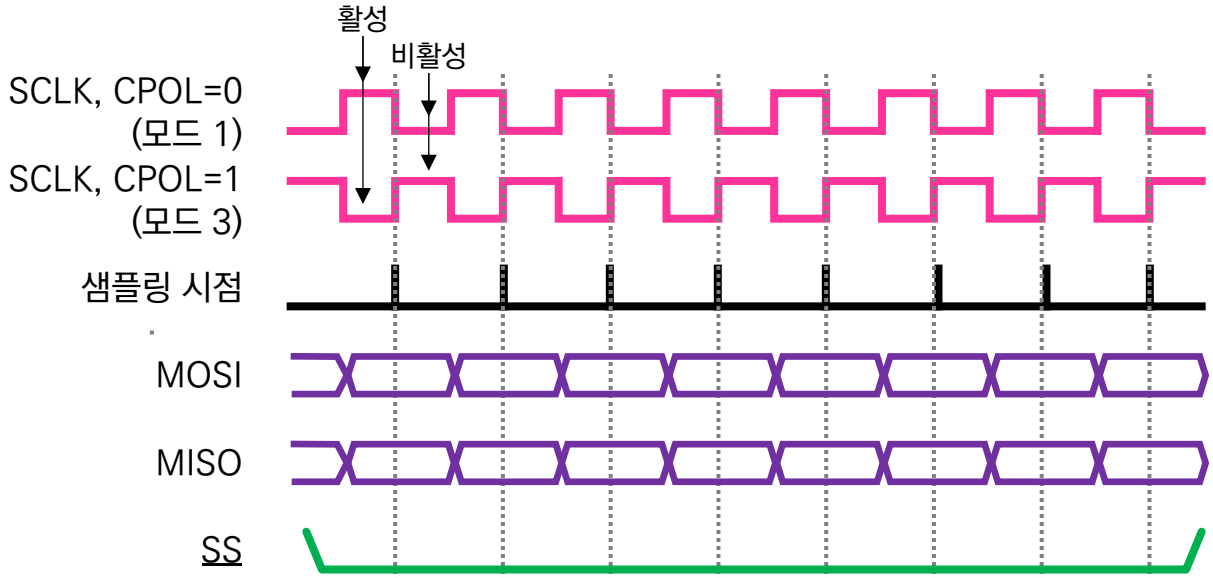



# SPI 데이터 전송 방식[1] 데이터 전송 다이어그램

- CPOL = 0
  비활성일 때 SCLK = LOW
  활성일 때 SCLK = HIGH
- CPOL = 1
  비활성일 때 SCLK = HIGH
  활성일 때 SCLK = LOW




- CPOL: SPI 버스가 유휴 상태일 때 클럭 값 결정 (Clock POLarity)



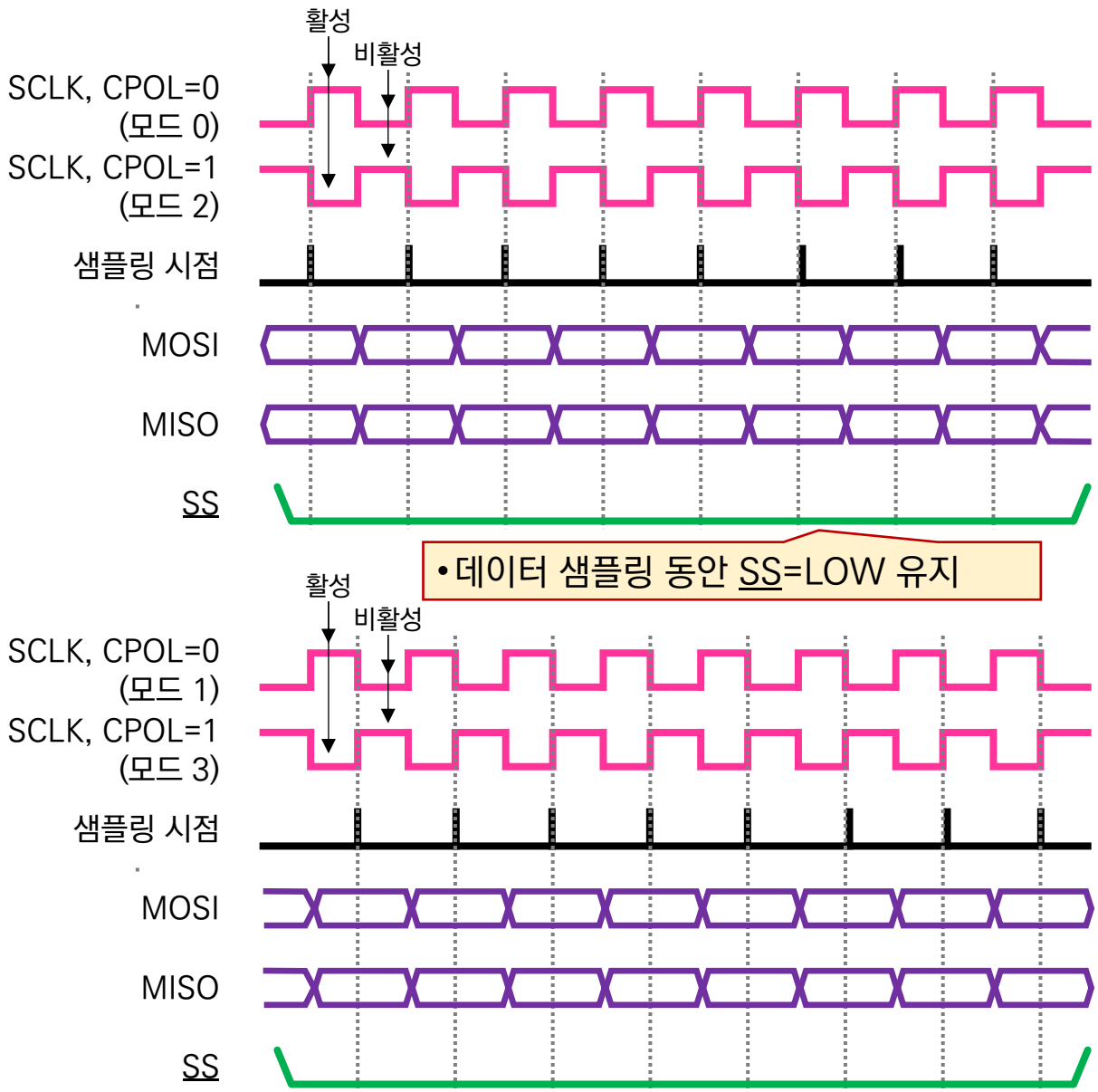
# SPI 데이터 전송 방식[1] 데이터 전송 다이어그램

- CPOL = 0
  비활성일 때 SCLK = LOW
  활성일 때 SCLK = HIGH
- CPOL = 1
  비활성일 때 SCLK = HIGH
  활성일 때 SCLK = LOW

- CPHA = 0
- 비활성→활성으로 바뀌는 에지에서 샘플링

- CPOL: SPI 버스가 유휴 상태일 때 클럭 값 결정 (Clock POLarity)
- CPHA: 데이터 샘플링 시점을 결정 (Clock PHAse)

- CPHA = 1
- 활성→비활성으로 바뀌는 에지에서 샘플링



# SPI 데이터 전송 방식[1] 데이터 전송 다이어그램

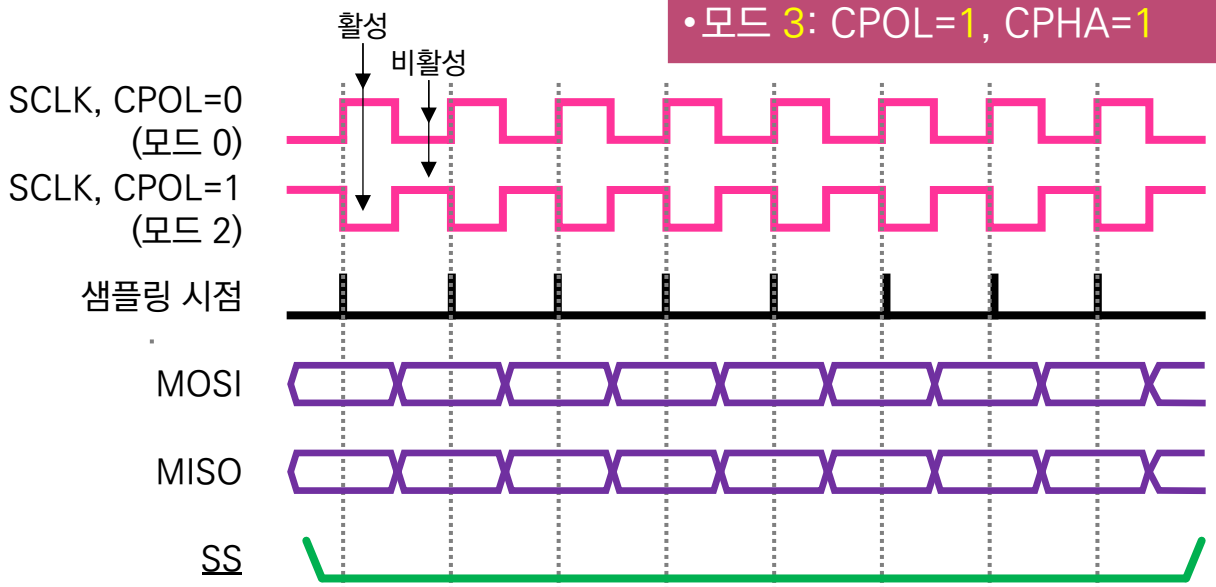
- 모드 0: CPOL=0, CPHA=0
- 모드 1: CPOL=0, CPHA=1
- 모드 2: CPOL=1, CPHA=0
- 모드 3: CPOL=1, CPHA=1

- CPOL = 0
  비활성일 때 SCLK = LOW
  활성일 때 SCLK = HIGH
- CPOL = 1
  비활성일 때 SCLK = HIGH
  활성일 때 SCLK = LOW

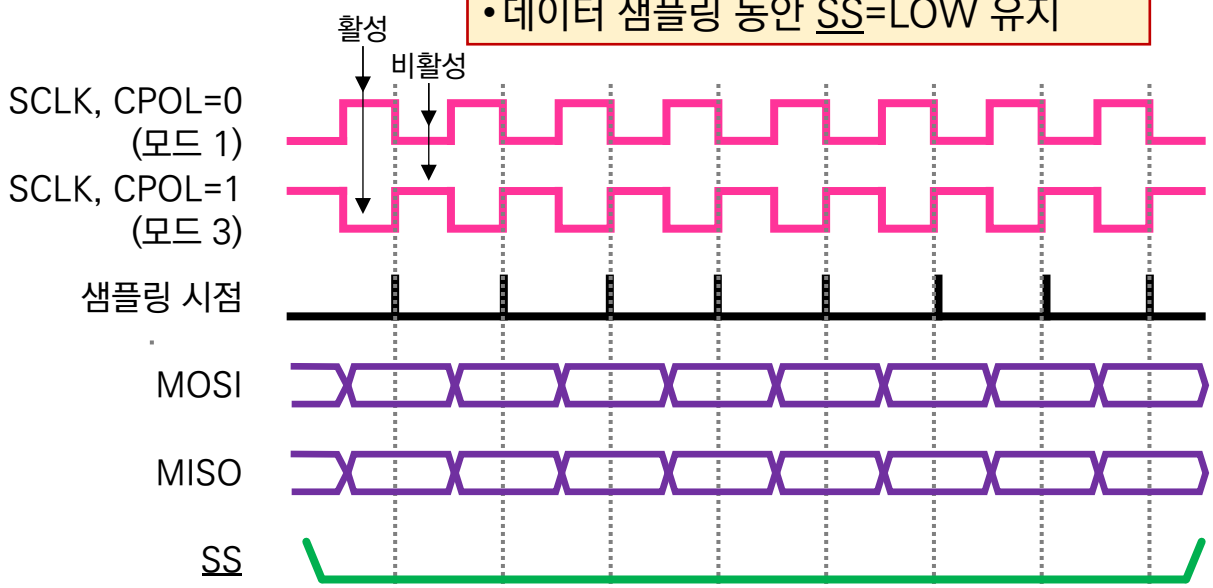
- CPHA = 0
- 비활성→활성으로 바뀌는 에지에서 샘플링

- CPOL: SPI 버스가 유휴 상태일 때 클럭 값 결정 (Clock POLarity)
- CPHA: 데이터 샘플링 시점을 결정 (Clock PHAse)

- CPHA = 1
- 활성→비활성으로 바뀌는 에지에서 샘플링




- 데이터 샘플링 동안 SS=LOW 유지

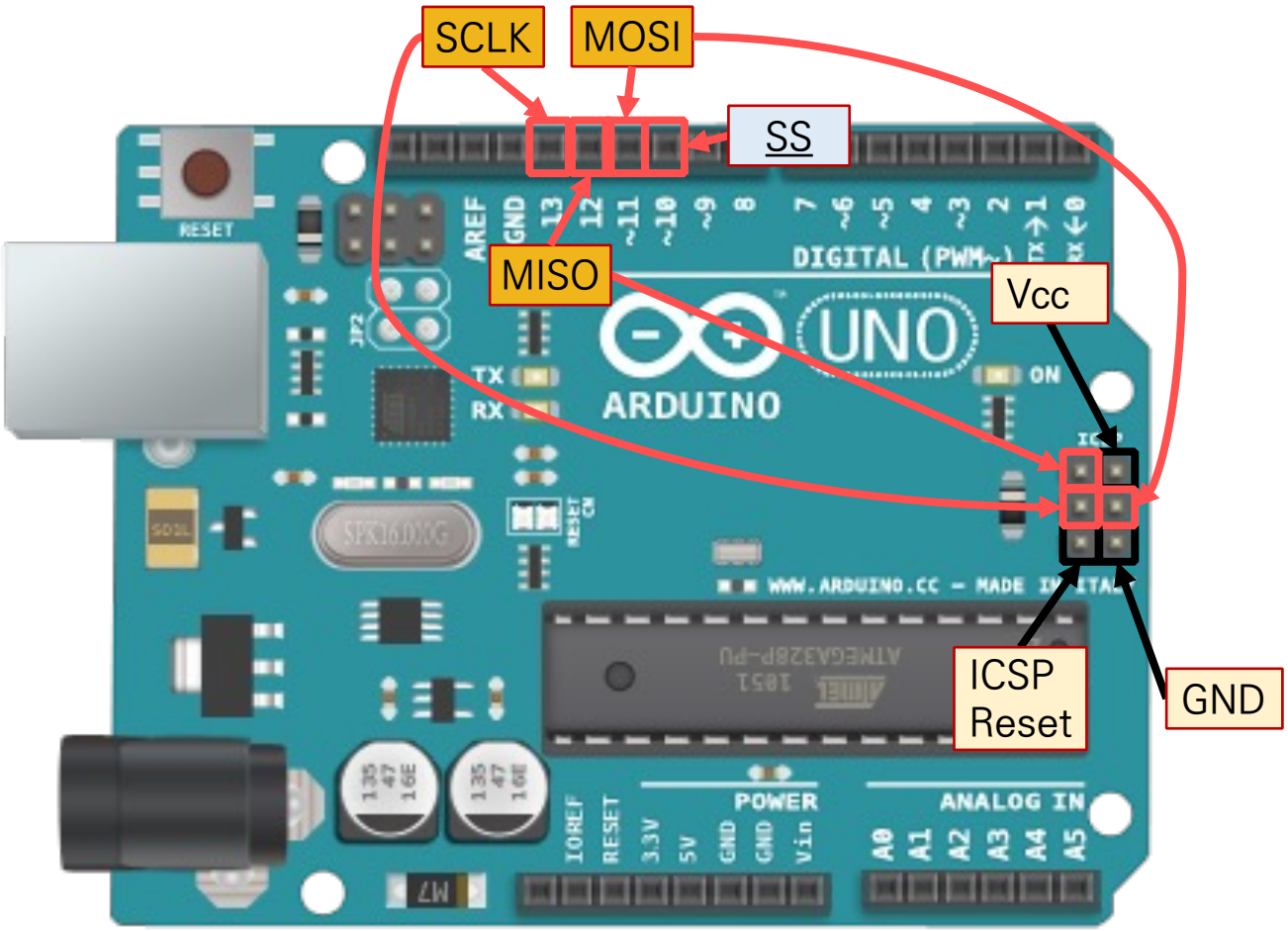


# 아두이노 SPI 연결

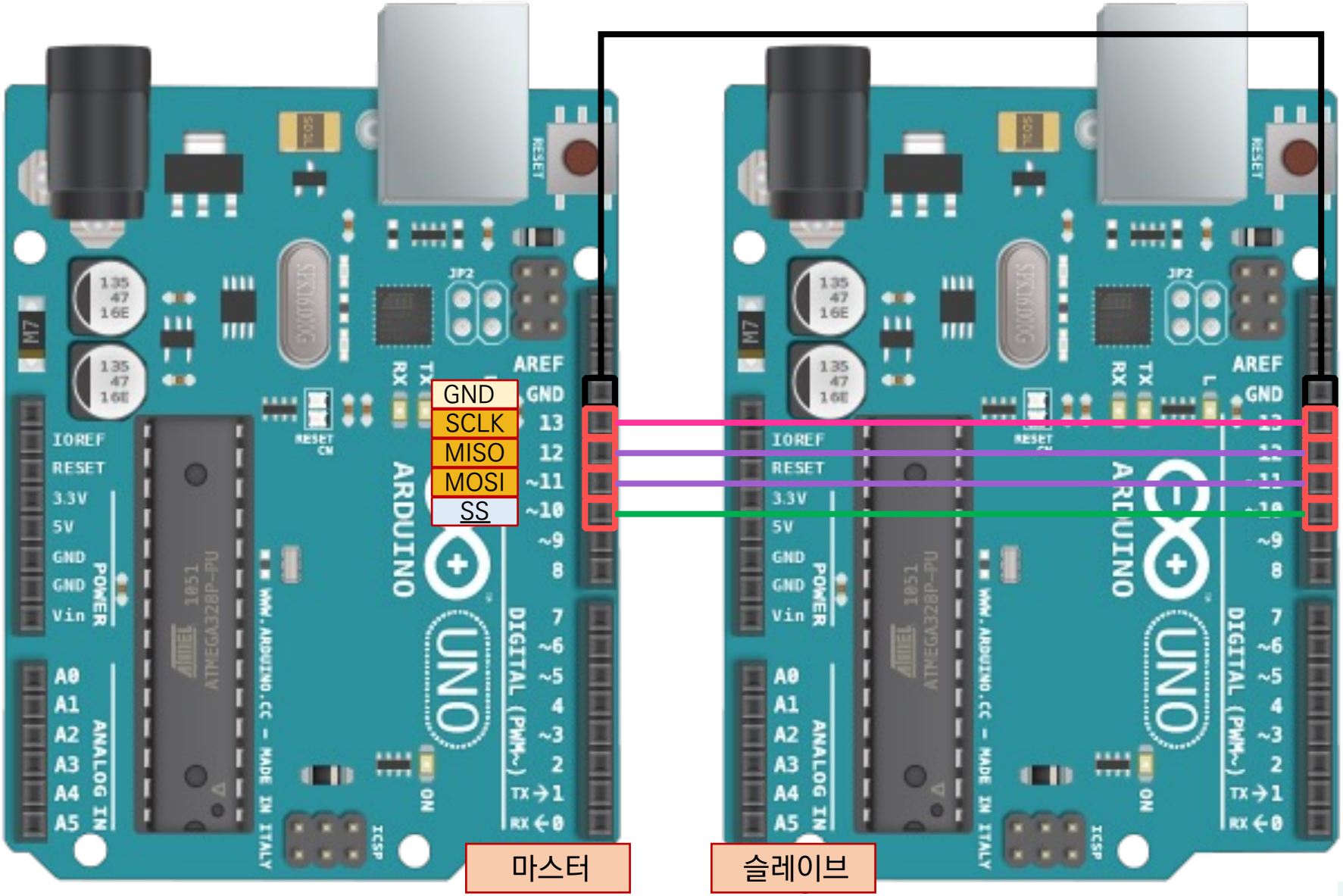
SCLK, MISO, MOSI, SS, GND

# 아두이노 우노 SPI 예약 핀 번호

- 슬레이브일 경우 SS는 예약된 10번을 사용하면 되지만 변경 가능
- 마스터일 경우 SS는 슬레이브 수만큼 필요하므로 추가 배정 필요



# 아두이노 우노 SPI 예약 핀 번호



# 아두이노 SPI 통신

SPI.h

# SPI 라이브러리[1]

## ■ SPI 통신을 위한 라이브러리 제공: SPI.h

- SPI 기본 설정 함수와 데이터 전송 함수
- SPI 오브젝트는 SPIClass에서 한 개만 생성 가능, 전역객체 SPI

## ■ 주요 멤버

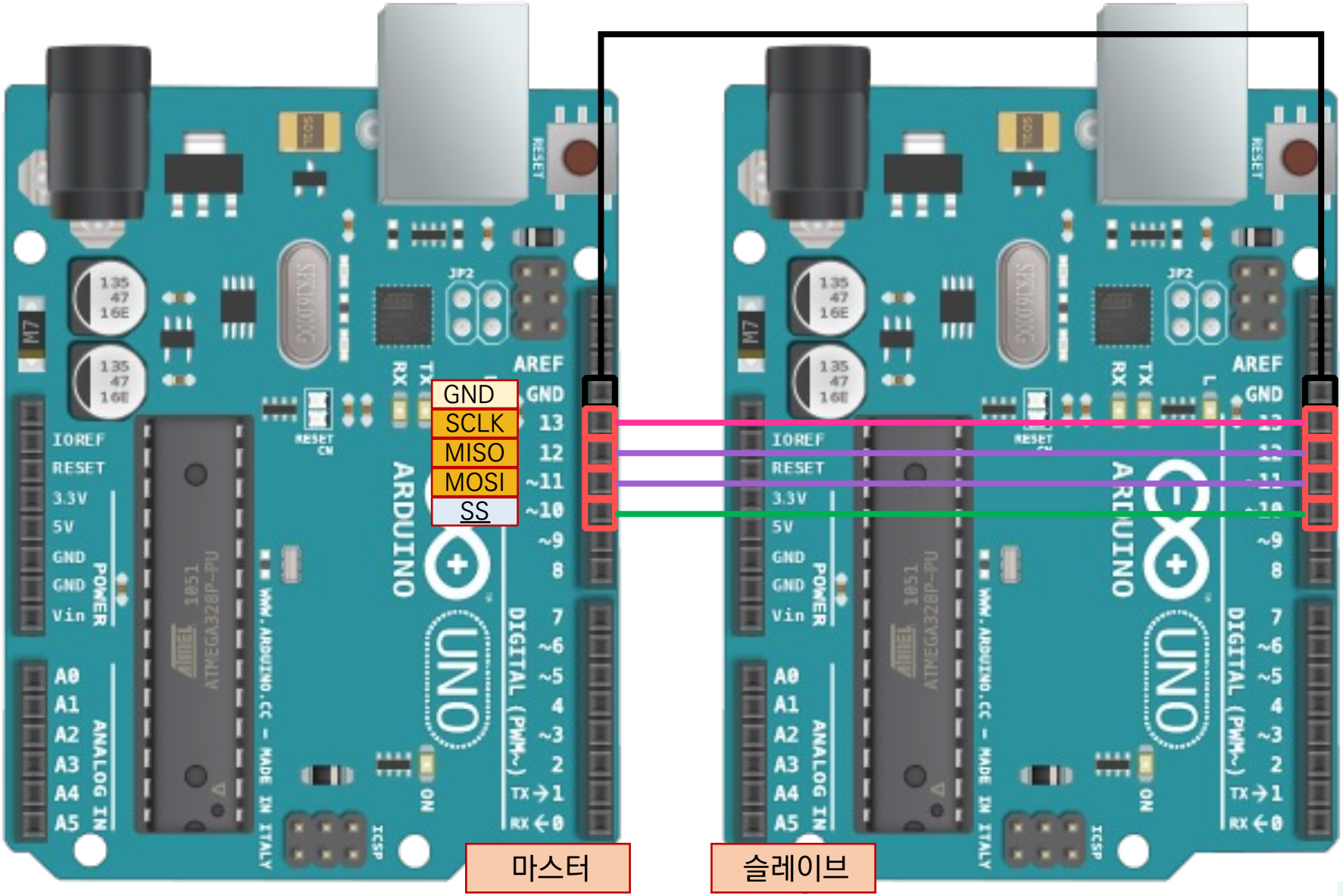
- begin(): SPI 통신 초기화
- end(): SPI 통신 종료
- setBitOrder(): 데이터 송수신 비트 순서 설정 (LSBFIRST 또는 MSBFIRST)
- setClockDivider(): 시스템 클럭에 대한 분주비 설정 (예, SPI_CLOCK_DIV4)
- setDataMode(): CPOL과 CPHA에 따른 모드 설정 (예, SPI_MODE0)
- beginTransaction(): SPISettings 오브젝트를 인수로 SPI 버스 초기화
  - ✓ SPISettings: 전송 속도, 송수신 비트 순서, 전송 모드를 갖는 설정 오브젝트
- endTransaction(): SPI 버스 사용 종료
- transfer(): (바이트) 데이터 전송

[1] https://www.arduino.cc/en/Reference/SPI

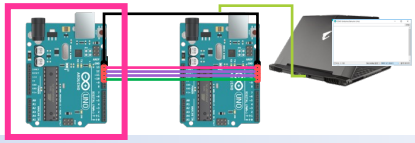
# SPI 통신 예제

1:1 단방향 문자열 송신

# SPI 통신 예제 1:1 단방향 문자열 송신 (마스터-슬레이브 연결)



# SPI 통신 예제 1:1 단방향 문자열 송신 (마스터 스케치)

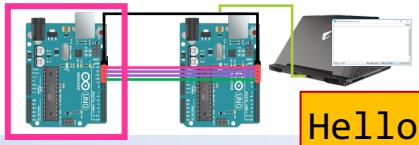


Practice

```
#include <SPI.h>

void setup() {

  SPI.begin();
  pinMode(SS, OUTPUT);
  digitalWrite(SS, HIGH);
  SPI.setClockDivider(SPI_CLOCK_DIV16);
}




void loop() {

  digitalWrite(SS, LOW);


  const byte *p = "Hello, Slave!\n";
  for (int i = 0; i < strlen(p); i++) {
    SPI.transfer(p[i]);
  }

  digitalWrite(SS, HIGH);
  delay(1000);
}
```

[1] https://www.arduino.cc/en/Reference/SPIBegin
[2] https://www.arduino.cc/en/Reference/SPISetClockDivider
[3] https://www.arduino.cc/en/Reference/SPITransfer

# SPI 통신 예제 1:1 단방향 문자열 송신 (마스터 스케치)



결과

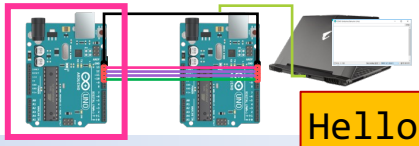
Hello, Slave!

- SPI 라이브러리 SPI.h 포함하여 전역 객체 SPI를 사용
- SPI.h는 마스터 용이며 슬레이브는 레지스터 접근 필요

```
#include <SPI.h>

void setup() {

  SPI.begin();
  pinMode(SS, OUTPUT);
  digitalWrite(SS, HIGH);
  SPI.setClockDivider(SPI_CLOCK_DIV16);
}





void loop() {

  digitalWrite(SS, LOW);


  const byte *p = "Hello, Slave!\n";
  for (int i = 0; i < strlen(p); i++) {
    SPI.transfer(p[i]);
  }

  digitalWrite(SS, HIGH);
  delay(1000);
}
```

[1] https://www.arduino.cc/en/Reference/SPIBegin
[2] https://www.arduino.cc/en/Reference/SPISetClockDivider
[3] https://www.arduino.cc/en/Reference/SPITransfer

# SPI 통신 예제 1:1 단방향 문자열 송신 (마스터 스케치)



결과

Hello, Slave!

```
#include <SPI.h>

void setup() {

  SPI.begin();
  pinMode(SS, OUTPUT);
  digitalWrite(SS, HIGH);
  SPI.setClockDivider(SPI_CLOCK_DIV16);
}
```

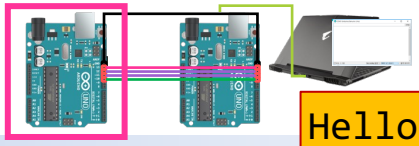
- SPI 라이브러리 SPI.h 포함하여 전역 객체 SPI를 사용
- SPI.h는 마스터 용이며 슬레이브는 레지스터 접근 필요

- begin()[1] 는 SPI 통신을 초기화
- 우노인 경우 SCLK (D13), MOSI (D12), MISO (D11) 설정
- SS는 D10으로 설정되어 있으나 변경 가능
- SS 핀을 디지털 출력으로 설정하고 'HIGH'를 출력하여 연결된 슬레이브를 '확실하게' 선택하지 않음으로 설정
- **setClockDivider()**[2] 은 시스템 클럭에 대한 분주비를 이용해 통신 속도를 설정
- 예로서, 인수가 SPI_CLOCK_DIV16 이면,
  SPI 통신 클럭 = 우노 시스템 클럭 (16 MHz) / 분주비 (16)

```
void loop() {

  digitalWrite(SS, LOW);


  const byte *p = "Hello, Slave!\n";
  for (int i = 0; i < strlen(p); i++) {
    SPI.transfer(p[i]);
  }

  digitalWrite(SS, HIGH);
  delay(1000);
}
```

[1] https://www.arduino.cc/en/Reference/SPIBegin
[2] https://www.arduino.cc/en/Reference/SPISetClockDivider
[3] https://www.arduino.cc/en/Reference/SPITransfer

# SPI 통신 예제 1:1 단방향 문자열 송신 (마스터 스케치)



결과

Hello, Slave!

```
#include <SPI.h>

void setup() {

  SPI.begin();
  pinMode(SS, OUTPUT);
  digitalWrite(SS, HIGH);
  SPI.setClockDivider(SPI_CLOCK_DIV16);
}
```

- SPI 라이브러리 SPI.h 포함하여 전역 객체 SPI를 사용
- SPI.h는 마스터 용이며 슬레이브는 레지스터 접근 필요

- begin()[1] 는 SPI 통신을 초기화
- 우노인 경우 SCLK (D13), MOSI (D12), MISO (D11) 설정
- SS는 D10으로 설정되어 있으나 변경 가능
- SS 핀을 디지털 출력으로 설정하고 'HIGH'를 출력하여 연결된 슬레이브를 '확실하게' 선택하지 않음으로 설정
- **setClockDivider()**[2] 은 시스템 클럭에 대한 분주비를 이용해 통신 속도를 설정
- 예로서, 인수가 SPI_CLOCK_DIV16 이면,
  SPI 통신 클럭 = 우노 시스템 클럭 (16 MHz) / 분주비 (16)

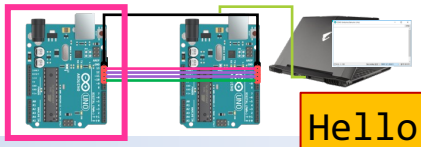
```
void loop() {

  digitalWrite(SS, LOW);


  const byte *p = "Hello, Slave!\n";
  for (int i = 0; i < strlen(p); i++) {
    SPI.transfer(p[i]);
  }

  digitalWrite(SS, HIGH);
  delay(1000);
}
```

- SS (D10)에 연결된 SPI 슬레이브를 '선택하며' 전송 준비

- SS (D10)에 연결된 SPI 슬레이브를 '놓아주며' 전송 완료

[1] https://www.arduino.cc/en/Reference/SPIBegin
[2] https://www.arduino.cc/en/Reference/SPISetClockDivider
[3] https://www.arduino.cc/en/Reference/SPITransfer

# SPI 통신 예제 1:1 단방향 문자열 송신 (마스터 스케치)



결과

Hello, Slave!

```
#include <SPI.h>

void setup() {

  SPI.begin();
  pinMode(SS, OUTPUT);
  digitalWrite(SS, HIGH);
  SPI.setClockDivider(SPI_CLOCK_DIV16);
}
```

- SPI 라이브러리 SPI.h 포함하여 전역 객체 SPI를 사용
- SPI.h는 마스터 용이며 슬레이브는 레지스터 접근 필요

- begin()[1] 는 SPI 통신을 초기화
- 우노인 경우 SCLK (D13), MOSI (D12), MISO (D11) 설정
- SS는 D10으로 설정되어 있으나 변경 가능
- SS 핀을 디지털 출력으로 설정하고 'HIGH'를 출력하여 연결된 슬레이브를 '확실하게' 선택하지 않음으로 설정
- **setClockDivider()**[2] 은 시스템 클럭에 대한 분주비를 이용해 통신 속도를 설정
- 예로서, 인수가 SPI_CLOCK_DIV16 이면, SPI 통신 클럭 = 우노 시스템 클럭 (16 MHz) / 분주비 (16)

```
void loop() {

  digitalWrite(SS, LOW);


  const byte *p = "Hello, Slave!\n";
  for (int i = 0; i < strlen(p); i++) {
    SPI.transfer(p[i]);
  }

  digitalWrite(SS, HIGH);
  delay(1000);
}
```

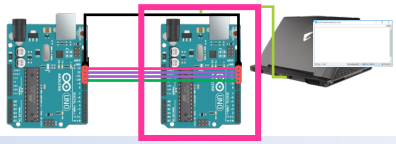
- SS (D10)에 연결된 SPI 슬레이브를 '선택하며' 전송 준비

- 문자열의 '\n'은 슬레이브에게 마지막 전송 바이트임을 알림
- transfer()[3] 는 SS 출력으로 선택된 슬레이브에게 MOSI 라인을 통해 '1 바이트 (byte)' 데이터를 전송
- 따라서 바이트보다 큰 데이터 (문자열, 큰 정수, 부동소수점 실수)를 전송하기 위해서는 바이트 배열로 재구성 필요!

- SS (D10)에 연결된 SPI 슬레이브를 '놓아주며' 전송 완료

[1] https://www.arduino.cc/en/Reference/SPIBegin
[2] https://www.arduino.cc/en/Reference/SPISetClockDivider
[3] https://www.arduino.cc/en/Reference/SPITransfer

# SPI 통신 예제 1:1 단방향 문자열 송신 (슬레이브 스케치)

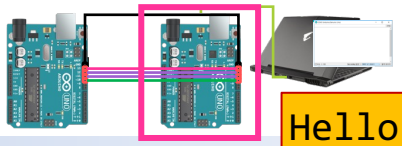


Practice

```
char buf[100];
volatile byte pos = 0;
volatile boolean done = false;

void setup() {
  Serial.begin(9600);
  SPCR |= _BV(SPE);
  SPCR |= _BV(SPIE);
  SPCR &= ~_BV(MSTR);
}


void loop() {
  if (done) {
    buf[pos] = 0;
    Serial.print(buf);
    pos = 0; done = false;
  }
}

ISR(SPI_STC_vect) {
  byte c = SPDR;
  if (pos < sizeof(buf)) {
    buf[pos++] = c;
    if (c == '\n')
      done = true;
  }
}
```

[1] https://www.arduino.cc/en/Tutorial/SPIEEPROM
[2] 인터럽트 테이블 헤더 파일 위치: Arduino\hardware\tools\avr\avr\include\avr\iom328p.h

# SPI 통신 예제 1:1 단방향 문자열 송신 (슬레이브 스케치)



결과

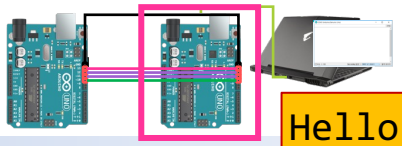
Hello, Slave!

- `volatile` 키워드는 변수를 저장 레지스터가 아닌 램에 두고 인터럽트 서비스 루틴 (ISR)에서 접근하기위한 목적

```
char buf[100];
volatile byte pos = 0;
volatile boolean done = false;

void setup() {
  Serial.begin(9600);
  SPCR |= _BV(SPE);
  SPCR |= _BV(SPIE);
  SPCR &= ~_BV(MSTR);
}


void loop() {
  if (done) {
    buf[pos] = 0;
    Serial.print(buf);
    pos = 0; done = false;
  }
}

ISR(SPI_STC_vect) {
  byte c = SPDR;
  if (pos < sizeof(buf)) {
    buf[pos++] = c;
    if (c == '\n')
      done = true;
  }
}
```

[1] https://www.arduino.cc/en/Tutorial/SPIEEPROM
[2] 인터럽트 테이블 헤더 파일 위치: `Arduino\hardware\tools\avr\avr\include\avr\iom328p.h`

# SPI 통신 예제 1:1 단방향 문자열 송신 (슬레이브 스케치)



결과

Hello, Slave!

- `volatile` 키워드는 변수를 저장 레지스터가 아닌 램에 두고 인터럽트 서비스 루틴 (ISR)에서 접근하기위한 목적

- SPI 라이브러리는 마스터 전용으로서 슬레이브는 직접 'SPCR (SPI Control Register)'를 접근하여 설정이 필요[1]
- SPE (SPI Enable)은 SPI 통신 가능 설정, SPIE (SPI Interrupt Enable)은 SPI 통신 인터럽트 가능 설정, MSTR (Master)는 마스터 (1), 슬레이브 (0, 기본) 설정이 필요
- `#define _BV(bit) (1 << (bit))` 를 활용해 SPCR 설정

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| SPIE | SPE | DORD | MSTR | CPOL | CPHA | SPR1 | SPR0 |

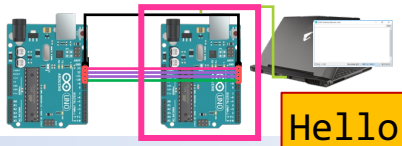
SPCR (SPI Control Register)

```
char buf[100];
volatile byte pos = 0;
volatile boolean done = false;

void setup() {
  Serial.begin(9600);
  SPCR |= _BV(SPE);
  SPCR |= _BV(SPIE);
  SPCR &= ~_BV(MSTR);
}

void loop() {
  if (done) {
    buf[pos] = 0;
    Serial.print(buf);
    pos = 0; done = false;
  }
}

ISR(SPI_STC_vect) {
  byte c = SPDR;
  if (pos < sizeof(buf)) {
    buf[pos++] = c;
    if (c == '\n')
      done = true;
  }
}
```

- SPI 가능 '1'
- SPI 인터럽트 '1'
- SPI 슬레이브 '0' (기본 0이므로 불필요)

[1] https://www.arduino.cc/en/Tutorial/SPIEEPROM
[2] 인터럽트 테이블 헤더 파일 위치: `Arduino\hardware\tools\avr\avr\include\avr\iom328p.h`

# SPI 통신 예제 1:1 단방향 문자열 송신 (슬레이브 스케치)



결과

Hello, Slave!

- `volatile` 키워드는 변수를 저장 레지스터가 아닌 램에 두고 인터럽트 서비스 루틴 (ISR)에서 접근하기위한 목적

```
char buf[100];
volatile byte pos = 0;
volatile boolean done = false;
```

- SPI 라이브러리는 마스터 전용으로서 슬레이브는 직접 'SPCR (SPI Control Register)'를 접근하여 설정이 필요[1]
- SPE (SPI Enable)은 SPI 통신 가능 설정, SPIE (SPI Interrupt Enable)은 SPI 통신 인터럽트 가능 설정, MSTR (Master)는 마스터 (1), 슬레이브 (0, 기본) 설정이 필요
- `#define _BV(bit) (1 << (bit))` 를 활용해 SPCR 설정

```
void setup() {
  Serial.begin(9600);
  SPCR |= _BV(SPE);
  SPCR |= _BV(SPIE);
  SPCR &= ~_BV(MSTR);
}
```

- SPI 가능 '1'
- SPI 인터럽트 '1'
- SPI 슬레이브 '0' (기본 0이므로 불필요)

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| SPIE | SPE | DORD | MSTR | CPOL | CPHA | SPR1 | SPR0 |

SPCR (SPI Control Register)

```
void loop() {
  if (done) {
    buf[pos] = 0;
    Serial.print(buf);
    pos = 0; done = false;
  }
}
```

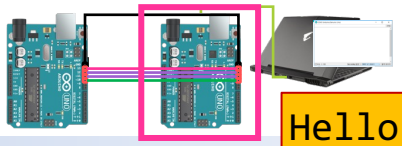
- ISR은 내부 인터럽트가 발생할 때 불리우는 함수
- 인수로 내부 인터럽트 테이블로 예약된 상수를 사용[2]
- `SPI_STC_vect` 는 SPI 통신이 끝났을 때 발생하는 인터럽트
- 'SPDR (SPI Data Register)'은 슬레이브에서 송신하거나 수신하는 바이트 데이터를 저장하는 레지스터
- 주의: `ISR()`에서 `delay()`, `Serial.print()` 등 실행에 '긴 시간이 필요한 함수'를 절대 사용하지 말 것! 최대한 짧게!

```
ISR(SPI_STC_vect) {
  byte c = SPDR;
  if (pos < sizeof(buf)) {
    buf[pos++] = c;
    if (c == '\n')
      done = true;
  }
}
```

- 수신 데이터를 c에 저장
- 수신 데이터가 '\n'이면 마스터가 데이터 전송 끝임을 알리는 것

[1] https://www.arduino.cc/en/Tutorial/SPIEEPROM
[2] 인터럽트 테이블 헤더 파일 위치: `Arduino\hardware\tools\avr\avr\include\avr\iom328p.h`

# SPI 통신 예제 1:1 단방향 문자열 송신 (슬레이브 스케치)



결과

Hello, Slave!

- `volatile` 키워드는 변수를 저장 레지스터가 아닌 램에 두고 인터럽트 서비스 루틴 (ISR)에서 접근하기위한 목적

```
char buf[100];
volatile byte pos = 0;
volatile boolean done = false;
```

- SPI 라이브러리는 마스터 전용으로서 슬레이브는 직접 'SPCR (SPI Control Register)'를 접근하여 설정이 필요[1]
- SPE (SPI Enable)은 SPI 통신 가능 설정, SPIE (SPI Interrupt Enable)은 SPI 통신 인터럽트 가능 설정, MSTR (Master)는 마스터 (1), 슬레이브 (0, 기본) 설정이 필요
- `#define _BV(bit) (1 << (bit))` 를 활용해 SPCR 설정

```
void setup() {
  Serial.begin(9600);
  SPCR |= _BV(SPE);
  SPCR |= _BV(SPIE);
  SPCR &= ~_BV(MSTR);
}
```

- SPI 가능 '1'
- SPI 인터럽트 '1'
- SPI 슬레이브 '0' (기본 0이므로 불필요)

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| SPIE | SPE | DORD | MSTR | CPOL | CPHA | SPR1 | SPR0 |

SPCR (SPI Control Register)

- done이 'true' 이면 (마스터로부터 데이터 전송 끝이면), buf 배열 마지막에 '\0'을 추가하고 시리얼 통신으로 출력
- 다음 데이터를 받기 위해 pos와 done을 초기화

```
void loop() {
  if (done) {
    buf[pos] = 0;
    Serial.print(buf);
    pos = 0; done = false;
  }
}
```

- 마스터가 데이터 전송을 마치면 시리얼 통신 출력

- ISR은 내부 인터럽트가 발생할 때 불리우는 함수
- 인수로 내부 인터럽트 테이블로 예약된 상수를 사용[2]
- `SPI_STC_vect` 는 SPI 통신이 끝났을 때 발생하는 인터럽트
- 'SPDR (SPI Data Register)'은 슬레이브에서 송신하거나 수신하는 바이트 데이터를 저장하는 레지스터
- 주의: `ISR()`에서 `delay()`, `Serial.print()` 등 실행에 '긴 시간이 필요한 함수'를 절대 사용하지 말 것! 최대한 짧게!

```
ISR(SPI_STC_vect) {
  byte c = SPDR;
  if (pos < sizeof(buf)) {
    buf[pos++] = c;
    if (c == '\n')
      done = true;
  }
}
```

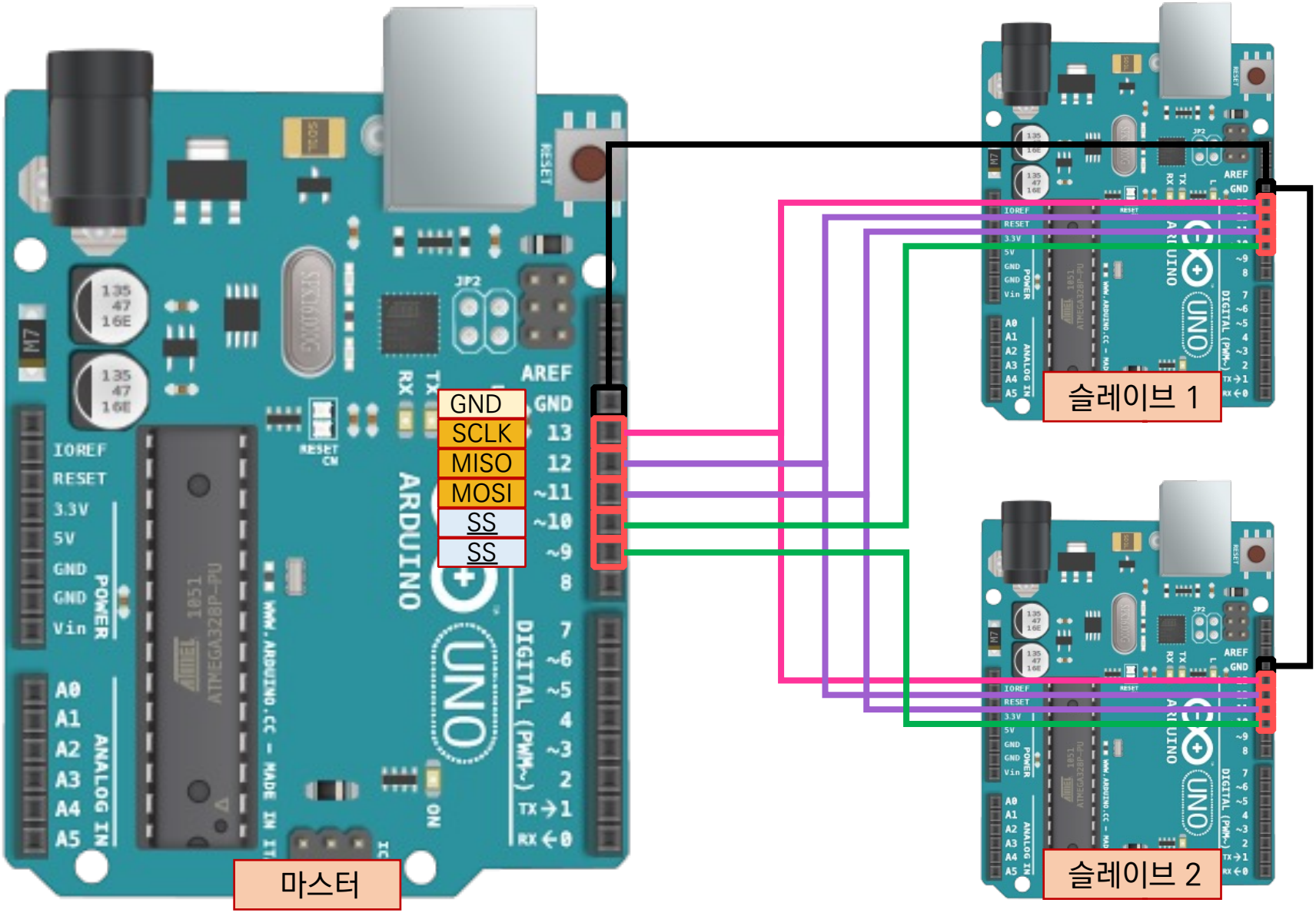
- 수신 데이터를 c에 저장
- 수신 데이터가 '\n'이면 마스터가 데이터 전송 끝임을 알리는 것

[1] https://www.arduino.cc/en/Tutorial/SPIEEPROM
[2] 인터럽트 테이블 헤더 파일 위치: `Arduino\hardware\tools\avr\avr\include\avr\iom328p.h`

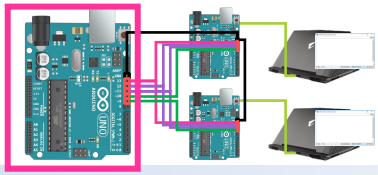
# SPI 통신 예제

1:2 단방향 문자열 송신

# SPI 통신 예제 1:2 단방향 문자열 송신 (마스터-슬레이브 연결)



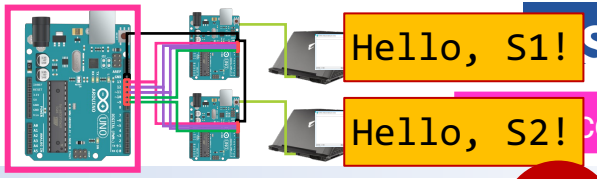
# SPI 통신 예제 1:2 단방향 문자열 송신 (마스터 스케치)



Practice

```
#include <SPI.h>
const int S1 = 10;
const int S2 = 9;

void setup() {
  SPI.begin();
  pinMode(S1, OUTPUT);
  pinMode(S2, OUTPUT);
  digitalWrite(S1, HIGH);
  digitalWrite(S2, HIGH);
  SPI.setClockDivider(SPI_CLOCK_DIV16);
}

void loop() {
  digitalWrite(S1, LOW);
  const byte *p1 = "Hello, S1!\n";
  for (int i = 0; i < strlen(p1); i++)
    SPI.transfer(p1[i]);
  digitalWrite(S1, HIGH);

  digitalWrite(S2, LOW);
  const byte *p2 = "Hello, S2!\n";
  for (int i = 0; i < strlen(p2); i++)
    SPI.transfer(p2[i]);
  digitalWrite(S2, HIGH);

  delay(1000);
}
```

# SPI 통신 예제 1:2 단방향 문자열 송신 (마스터 스케치)

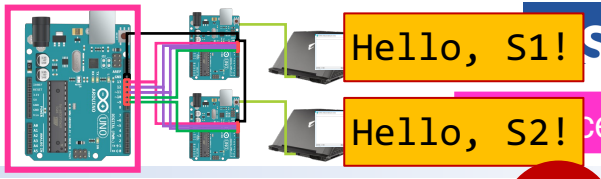



```
#include <SPI.h>
const int S1 = 10;
const int S2 = 9;

void setup() {
  SPI.begin();
  pinMode(S1, OUTPUT);
  pinMode(S2, OUTPUT);
  digitalWrite(S1, HIGH);
  digitalWrite(S2, HIGH);
  SPI.setClockDivider(SPI_CLOCK_DIV16);
}

void loop() {
  digitalWrite(S1, LOW);
  const byte *p1 = "Hello, S1!\n";
  for (int i = 0; i < strlen(p1); i++)
    SPI.transfer(p1[i]);
  digitalWrite(S1, HIGH);

  digitalWrite(S2, LOW);
  const byte *p2 = "Hello, S2!\n";
  for (int i = 0; i < strlen(p2); i++)
    SPI.transfer(p2[i]);
  digitalWrite(S2, HIGH);

  delay(1000);
}
```

- 슬레이브 1을 위한 SS 핀 번호를 D10으로 설정
- 슬레이브 2를 위한 SS 핀 번호를 D9로 설정

결과

# SPI 통신 예제 1:2 단방향 문자열 송신 (마스터 스케치)



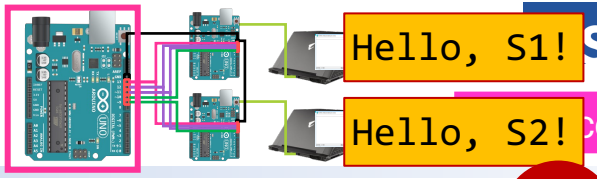

```
#include <SPI.h>
const int S1 = 10;
const int S2 = 9;

void setup() {
  SPI.begin();
  pinMode(S1, OUTPUT);
  pinMode(S2, OUTPUT);
  digitalWrite(S1, HIGH);
  digitalWrite(S2, HIGH);
  SPI.setClockDivider(SPI_CLOCK_DIV16);
}

void loop() {
  digitalWrite(S1, LOW);
  const byte *p1 = "Hello, S1!\n";
  for (int i = 0; i < strlen(p1); i++)
    SPI.transfer(p1[i]);
  digitalWrite(S1, HIGH);

  digitalWrite(S2, LOW);
  const byte *p2 = "Hello, S2!\n";
  for (int i = 0; i < strlen(p2); i++)
    SPI.transfer(p2[i]);
  digitalWrite(S2, HIGH);

  delay(1000);
}
```

결과

- 슬레이브 1을 위한 SS 핀 번호를 D10으로 설정
- 슬레이브 2를 위한 SS 핀 번호를 D9로 설정

- S1, S2핀을 출력으로 설정하고 연결된 슬레이브를 확실하게 선택하지 않음으로 설정
- 시스템 클럭에 대한 분주비 (SPI_CLOCK_DIV16)를 이용해 SPI 통신 속도 1 MHz로 설정

# SPI 통신 예제 1:2 단방향 문자열 송신 (마스터 스케치)




결과

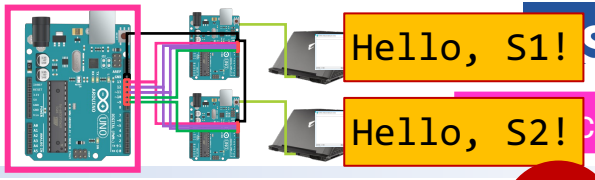
```
#include <SPI.h>
const int S1 = 10;
const int S2 = 9;

void setup() {
  SPI.begin();
  pinMode(S1, OUTPUT);
  pinMode(S2, OUTPUT);
  digitalWrite(S1, HIGH);
  digitalWrite(S2, HIGH);
  SPI.setClockDivider(SPI_CLOCK_DIV16);
}

void loop() {
  digitalWrite(S1, LOW);
  const byte *p1 = "Hello, S1!\n";
  for (int i = 0; i < strlen(p1); i++)
    SPI.transfer(p1[i]);
  digitalWrite(S1, HIGH);

  digitalWrite(S2, LOW);
  const byte *p2 = "Hello, S2!\n";
  for (int i = 0; i < strlen(p2); i++)
    SPI.transfer(p2[i]);
  digitalWrite(S2, HIGH);

  delay(1000);
}
```

- 슬레이브 1을 위한 SS 핀 번호를 D10으로 설정
- 슬레이브 2를 위한 SS 핀 번호를 D9로 설정

- S1, S2핀을 출력으로 설정하고 연결된 슬레이브를 확실하게 선택하지 않음으로 설정
- 시스템 클럭에 대한 분주비 (SPI_CLOCK_DIV16)를 이용해 SPI 통신 속도 1 MHz로 설정

- S1 (D10)에 연결된 SPI 슬레이브를 '선택하며' 전송 준비
- 슬레이브 1에게 SPI 통신으로 "Hello, S1!" 문자열 전송
- S1 (D10)에 연결된 SPI 슬레이브를 '놓아주며' 전송 완료

# SPI 통신 예제 1:2 단방향 문자열 송신 (마스터 스케치)




결과

```
#include <SPI.h>
const int S1 = 10;
const int S2 = 9;

void setup() {
  SPI.begin();
  pinMode(S1, OUTPUT);
  pinMode(S2, OUTPUT);
  digitalWrite(S1, HIGH);
  digitalWrite(S2, HIGH);
  SPI.setClockDivider(SPI_CLOCK_DIV16);
}

void loop() {
  digitalWrite(S1, LOW);
  const byte *p1 = "Hello, S1!\n";
  for (int i = 0; i < strlen(p1); i++)
    SPI.transfer(p1[i]);
  digitalWrite(S1, HIGH);

  digitalWrite(S2, LOW);
  const byte *p2 = "Hello, S2!\n";
  for (int i = 0; i < strlen(p2); i++)
    SPI.transfer(p2[i]);
  digitalWrite(S2, HIGH);

  delay(1000);
}
```

- 슬레이브 1을 위한 SS 핀 번호를 D10으로 설정
- 슬레이브 2를 위한 SS 핀 번호를 D9로 설정

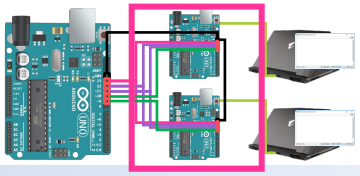
- S1, S2핀을 출력으로 설정하고 연결된 슬레이브를 확실하게 선택하지 않음으로 설정
- 시스템 클럭에 대한 분주비 (SPI_CLOCK_DIV16)를 이용해 SPI 통신 속도 1 MHz로 설정

- S1 (D10)에 연결된 SPI 슬레이브를 '선택하며' 전송 준비
- 슬레이브 1에게 SPI 통신으로 "Hello, S1!" 문자열 전송
- S1 (D10)에 연결된 SPI 슬레이브를 '놓아주며' 전송 완료

- S2 (D9)에 연결된 SPI 슬레이브를 '선택하며' 전송 준비
- 슬레이브 2에게 SPI 통신으로 "Hello, S2!" 문자열 전송
- S2 (D9)에 연결된 SPI 슬레이브를 '놓아주며' 전송 완료

- 슬레이브가 더 있더라도 같은 방식으로 해당 슬레이브를 위한 SS 을 설정하고 슬레이브 선택 후 데이터를 전송

# SPI 통신 예제 1:2 단방향 문자열 송신 (슬레이브 스케치)



Practice

- volatile 키워드는 변수를 저장 레지스터가 아닌 램에 두고 인터럽트 서비스 루틴 (ISR)에서 접근하기위한 목적

```
char buf[100];
volatile byte pos = 0;
volatile boolean done = false;
```

- SPI 라이브러리는 마스터 전용으로서 슬레이브는 직접 'SPCR (SPI Control Register)'를 접근하여 설정이 필요[1]
- SPE (SPI Enable)은 SPI 통신 가능 설정, SPIE (SPI Interrupt Enable)은 SPI 통신 인터럽트 가능 설정, MSTR (Master)는 마스터 (1), 슬레이브 (0, 기본) 설정이 필요
- #define _BV(bit) (1 << (bit)) 를 활용해 SPCR 설정

```
void setup() {
  Serial.begin(9600);
  SPCR |= _BV(SPE);
  SPCR |= _BV(SPIE);
  SPCR &= ~_BV(MSTR);
}
```

- SPI 가능 '1'
- SPI 인터럽트 '1'
- SPI 슬레이브 '0' (기본 0이므로 불필요)

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| SPIE | SPE | DORD | MSTR | CPOL | CPHA | SPR1 | SPR0 |

SPCR (SPI Control Register)

- done이 'true' 이면 (마스터로부터 데이터 전송 끝이면), buf 배열 마지막에 '\0'을 추가하고 시리얼 통신으로 출력
- 다음 데이터를 받기 위해 pos와 done을 초기화

```
void loop() {
  if (done) {
    buf[pos] = 0;
    Serial.print(buf);
    pos = 0; done = false;
  }
}
```

- 마스터가 데이터 전송을 마치면 시리얼 통신 출력

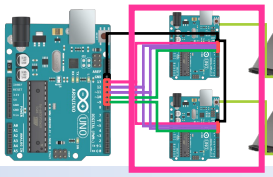
- ISR은 내부 인터럽트가 발생할 때 불리우는 함수
- 인수로 내부 인터럽트 테이블로 예약된 상수를 사용[2]
- SPI_STC_vect 는 SPI 통신이 끝났을 때 발생하는 인터럽트
- 'SPDR (SPI Data Register)'은 슬레이브에서 송신하거나 수신하는 바이트 데이터를 저장하는 레지스터
- 주의: ISR()에서 delay(), Serial.print() 등 실행에 '긴 시간이 필요한 함수'를 절대 사용하지 말 것! 최대한 짧게!

```
ISR(SPI_STC_vect) {
  byte c = SPDR;
  if (pos < sizeof(buf)) {
    buf[pos++] = c;
    if (c == '\n')
      done = true;
  }
}
```

- 수신 데이터를 c에 저장
- 수신 데이터가 '\n'이면 마스터가 데이터 전송 끝임을 알리는 것

[1] https://www.arduino.cc/en/Tutorial/SPIEEPROM
[2] 인터럽트 테이블 헤더 파일 위치: Arduino\hardware\tools\avr\avr\include\avr\io328p.h

# SPI 통신 예제 1:2 단방향 문자열 송신 (슬레이브 스케치)




결과

- volatile 키워드는 변수를 저장 레지스터가 아닌 램에 두고 인터럽트 서비스 루틴 (ISR)에서 접근하기위한 목적

```
char buf[100];
volatile byte pos = 0;
volatile boolean done = false;
```

- SPI 라이브러리는 마스터 전용으로서 슬레이브는 직접 'SPCR (SPI Control Register)'를 접근하여 설정이 필요[1]
- SPE (SPI Enable)은 SPI 통신 가능 설정, SPIE (SPI Interrupt Enable)은 SPI 통신 인터럽트 가능 설정, MSTR (Master)는 마스터 (1), 슬레이브 (0, 기본) 설정이 필요
- #define _BV(bit) (1 << (bit)) 를 활용해 SPCR 설정

```
void setup() {
  Serial.begin(9600);
  SPCR |= _BV(SPE);
  SPCR |= _BV(SPIE);
  SPCR &= ~_BV(MSTR);
}
```

- SPI 가능 '1'
- SPI 인터럽트 '1'
- SPI 슬레이브 '0' (기본 0이므로 불필요)

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| SPIE | SPE | DORD | MSTR | CPOL | CPHA | SPR1 | SPR0 |

SPCR (SPI Control Register)

- done이 'true' 이면 (마스터로부터 데이터 전송 끝이면), buf 배열 마지막에 '\0'을 추가하고 시리얼 통신으로 출력
- 다음 데이터를 받기 위해 pos와 done을 초기화

```
void loop() {
  if (done) {
    buf[pos] = 0;
    Serial.print(buf);
    pos = 0; done = false;
  }
}
```

- 마스터가 데이터 전송을 마치면 시리얼 통신 출력

- ISR은 내부 인터럽트가 발생할 때 불리우는 함수
- 인수로 내부 인터럽트 테이블로 예약된 상수를 사용[2]
- SPI_STC_vect 는 SPI 통신이 끝났을 때 발생하는 인터럽트
- 'SPDR (SPI Data Register)'은 슬레이브에서 송신하거나 수신하는 바이트 데이터를 저장하는 레지스터
- 주의: ISR()에서 delay(), Serial.print() 등 실행에 '긴 시간이 필요한 함수'를 절대 사용하지 말 것! 최대한 짧게!

```
ISR(SPI_STC_vect) {
  byte c = SPDR;
  if (pos < sizeof(buf)) {
    buf[pos++] = c;
    if (c == '\n')
      done = true;
  }
}
```

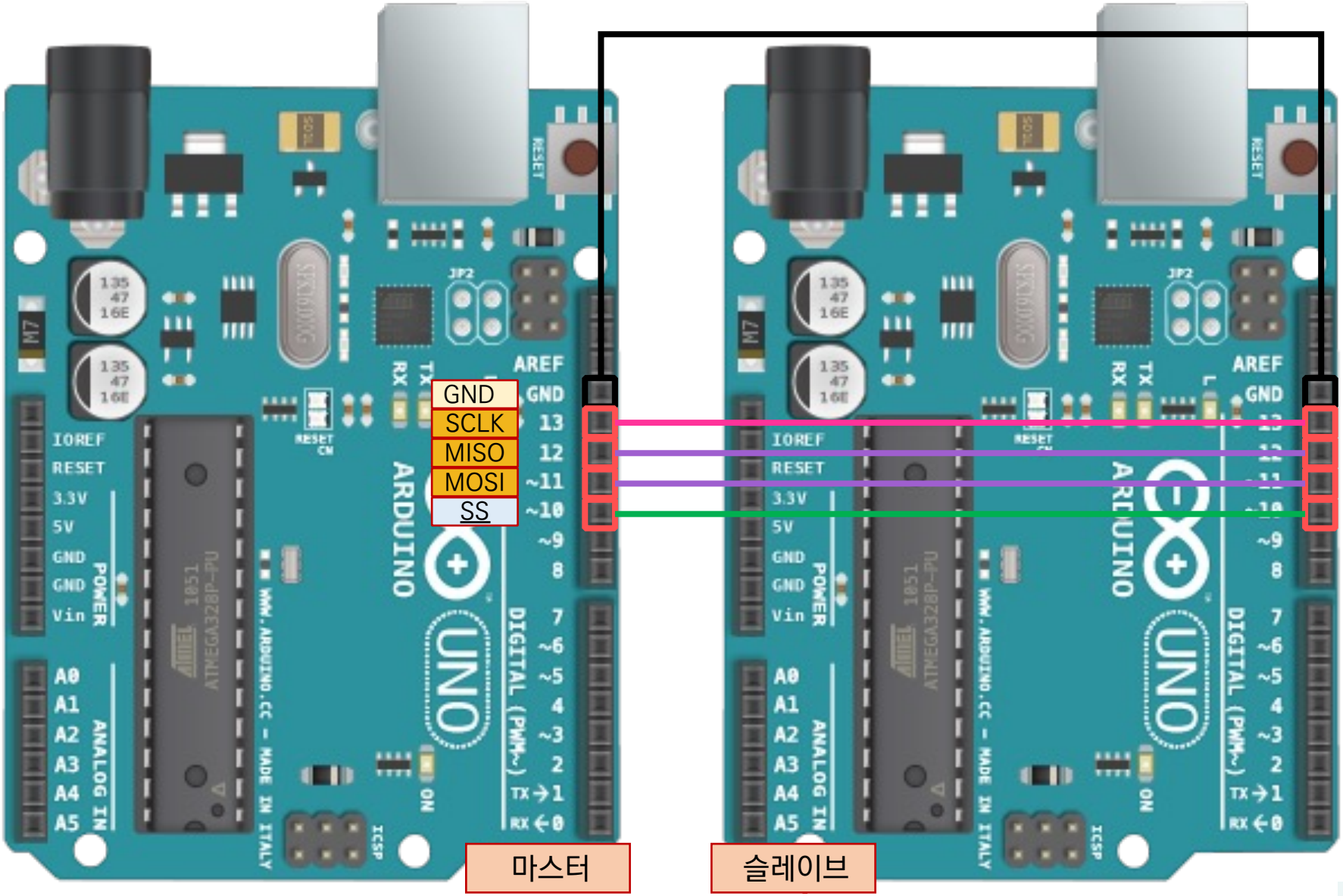
- 수신 데이터를 c에 저장
- 수신 데이터가 '\n'이면 마스터가 데이터 전송 끝임을 알리는 것

[1] https://www.arduino.cc/en/Tutorial/SPIEEPROM
[2] 인터럽트 테이블 헤더 파일 위치: Arduino\hardware\tools\avr\avr\include\avr\io328p.h

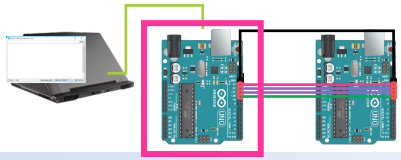
# SPI 통신 예제

1:1 바이트 데이터 요청 (센싱)

# SPI 통신 예제 1:1 바이트 데이터 교환 (마스터-슬레이브 연결)



# SPI 통신 예제 1:1 바이트 데이터 교환 (마스터 스케치)



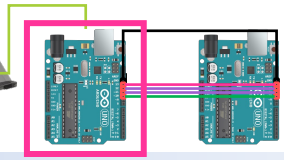
Practice

```
#include <SPI.h>

void setup() {
  Serial.begin(9600);
  pinMode(SS, OUTPUT);
  digitalWrite(SS, HIGH);
  SPI.begin();
  SPI.setClockDivider(SPI_CLOCK_DIV16);
}

byte transferAndWait(const byte what){
  byte a = SPI.transfer(what);
  delayMicroseconds(20);
  return a;
}

void loop() {
  byte s;
  digitalWrite(SS, LOW);
  transferAndWait('s');
  transferAndWait(0);
  s = transferAndWait(0);
  digitalWrite(SS, HIGH);

  Serial.println ("Sensed data:");
  Serial.println (s, DEC);
  delay (1000);
}
```

# SPI 통신 예제 1:1 바이트 데이터 교환 (마스터 스케치)

결과

34
2
72



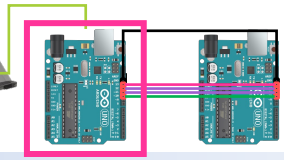
Practice

```
#include <SPI.h>

void setup() {
  Serial.begin(9600);
  pinMode(SS, OUTPUT);
  digitalWrite(SS, HIGH);
  SPI.begin();
  SPI.setClockDivider(SPI_CLOCK_DIV16);
}

byte transferAndWait(const byte what){
  byte a = SPI.transfer(what);
  delayMicroseconds(20);
  return a;
}

void loop() {
  byte s;
  digitalWrite(SS, LOW);

  transferAndWait('s');
  transferAndWait(0);
  s = transferAndWait(0);

  digitalWrite(SS, HIGH);

  Serial.println ("Sensed data:");
  Serial.println (s, DEC);
  delay (1000);
}
```

- 시리얼 통신 초기화
- SS 핀을 디지털 출력으로 설정하고 'HIGH'를 출력하여 연결된 슬레이브를 '확실하게' 선택하지 않음으로 설정시스템 클럭에 대한 분주비 (SPI_CLOCK_DIV16)를 이용해 SPI 통신 속도 1 MHz로 설정

# SPI 통신 예제 1:1 바이트 데이터 교환 (마스터 스케치)

결과
34
2
72



Practice

```
#include <SPI.h>

void setup() {
  Serial.begin(9600);
  pinMode(SS, OUTPUT);
  digitalWrite(SS, HIGH);
  SPI.begin();
  SPI.setClockDivider(SPI_CLOCK_DIV16);
}

byte transferAndWait(const byte what){
  byte a = SPI.transfer(what);
  delayMicroseconds(20);
  return a;
}

void loop() {
  byte s;
  digitalWrite(SS, LOW);

  transferAndWait('s');
  transferAndWait(0);
  s = transferAndWait(0);

  digitalWrite(SS, HIGH);

  Serial.println ("Sensed data:");
  Serial.println (s, DEC);
  delay (1000);
}
```

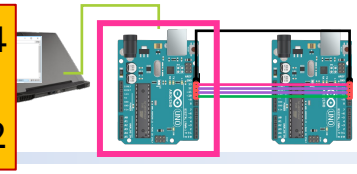
- 시리얼 통신 초기화
- SS 핀을 디지털 출력으로 설정하고 'HIGH'를 출력하여 연결된 슬레이브를 '확실하게' 선택하지 않음으로 설정시스템 클럭에 대한 분주비 (SPI_CLOCK_DIV16)를 이용해 SPI 통신 속도 1 MHz로 설정

- SPI 구조 때문에 마스터가 데이터 요청 명령을 보냄과 동시에 슬레이브가 대응할 수 없음 (원형 큐임을 기억!)
- 마스터의 요청에 대한 대응은 다음에 '아무거나 바이트'를 보냄과 동시에 슬레이브가 보내오는 데이터
- 이 때 슬레이브가 자기 데이터 버퍼 안에 대응 값을 넣어 놓을 충분한 시간을 주기 위해 transferAndWait() 활용

- 바이트 전송 후 20$\mu$s 동안 기다려 줌

# SPI 통신 예제 1:1 바이트 데이터 교환 (마스터 스케치)

결과
34
2
72



Practice

```
#include <SPI.h>

void setup() {
  Serial.begin(9600);
  pinMode(SS, OUTPUT);
  digitalWrite(SS, HIGH);
  SPI.begin();
  SPI.setClockDivider(SPI_CLOCK_DIV16);
}

byte transferAndWait(const byte what){
  byte a = SPI.transfer(what);
  delayMicroseconds(20);
  return a;
}

void loop() {
  byte s;
  digitalWrite(SS, LOW);

  transferAndWait('s');
  transferAndWait(0);
  s = transferAndWait(0);

  digitalWrite(SS, HIGH);

  Serial.println ("Sensed data:");
  Serial.println (s, DEC);
  delay (1000);
}
```

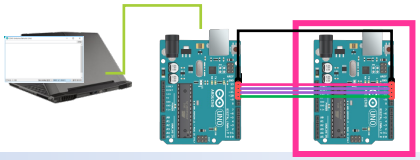
- 바이트 전송 후 20$\mu$s 동안 기다려 줌

- 시리얼 통신 초기화
- SS 핀을 디지털 출력으로 설정하고 'HIGH'를 출력하여 연결된 슬레이브를 '확실하게' 선택하지 않음으로 설정시스템 클럭에 대한 분주비 (SPI_CLOCK_DIV16)를 이용해 SPI 통신 속도 1 MHz로 설정

- SPI 구조 때문에 마스터가 데이터 요청 명령을 보냄과 동시에 슬레이브가 대응할 수 없음 (원형 큐임을 기억!)
- 마스터의 요청에 대한 대응은 다음에 '아무거나 바이트'를 보냄과 동시에 슬레이브가 보내오는 데이터
- 이 때 슬레이브가 자기 데이터 버퍼 안에 대응 값을 넣어 놓을 충분한 시간을 주기 위해 transferAndWait() 활용

- 세 번에 걸쳐서 슬레이브에게 요구한 데이터를 받는 과정
- 첫 번째 transferAndWait('s')는 슬레이브에게 's' 를 전송하여 대응 (예, 센서 값)을 요구, 동시에 슬레이브가 보내온 데이터는 '아무거나' 데이터
- 두 번째 transferAndWait(0)은 '아무거나' 데이터를 보내어 슬레이브가 대응 데이터를 버퍼에 넣을 기회 제공, 동시에 슬레이브가 보내온 데이터도 '아무거나' 데이터
- 세 번째 transferAndWait(0)은 '아무거나' 데이터를 보내어 슬레이브가 자신의 버퍼에 넣어 놓은 대응 데이터 (즉, 센서 값)를 전송할 기회 제공, 데이터를 획득하여 저장
- 둘 이상 명령어 (예, 's', 'a' 등) 때문에 세 번에 걸쳐서 데이터를 받는 과정이며 명령어가 없다면 두 번에 가능

# SPI 통신 예제 1:1 단방향 문자열 송신 (슬레이브 스케치)

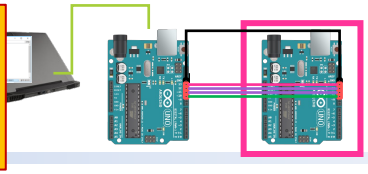


Practice

```
volatile byte command = 0;
volatile byte data = 0;

void setup () {
  pinMode(MISO, OUTPUT);
  SPCR |= _BV(SPE);
  SPCR &= ~_BV(MSTR);
  SPCR |= _BV(SPIE);
}

ISR(SPI_STC_vect) {
  byte c = SPDR;
  switch (command) {
  case 0:
    command = c;
    SPDR = 0;
    break;

  case 's':
    SPDR = data;
    break;

  }
}


void loop() {
  data = (byte)random(0, 100);
  if (digitalRead(SS) == HIGH)
    command = 0;
}
```

# SPI 통신 예제 1:1 단방향 문자열 송신 (슬레이브 스케치)

결과
34
2
72



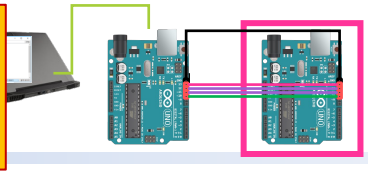
Practice

- command는 마스터가 보내오는 명령을 저장하는 변수
- data는 마스터에게 보낼 데이터를 저장하는 변수

```
volatile byte command = 0;
volatile byte data = 0;

void setup () {
  pinMode(MISO, OUTPUT);
  SPCR |= _BV(SPE);
  SPCR &= ~_BV(MSTR);
  SPCR |= _BV(SPIE);
}

ISR(SPI_STC_vect) {
  byte c = SPDR;
  switch (command) {
  case 0:
    command = c;
    SPDR = 0;
    break;

  case 's':
    SPDR = data;
    break;

  }
}


void loop() {
  data = (byte)random(0, 100);
  if (digitalRead(SS) == HIGH)
    command = 0;
}
```

# SPI 통신 예제 1:1 단방향 문자열 송신 (슬레이브 스케치)

결과 34 2 72



Practice

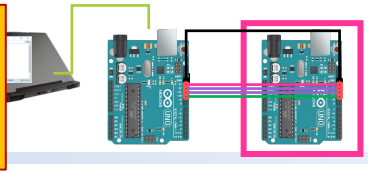
- command는 마스터가 보내오는 명령을 저장하는 변수
- data는 마스터에게 보낼 데이터를 저장하는 변수

- 이 예제에서는 슬레이브가 마스터에게 데이터를 전송하므로 MISO 핀을 출력으로 설정
- SPI 가능, SPI 인터럽트 가능, 슬레이브 레지스터 설정

```
volatile byte command = 0;
volatile byte data = 0;

void setup () {
  pinMode(MISO, OUTPUT);
  SPCR |= _BV(SPE);
  SPCR &= ~_BV(MSTR);
  SPCR |= _BV(SPIE);
}

ISR(SPI_STC_vect) {
  byte c = SPDR;
  switch (command) {
  case 0:
    command = c;
    SPDR = 0;
    break;

  case 's':
    SPDR = data;
    break;

  }
}


void loop() {
  data = (byte)random(0, 100);
  if (digitalRead(SS) == HIGH)
    command = 0;
}
```

# SPI 통신 예제 1:1 단방향 문자열 송신 (슬레이브 스케치)

결과 34 2 72



Practice

- command는 마스터가 보내오는 명령을 저장하는 변수
- data는 마스터에게 보낼 데이터를 저장하는 변수

- 이 예제에서는 슬레이브가 마스터에게 데이터를 전송하므로 MISO 핀을 출력으로 설정
- SPI 가능, SPI 인터럽트 가능, 슬레이브 레지스터 설정

- 마스터로 부터 명령을 받은 상태가 아니라면 (command == 0) 수신한 바이트를 명령으로 설정 (command = SPDR) 후 ISR() 실행을 마침
- 마스터가 보내온 명령 ('s')를 해석하고 대응하는 과정은 다음 SPI 데이터 교환 과정에서 이루어짐
- 즉, 마스터가 다음 데이터를 보내 인터럽트가 일어나서 ISR()이 실행되야 switch() 문의 해당 case 's':가 실행

```
volatile byte command = 0;
volatile byte data = 0;

void setup () {
  pinMode(MISO, OUTPUT);
  SPCR |= _BV(SPE);
  SPCR &= ~_BV(MSTR);
  SPCR |= _BV(SPIE);
}

ISR(SPI_STC_vect) {
  byte c = SPDR;
  switch (command) {
  case 0:
    command = c;
    SPDR = 0;
    break;

  case 's':
    SPDR = data;
    break;

  }
}


void loop() {
  data = (byte)random(0, 100);
  if (digitalRead(SS) == HIGH)
    command = 0;
}
```

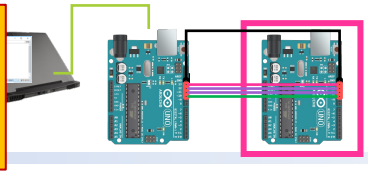
마스터

```
transferAndWait('s');
transferAndWait(0);
s = transferAndWait(0);
```

- 명령 상태가 아니면, 현재 받은 문자를 명령으로 설정

# SPI 통신 예제 1:1 단방향 문자열 송신 (슬레이브 스케치)

결과 34 2 72



Practice

- command는 마스터가 보내오는 명령을 저장하는 변수
- data는 마스터에게 보낼 데이터를 저장하는 변수

- 이 예제에서는 슬레이브가 마스터에게 데이터를 전송하므로 MISO 핀을 출력으로 설정
- SPI 가능, SPI 인터럽트 가능, 슬레이브 레지스터 설정

- 마스터로 부터 명령을 받은 상태가 아니라면 (command == 0) 수신한 바이트를 명령으로 설정 (command = SPDR) 후 ISR() 실행을 마침
- 마스터가 보내온 명령 ('s')를 해석하고 대응하는 과정은 다음 SPI 데이터 교환 과정에서 이루어짐
- 즉, 마스터가 다음 데이터를 보내 인터럽트가 일어나서 ISR()이 실행되야 switch() 문의 해당 case 's':가 실행

- 마스터가 다음 '아무거나' 데이터를 보내오면, 명령 상태 (command == 's')이기 때문에 대응 데이터 (예, 센서 값)를 슬레이브 자신의 데이터 버퍼 SPDR 에 넣어 놓음
- SPDR 에 넣어 놓은 대응 데이터를 마스터로 보내는 전송 과정은 다음 SPI 교환 과정에서 이루어짐

```
volatile byte command = 0;
volatile byte data = 0;

void setup () {
  pinMode(MISO, OUTPUT);
  SPCR |= _BV(SPE);
  SPCR &= ~_BV(MSTR);
  SPCR |= _BV(SPIE);
}

ISR(SPI_STC_vect) {
  byte c = SPDR;
  switch (command) {
  case 0:
    command = c;
    SPDR = 0;
    break;

  case 's':
    SPDR = data;
    break;

  }
}

void loop() {
  data = (byte)random(0, 100);
  if (digitalRead(SS) == HIGH)
    command = 0;
}
```

마스터

```
transferAndWait('s');
transferAndWait(0);
s = transferAndWait(0);
```

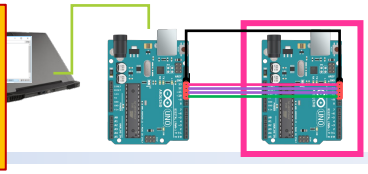
- 명령 상태가 아니면, 현재 받은 문자를 명령으로 설정

- 명령 상태 이면, data를 원형 큐 데이터 버퍼에 설정

- random() 함수로 센서 값을 모사

# SPI 통신 예제 1:1 단방향 문자열 송신 (슬레이브 스케치)

결과 34 2 72



Practice

- command는 마스터가 보내오는 명령을 저장하는 변수
- data는 마스터에게 보낼 데이터를 저장하는 변수

```
volatile byte command = 0;
volatile byte data = 0;
```

- 이 예제에서는 슬레이브가 마스터에게 데이터를 전송하므로 MISO 핀을 출력으로 설정
- SPI 가능, SPI 인터럽트 가능, 슬레이브 레지스터 설정

```
void setup () {
  pinMode(MISO, OUTPUT);
  SPCR |= _BV(SPE);
  SPCR &= ~_BV(MSTR);
  SPCR |= _BV(SPIE);
}
```

- 마스터로 부터 명령을 받은 상태가 아니라면 (command == 0) 수신한 바이트를 명령으로 설정 (command = SPDR) 후 ISR() 실행을 마침
- 마스터가 보내온 명령 ('s')를 해석하고 대응하는 과정은 다음 SPI 데이터 교환 과정에서 이루어짐
- 즉, 마스터가 다음 데이터를 보내 인터럽트가 일어나서 ISR()이 실행되야 switch() 문의 해당 case 's':가 실행

```
ISR(SPI_STC_vect) {
  byte c = SPDR;
  switch (command) {
  case 0:
    command = c;
    SPDR = 0;
    break;

  case 's':
    SPDR = data;
    break;

  }
}
```

마스터

```
transferAndWait('s');
transferAndWait(0);
s = transferAndWait(0);
```

- 명령 상태가 아니면, 현재 받은 문자를 명령으로 설정

- 명령 상태 이면, data를 원형 큐 데이터 버퍼에 설정

- 마스터가 다음 '아무거나' 데이터를 보내오면, 명령 상태 (command == 's')이기 때문에 대응 데이터 (예, 센서 값)를 슬레이브 자신의 데이터 버퍼 SPDR 에 넣어 놓음
- SPDR 에 넣어 놓은 대응 데이터를 마스터로 보내는 전송 과정은 다음 SPI 교환 과정에서 이루어짐

- 마스터가 다음 '아무거나' 데이터를 보내오면, '명령 해석과 관계 없이' 원형 큐를 통해 슬레이브가 전송하는 데이터가 대응 데이터, 즉 마스터가 원래 요청했던 데이터 (센서 값)

- random() 함수로 센서 값을 모사

```
void loop() {
  data = (byte)random(0, 100);
  if (digitalRead(SS) == HIGH)
    command = 0;
}
```

- 마스터가 요청했던 데이터를 받은 후 슬레이브를 놓아주면 (SS를 HIGH로 하면) 명령 상태를 해제 (command = 0)

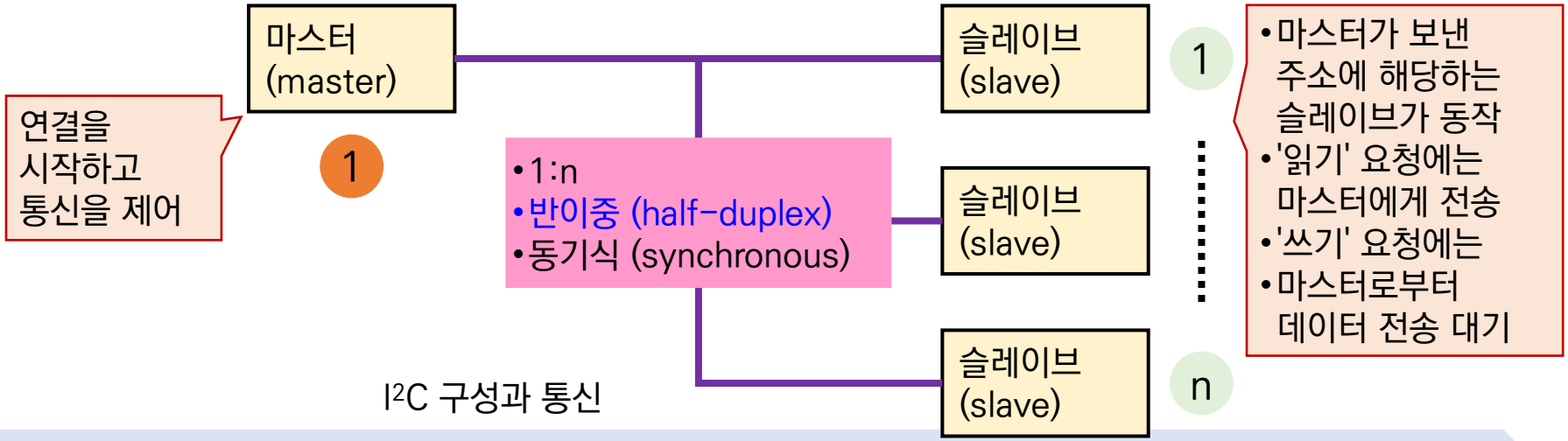
# $I^2C$ 통신

Inter-Integrated Circuit

# I²C Inter-Integrated Circuit

## ■ Inter-Integrated Circuit[1,2]

- 마스터 (master) 장치와 하나 이상의 슬레이브 (slave) 장치로 구성
  - ✓ 1 (마스터) : n (슬레이브) 통신이 가능
- 송신과 수신이 동시에 진행될 수 없는 반이중 (half-duplex) 통신 표준
- 소프트웨어로 슬레이브의 주소를 지정해서 1:n 통신 구현 (SPI와 비교)
- 마이크로프로세서와 주변장치 (peripheral) 간 '저속, 근거리' 통신 (버스, bus[4])에서 많이 활용
  - ✓ 예, 다양한 센서, LCD 디스플레이 등




I²C 구성과 통신

[1] https://en.wikipedia.org/wiki/I%C2%B2C
[2] https://learn.sparkfun.com/tutorials/i2c/all
[3] de facto 표준이란 대중성과 시장 장악력을 바탕으로 어느 회사나 단체의 특정 방식 (예, 제조, 통신 등)이 표준 처럼 인정되는 것, 예로서 HDMI 포트
[4] 컴퓨터 구조에서 '버스'란 컴퓨터 내부에서 컴포넌트 사이에 또는 컴퓨터 사이에 데이터를 주고 받는 통신 시스템을 통칭하는 말

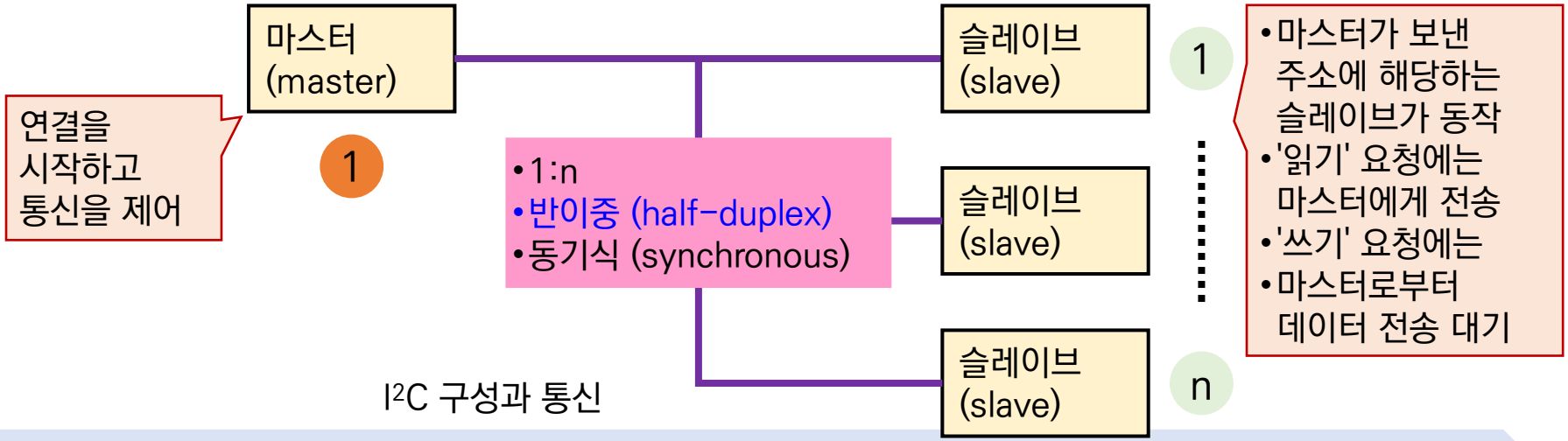
# I²C Inter-Integrated Circuit

- 필립스 반도체 (현재, NXT 반도체) 에서 개발한 산업 (de facto) 표준[3]
- 인텔이 개발한 SMBus 등 변형 표준도 있음
- 장치마다 정확한 사용을 위해 데이터 시트 확인!

- 아이-스퀘어드-씨 (I-squared-C), 아이-투-씨

## ■ Inter-Integrated Circuit[1,2]

- 마스터 (master) 장치와 하나 이상의 슬레이브 (slave) 장치로 구성
  - ✓ 1 (마스터) : n (슬레이브) 통신이 가능
- 송신과 수신이 동시에 진행될 수 없는 반이중 (half-duplex) 통신 표준
- 소프트웨어로 슬레이브의 주소를 지정해서 1:n 통신 구현 (SPI와 비교)
- 마이크로프로세서와 주변장치 (peripheral) 간 '저속, 근거리' 통신 (버스, bus[4])에서 많이 활용
  - ✓ 예, 다양한 센서, LCD 디스플레이 등



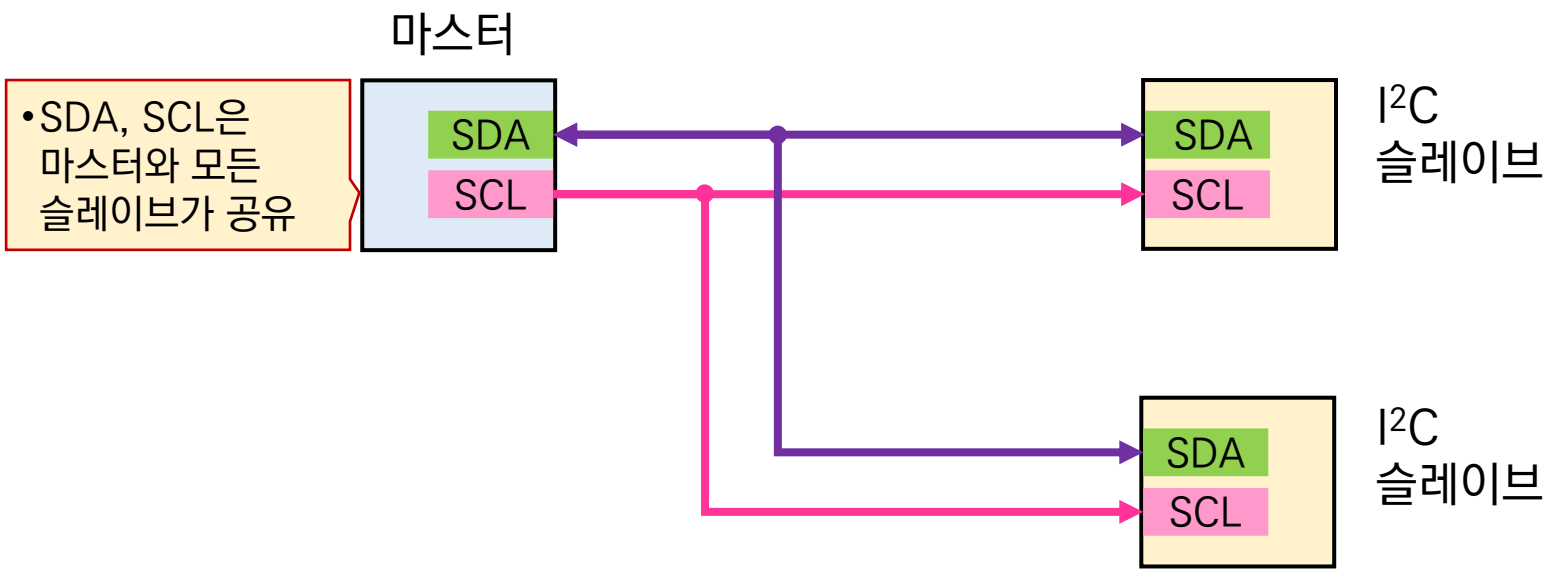

I²C 구성과 통신

[1] https://en.wikipedia.org/wiki/I%C2%B2C
[2] https://learn.sparkfun.com/tutorials/i2c/all
[3] de facto 표준이란 대중성과 시장 장악력을 바탕으로 어느 회사나 단체의 특정 방식 (예, 제조, 통신 등)이 표준 처럼 인정되는 것, 예로서 HDMI 포트
[4] 컴퓨터 구조에서 '버스'란 컴퓨터 내부에서 컴포넌트 사이에 또는 컴퓨터 사이에 데이터를 주고 받는 통신 시스템을 통칭하는 말

# I²C Inter-Integrated Circuit

## ■ I²C 연결선 (wire)과 연결 방법

- SCL (Serial CLock): 마스터-슬레이브 장치 간 동기화를 위한 클럭 전송
- SDA (Serial DAta): 마스터-슬레이브 장치 간 주고 받는 데이터 전송

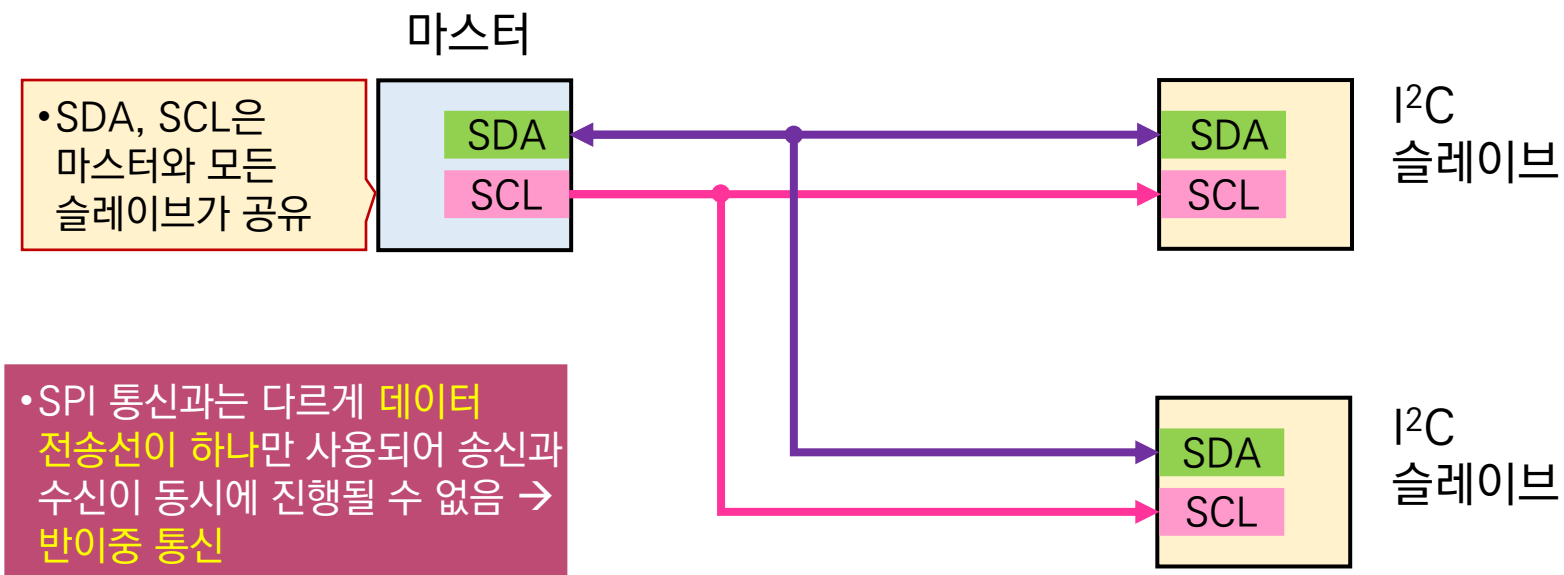



I²C 통신 연결 방법 (1:2)

# I²C Inter-Integrated Circuit

## ■ I²C 연결선 (wire)과 연결 방법

- SCL (Serial CLock): 마스터-슬레이브 장치 간 동기화를 위한 클럭 전송
- SDA (Serial DAta): 마스터-슬레이브 장치 간 주고 받는 데이터 전송

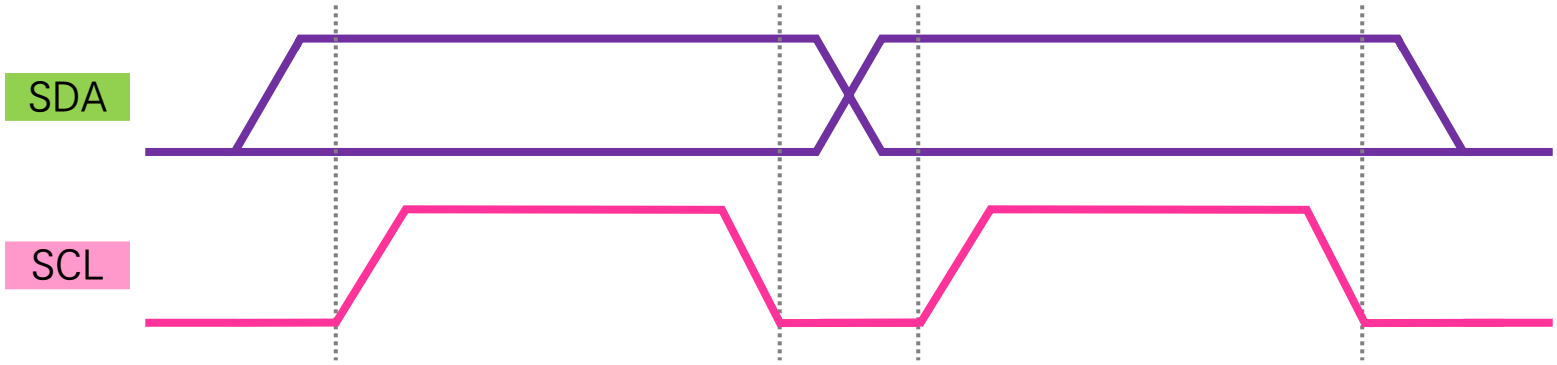



- SPI 통신과는 다르게 데이터 전송선이 하나만 사용되어 송신과 수신이 동시에 진행될 수 없음 → 반이중 통신
- SPI 통신의 SS를 대신하여 슬레이브를 지정할 수 있는 방법이 필요 → 주소

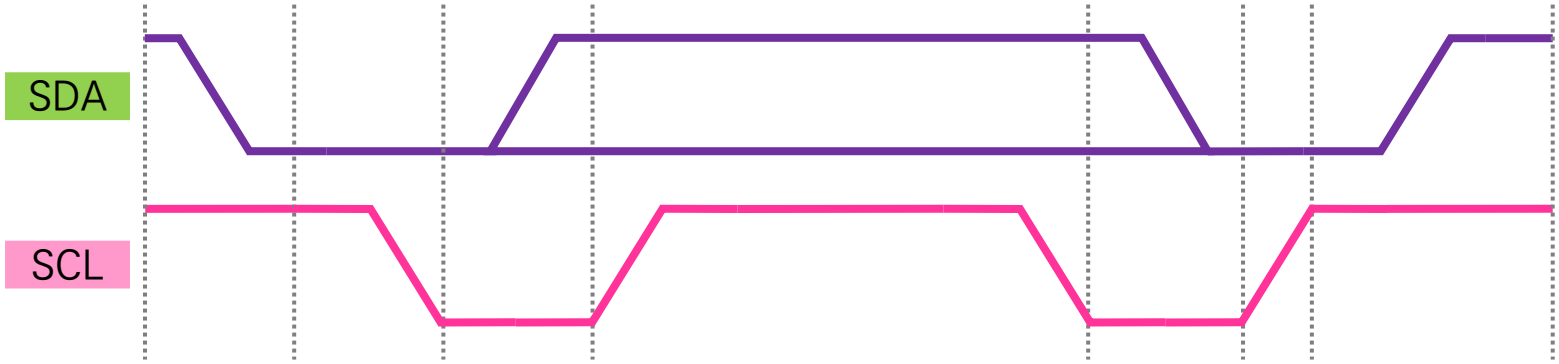
I²C 통신 연결 방법 (1:2)

# I²C 데이터 전송 방식[1]

■ 데이터 샘플링과 변화 허용




■ 데이터 전송의 시작과 종료

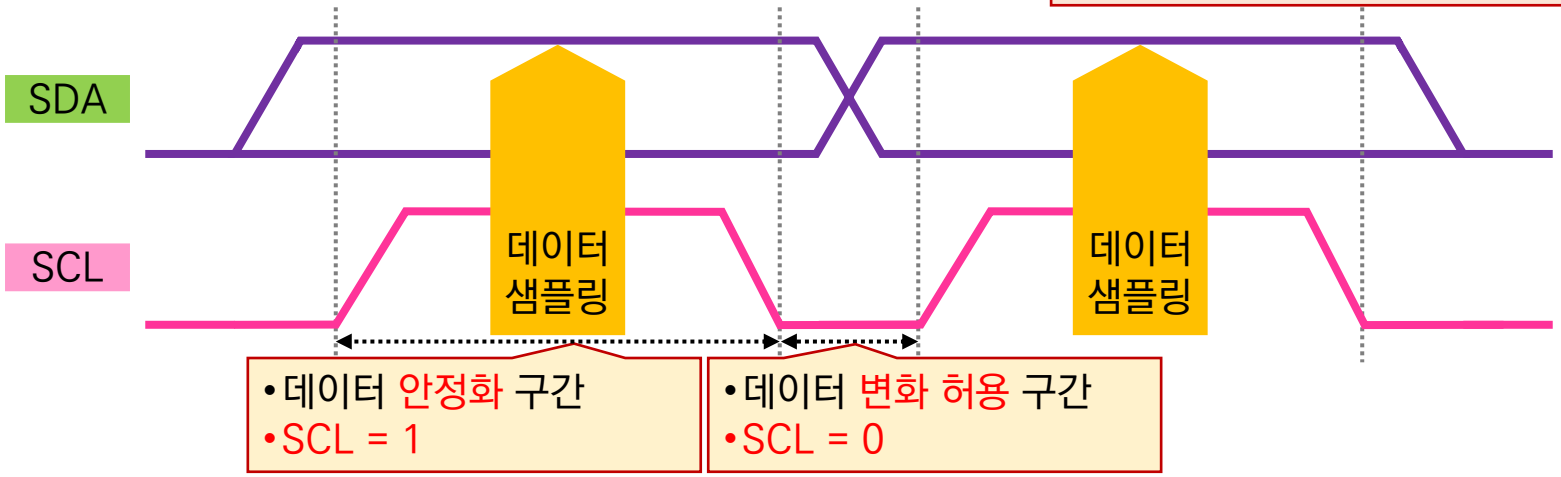



[1] 허경용, 사물인터넷을 품은 아두이노, p69

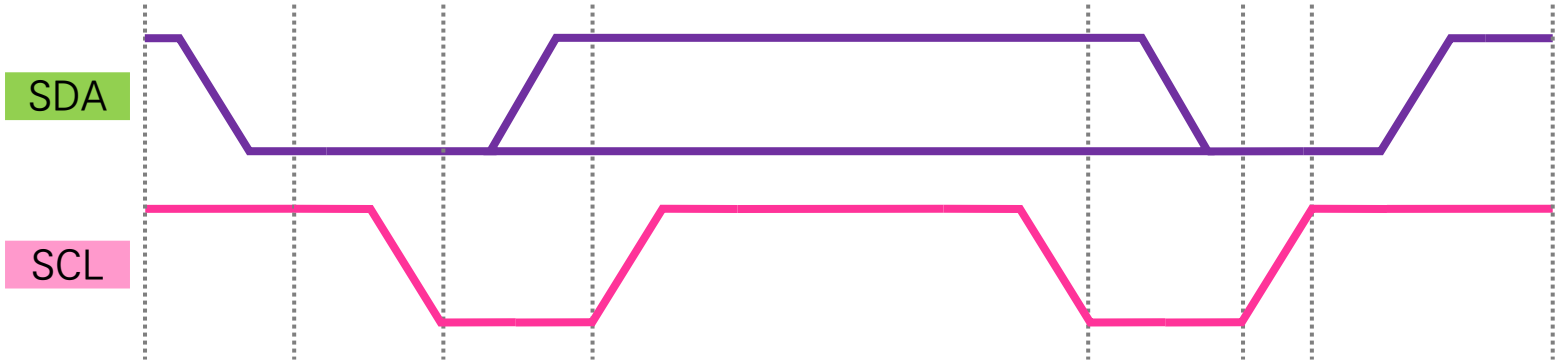
# I²C 데이터 전송 방식[1]

## ■ 데이터 샘플링과 변화 허용

- SCL = HIGH, 수신 데이터 샘플링
- SCL = LOW, 데이터 변화




## ■ 데이터 전송의 시작과 종료

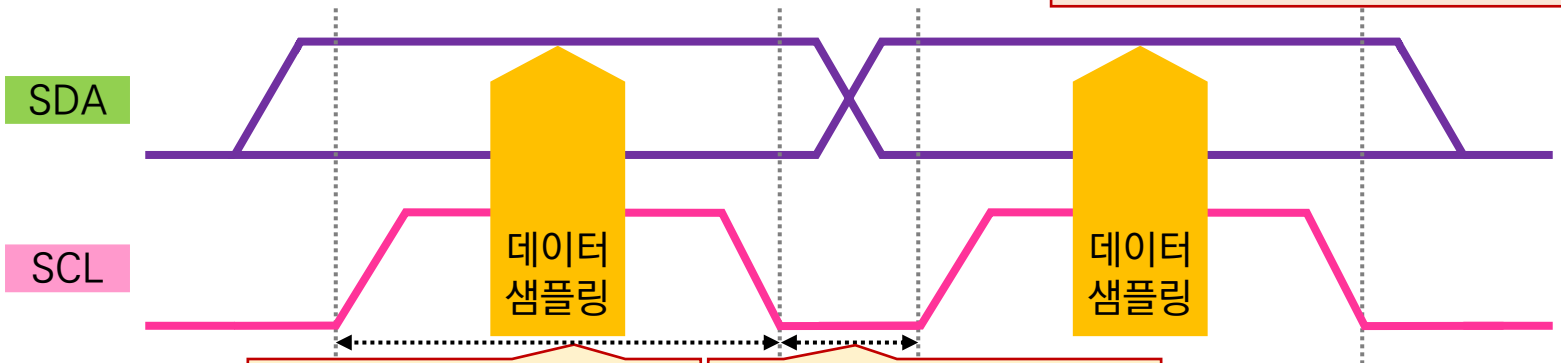



[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 데이터 샘플링과 변화 허용

- SCL = HIGH, 수신 데이터 샘플링
- SCL = LOW, 데이터 변화

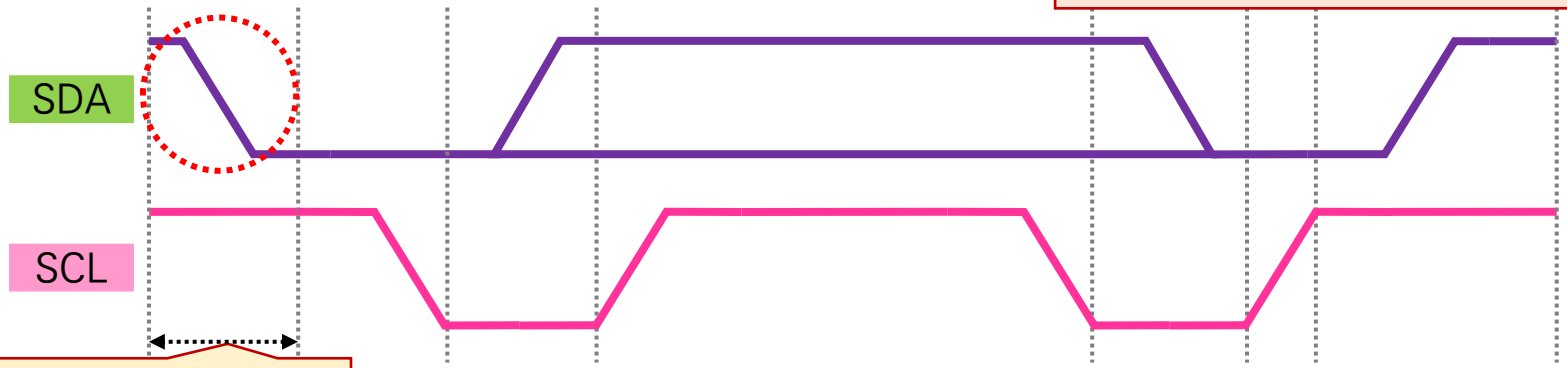



- 데이터 안정화 구간
- SCL = 1

- 데이터 변화 허용 구간
- SCL = 0

## ■ 데이터 전송의 시작과 종료

- SCL = HIGH, SDA 변화하는 경우 데이터 전송의 시작과 종료



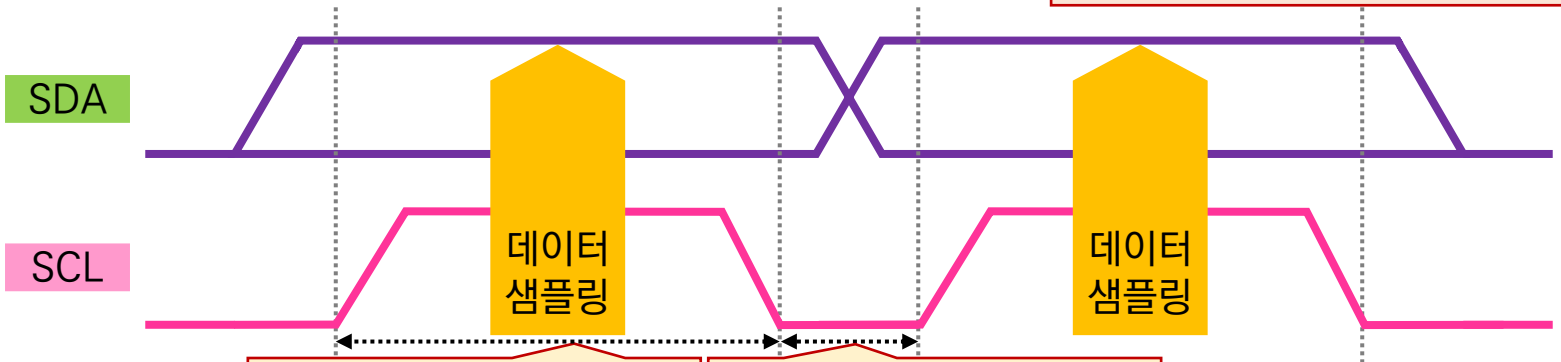

- 데이터 전송 시작 조건
- SCL = 1, SDA: H→L

[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 데이터 샘플링과 변화 허용

- SCL = HIGH, 수신 데이터 샘플링
- SCL = LOW, 데이터 변화

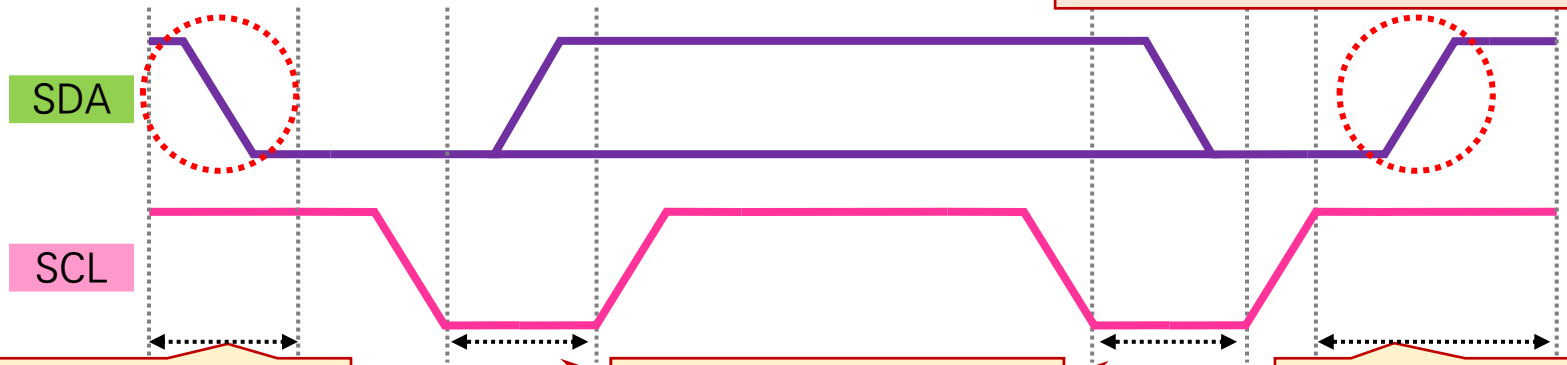



- 데이터 안정화 구간
- SCL = 1

- 데이터 변화 허용 구간
- SCL = 0

## ■ 데이터 전송의 시작과 종료

- SCL = HIGH, SDA 변화하는 경우 데이터 전송의 시작과 종료




- 데이터 전송 시작 조건
- SCL = 1, SDA: H→L

- 데이터 변화 허용 구간
- SCL = 0

- 데이터 전송 마침 조건
- SCL = 1, SDA: L→H

[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 슬레이브 주소 지정

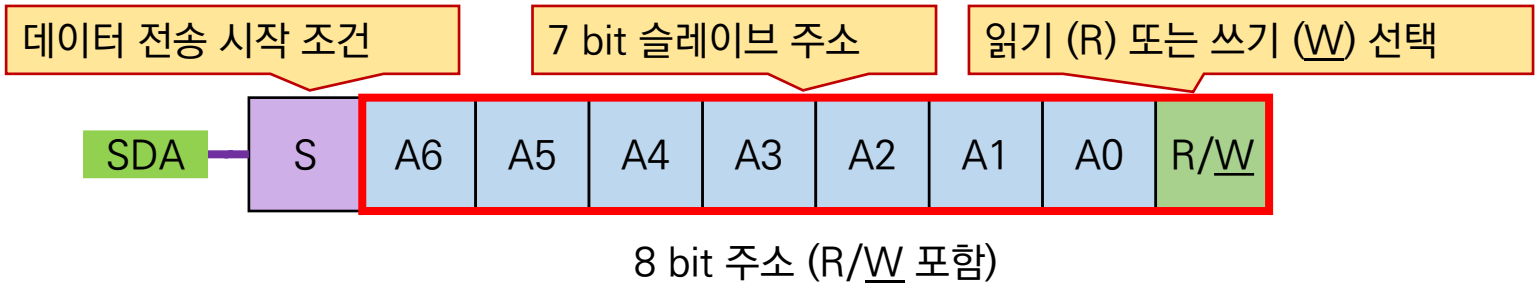
- 슬레이브 주소 (A6–A0, 7 bit) + 읽고/쓰기 (R/W, 1 bit)
  - ✓ 슬레이브 주소 + R: 슬레이브에게 데이터 전송을 '요청 (requesting data)'
    - 슬레이브는 1 byte 데이터를 마스터로 전송
  - ✓ 슬레이브 주소 + W: 슬레이브에게 데이터 전송을 '알림 (sending data)'
    - 슬레이브는 1 byte 데이터를 마스터로부터 수신할 준비

[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 슬레이브 주소 지정

- 슬레이브 주소 (A6–A0, 7 bit) + 읽고/쓰기 (R/W, 1 bit)
  - ✓ 슬레이브 주소 + R: 슬레이브에게 데이터 전송을 '요청 (requesting data)'
    - 슬레이브는 1 byte 데이터를 마스터로 전송
  - ✓ 슬레이브 주소 + W: 슬레이브에게 데이터 전송을 '알림 (sending data)'
    - 슬레이브는 1 byte 데이터를 마스터로부터 수신할 준비

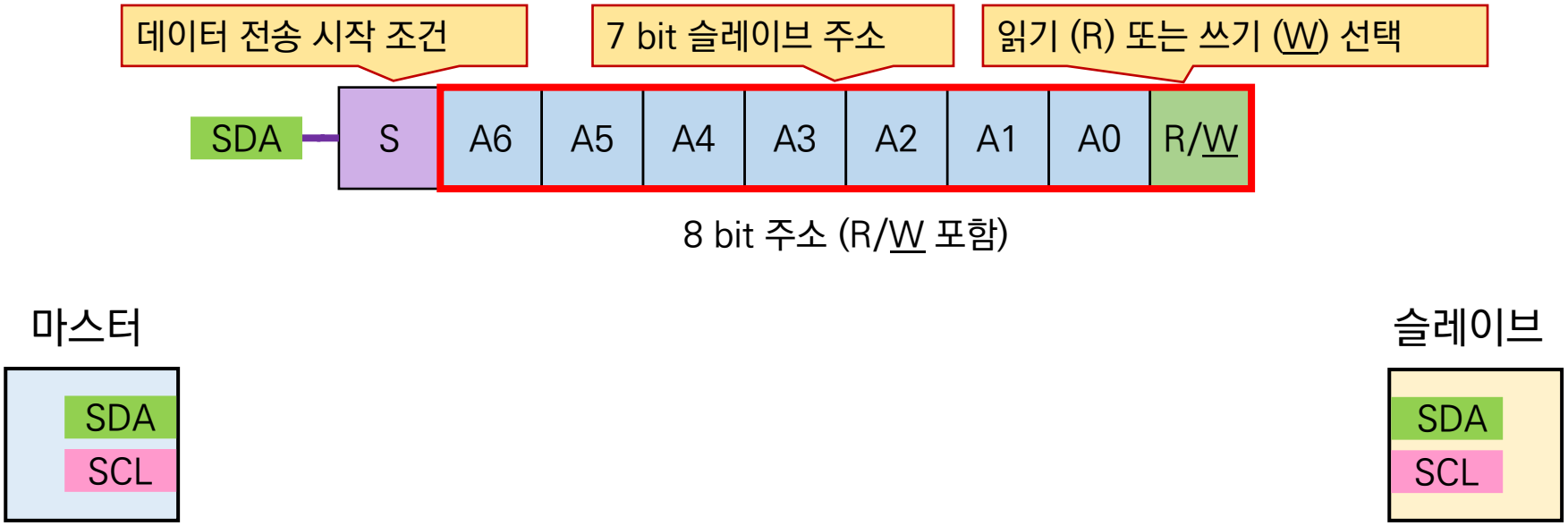



[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 슬레이브 주소 지정

- 슬레이브 주소 (A6–A0, 7 bit) + 읽고/쓰기 (R/W, 1 bit)
  - ✓ 슬레이브 주소 + R: 슬레이브에게 데이터 전송을 '요청 (requesting data)'
    - 슬레이브는 1 byte 데이터를 마스터로 전송
  - ✓ 슬레이브 주소 + W: 슬레이브에게 데이터 전송을 '알림 (sending data)'
    - 슬레이브는 1 byte 데이터를 마스터로부터 수신할 준비




I²C 슬레이브 주소 지정과 R/W bit 설정

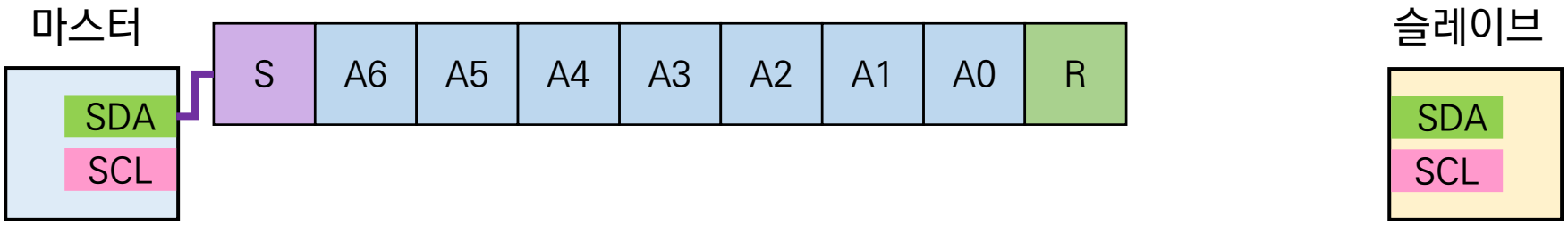
[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 슬레이브 주소 지정

- 슬레이브 주소 (A6–A0, 7 bit) + 읽고/쓰기 (R/W, 1 bit)
  - ✓ 슬레이브 주소 + R: 슬레이브에게 데이터 전송을 '요청 (requesting data)'
    - 슬레이브는 1 byte 데이터를 마스터로 전송
  - ✓ 슬레이브 주소 + W: 슬레이브에게 데이터 전송을 '알림 (sending data)'
    - 슬레이브는 1 byte 데이터를 마스터로부터 수신할 준비

8 bit 주소 (R/W 포함)




I²C 슬레이브 주소 지정과 R/W bit 설정

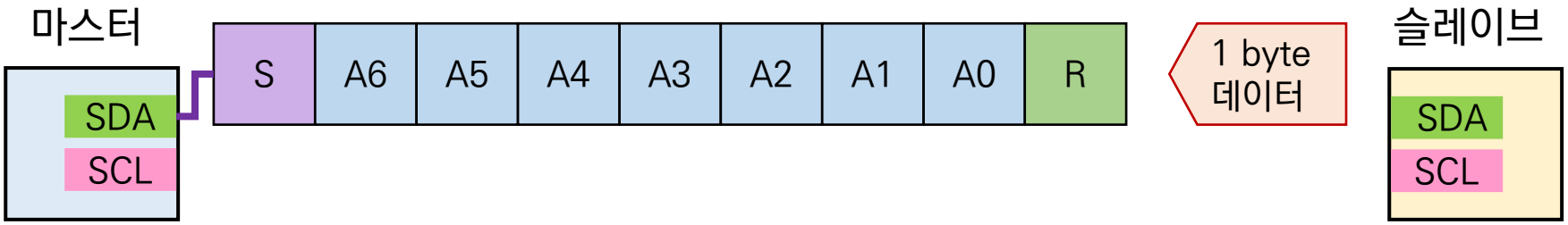
[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 슬레이브 주소 지정

- 슬레이브 주소 (A6–A0, 7 bit) + 읽고/쓰기 (R/W, 1 bit)
  - ✓ 슬레이브 주소 + R: 슬레이브에게 데이터 전송을 '요청 (requesting data)'
    - 슬레이브는 1 byte 데이터를 마스터로 전송
  - ✓ 슬레이브 주소 + W: 슬레이브에게 데이터 전송을 '알림 (sending data)'
    - 슬레이브는 1 byte 데이터를 마스터로부터 수신할 준비

8 bit 주소 (R/W 포함)




I²C 슬레이브 주소 지정과 R/W bit 설정

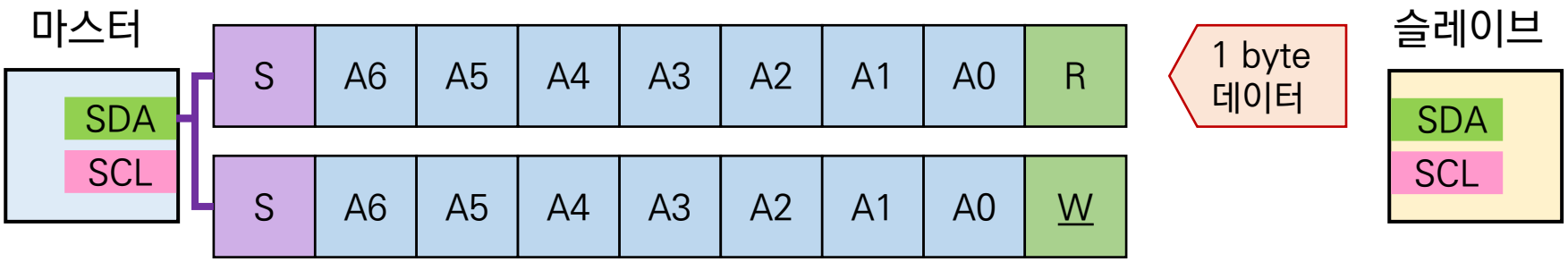
[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 슬레이브 주소 지정

- 슬레이브 주소 (A6–A0, 7 bit) + 읽고/쓰기 (R/W, 1 bit)
  - ✓ 슬레이브 주소 + R: 슬레이브에게 데이터 전송을 '요청 (requesting data)'
    - 슬레이브는 1 byte 데이터를 마스터로 전송
  - ✓ 슬레이브 주소 + W: 슬레이브에게 데이터 전송을 '알림 (sending data)'
    - 슬레이브는 1 byte 데이터를 마스터로부터 수신할 준비

8 bit 주소 (R/W 포함)




I²C 슬레이브 주소 지정과 R/W bit 설정

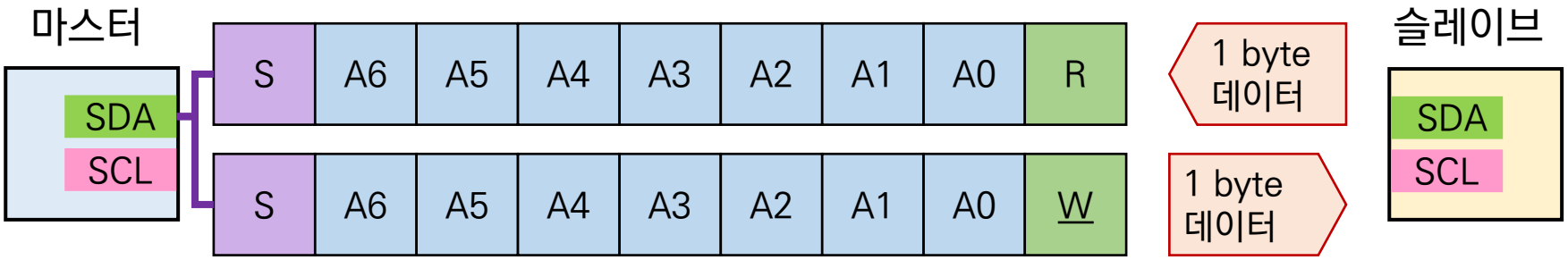
[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 슬레이브 주소 지정

- 슬레이브 주소 (A6–A0, 7 bit) + 읽고/쓰기 (R/W, 1 bit)
  - ✓ 슬레이브 주소 + R: 슬레이브에게 데이터 전송을 '요청 (requesting data)'
    - 슬레이브는 1 byte 데이터를 마스터로 전송
  - ✓ 슬레이브 주소 + W: 슬레이브에게 데이터 전송을 '알림 (sending data)'
    - 슬레이브는 1 byte 데이터를 마스터로부터 수신할 준비

8 bit 주소 (R/W 포함)




I²C 슬레이브 주소 지정과 R/W bit 설정

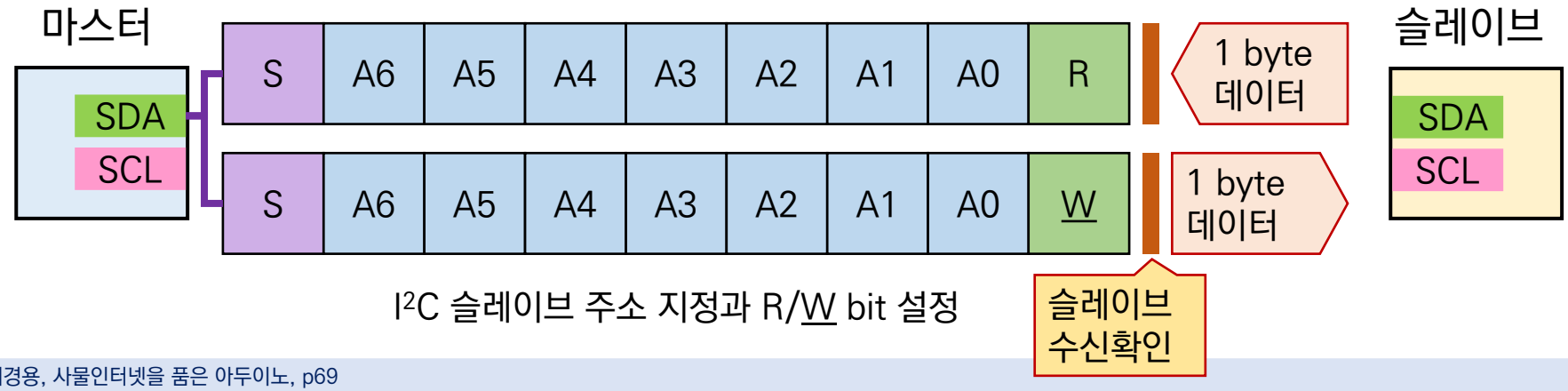
[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 슬레이브 주소 지정

- 슬레이브 주소 (A6–A0, 7 bit) + 읽고/쓰기 (R/W, 1 bit)
  - ✓ 슬레이브 주소 + R: 슬레이브에게 데이터 전송을 '요청 (requesting data)'
    - 슬레이브는 1 byte 데이터를 마스터로 전송
  - ✓ 슬레이브 주소 + W: 슬레이브에게 데이터 전송을 '알림 (sending data)'
    - 슬레이브는 1 byte 데이터를 마스터로부터 수신할 준비

8 bit 주소 (R/W 포함)



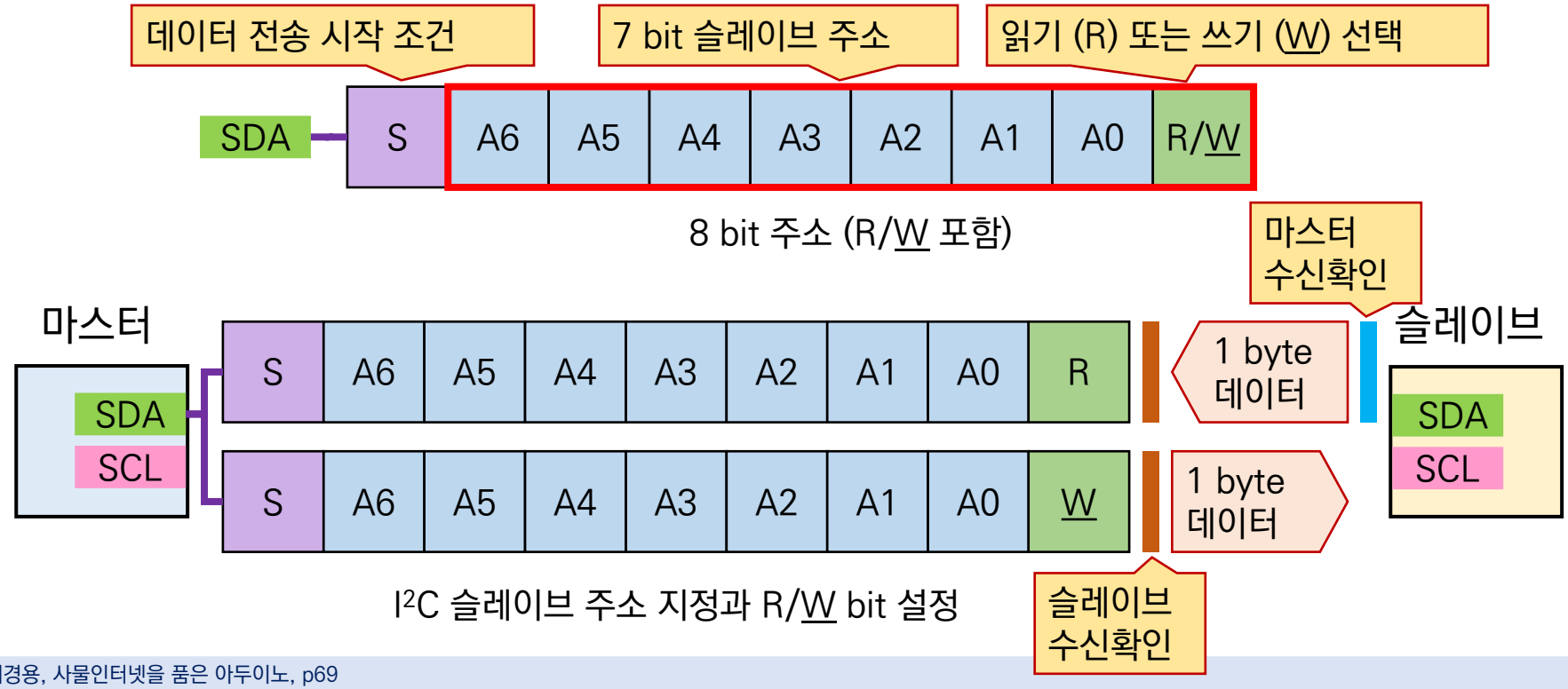

I²C 슬레이브 주소 지정과 R/W bit 설정

[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 슬레이브 주소 지정

- 슬레이브 주소 (A6–A0, 7 bit) + 읽고/쓰기 (R/W, 1 bit)
  - ✓ 슬레이브 주소 + R: 슬레이브에게 데이터 전송을 '요청 (requesting data)'
    - 슬레이브는 1 byte 데이터를 마스터로 전송
  - ✓ 슬레이브 주소 + W: 슬레이브에게 데이터 전송을 '알림 (sending data)'
    - 슬레이브는 1 byte 데이터를 마스터로부터 수신할 준비

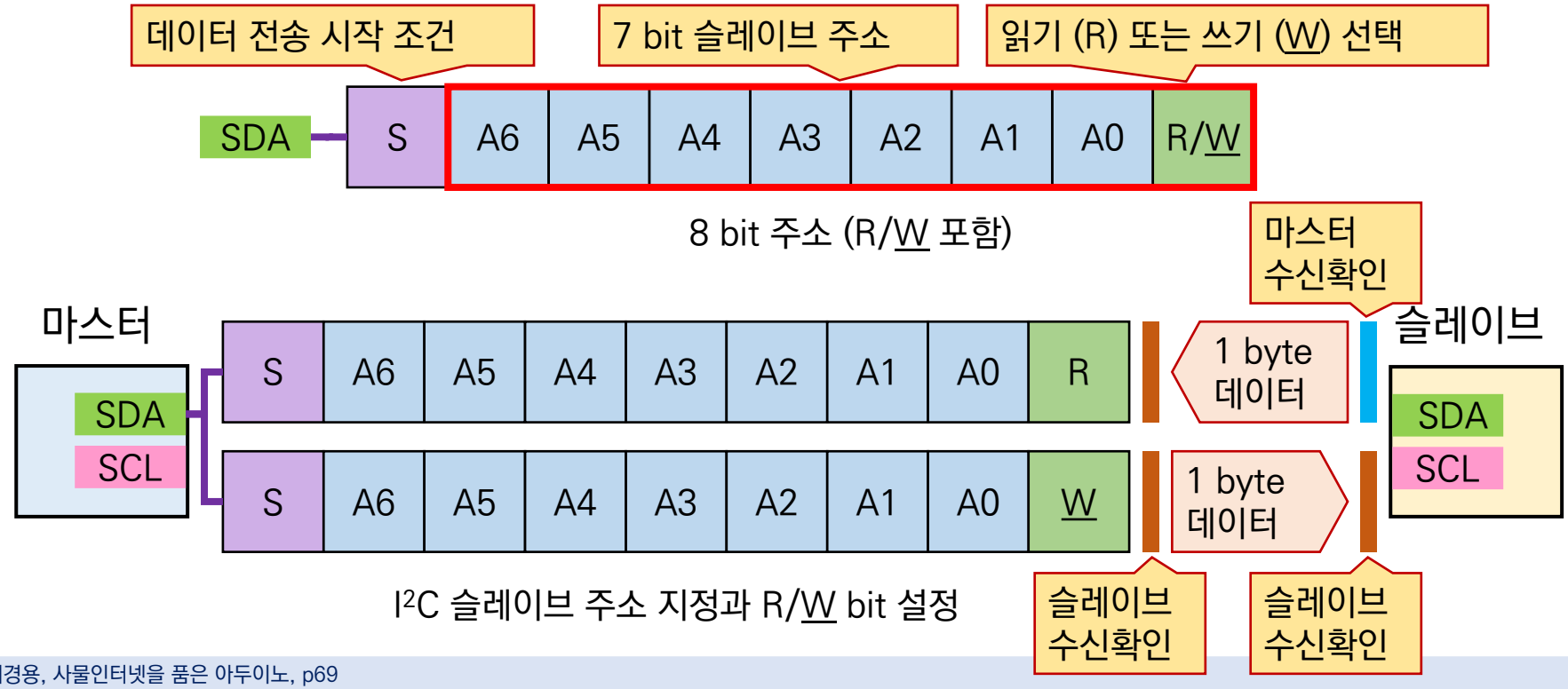



[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 슬레이브 주소 지정

- 슬레이브 주소 (A6–A0, 7 bit) + 읽고/쓰기 (R/W, 1 bit)
  - ✓ 슬레이브 주소 + R: 슬레이브에게 데이터 전송을 '요청 (requesting data)'
    - 슬레이브는 1 byte 데이터를 마스터로 전송
  - ✓ 슬레이브 주소 + W: 슬레이브에게 데이터 전송을 '알림 (sending data)'
    - 슬레이브는 1 byte 데이터를 마스터로부터 수신할 준비

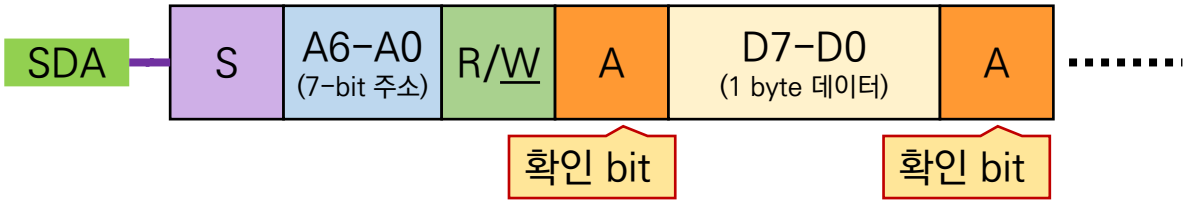



[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 데이터 수신 여부 확인 (ACK/NACK)

- 수신 측은 데이터를 잘 받았음을 송신 측에 알리기 위해 1 bit를 전송
  - ✓ 잘 받았을 때 '오류 없음 (acknowledgement)'을 의미하는 ACK (LOW) 전송
  - ✓ 수신 과정에서 문제가 있었을 때 '오류 있음 (negative acknowledgement)'을 의미하는 NACK (HIGH) 전송
- 주소 (1 byte)와 데이터 (1 byte) 전송 때 모두 수신측은 확인 bit 전송

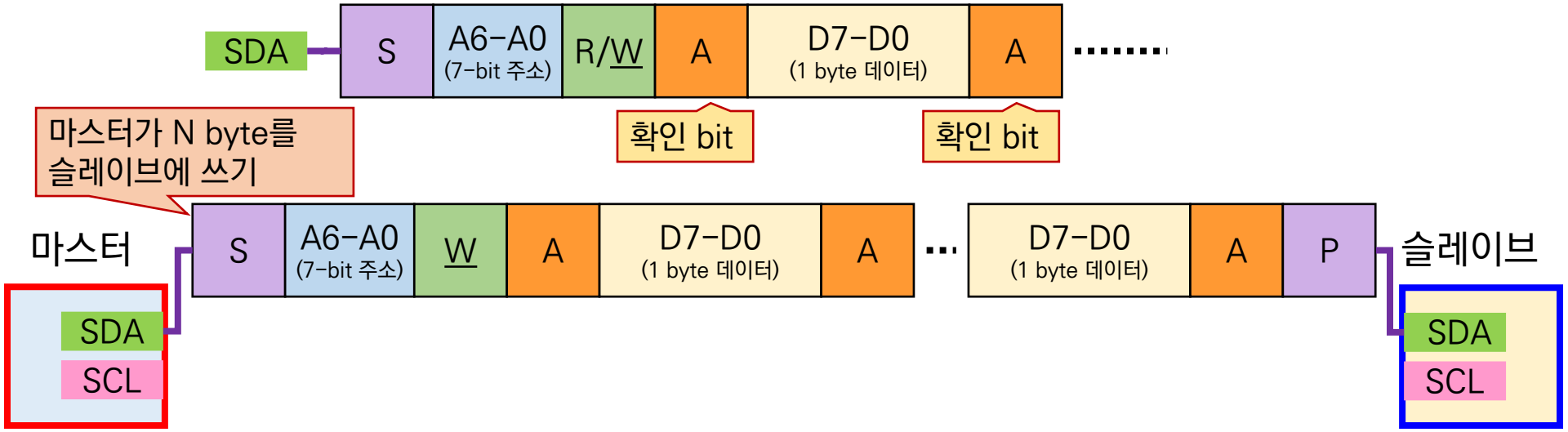



[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 데이터 수신 여부 확인 (ACK/NACK)

- 수신 측은 데이터를 잘 받았음을 송신 측에 알리기 위해 1 bit를 전송
  - ✓ 잘 받았을 때 '오류 없음 (acknowledgement)'을 의미하는 ACK (LOW) 전송
  - ✓ 수신 과정에서 문제가 있었을 때 '오류 있음 (negative acknowledgement)'을 의미하는 NACK (HIGH) 전송
- 주소 (1 byte)와 데이터 (1 byte) 전송 때 모두 수신측은 확인 bit 전송

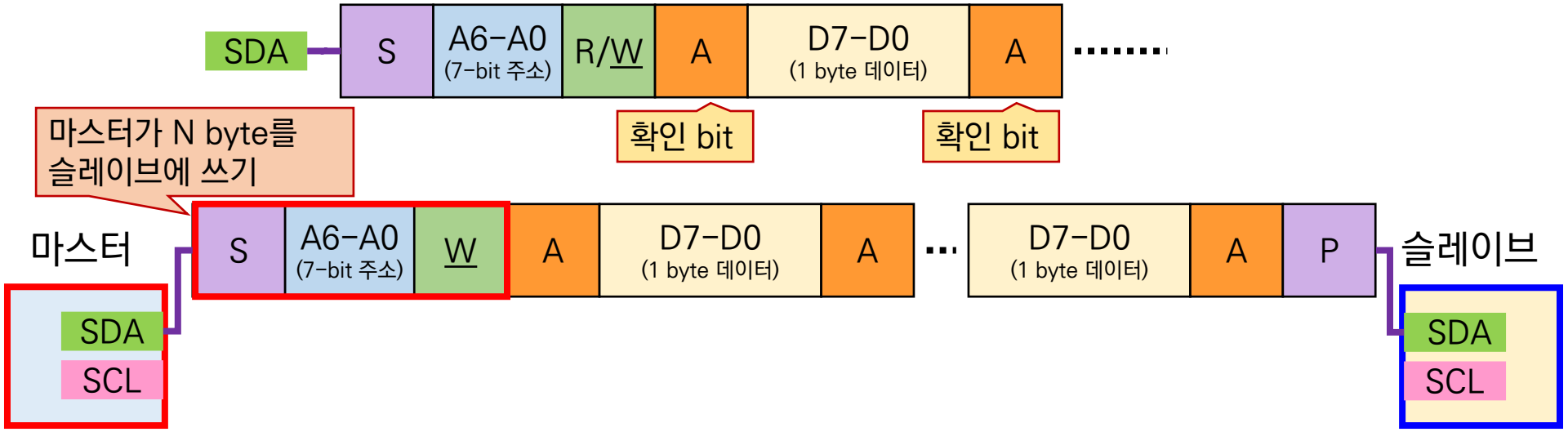



[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 데이터 수신 여부 확인 (ACK/NACK)

- 수신 측은 데이터를 잘 받았음을 송신 측에 알리기 위해 1 bit를 전송
  - ✓ 잘 받았을 때 '오류 없음 (acknowledgement)'을 의미하는 ACK (LOW) 전송
  - ✓ 수신 과정에서 문제가 있었을 때 '오류 있음 (negative acknowledgement)'을 의미하는 NACK (HIGH) 전송
- 주소 (1 byte)와 데이터 (1 byte) 전송 때 모두 수신측은 확인 bit 전송

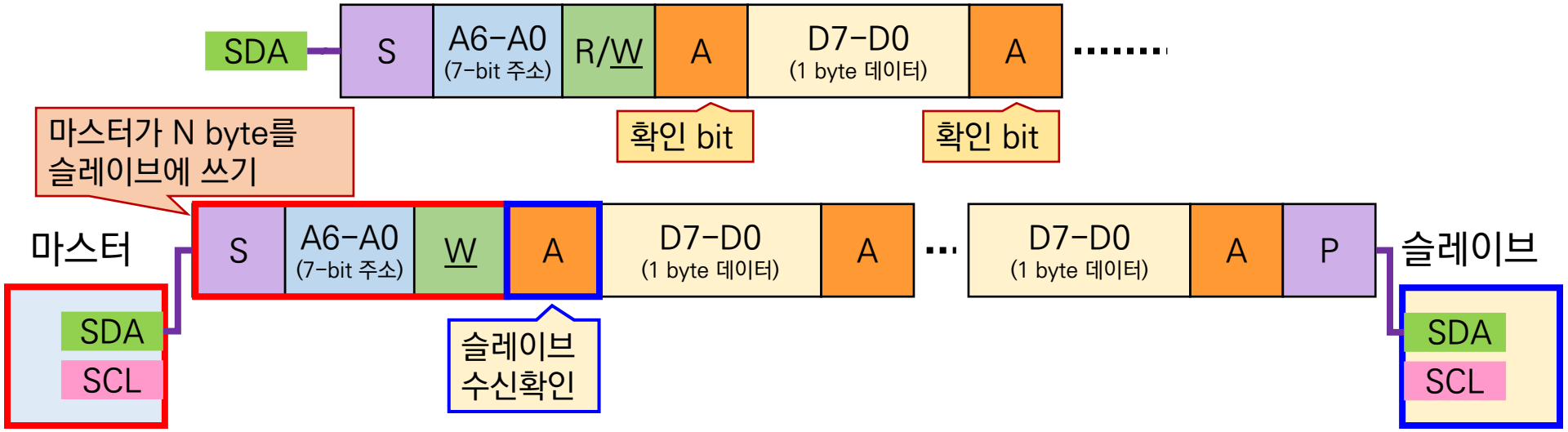



[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 데이터 수신 여부 확인 (ACK/NACK)

- 수신 측은 데이터를 잘 받았음을 송신 측에 알리기 위해 1 bit를 전송
  - ✓ 잘 받았을 때 '오류 없음 (acknowledgement)'을 의미하는 ACK (LOW) 전송
  - ✓ 수신 과정에서 문제가 있었을 때 '오류 있음 (negative acknowledgement)'을 의미하는 NACK (HIGH) 전송
- 주소 (1 byte)와 데이터 (1 byte) 전송 때 모두 수신측은 확인 bit 전송

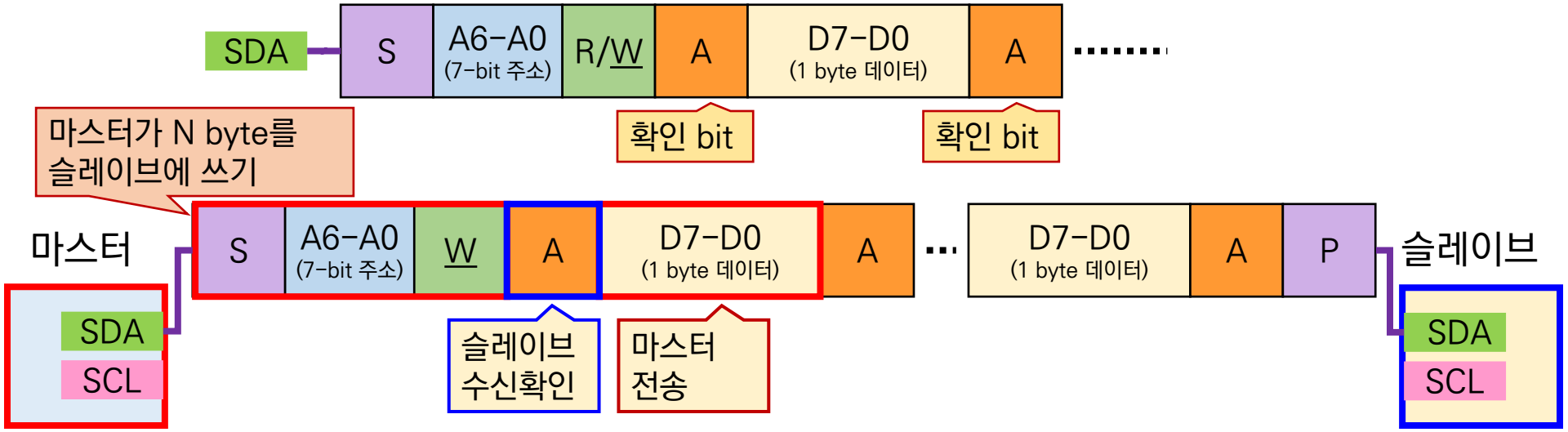



[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 데이터 수신 여부 확인 (ACK/NACK)

- 수신 측은 데이터를 잘 받았음을 송신 측에 알리기 위해 1 bit를 전송
  - ✓ 잘 받았을 때 '오류 없음 (acknowledgement)'을 의미하는 ACK (LOW) 전송
  - ✓ 수신 과정에서 문제가 있었을 때 '오류 있음 (negative acknowledgement)'을 의미하는 NACK (HIGH) 전송
- 주소 (1 byte)와 데이터 (1 byte) 전송 때 모두 수신측은 확인 bit 전송

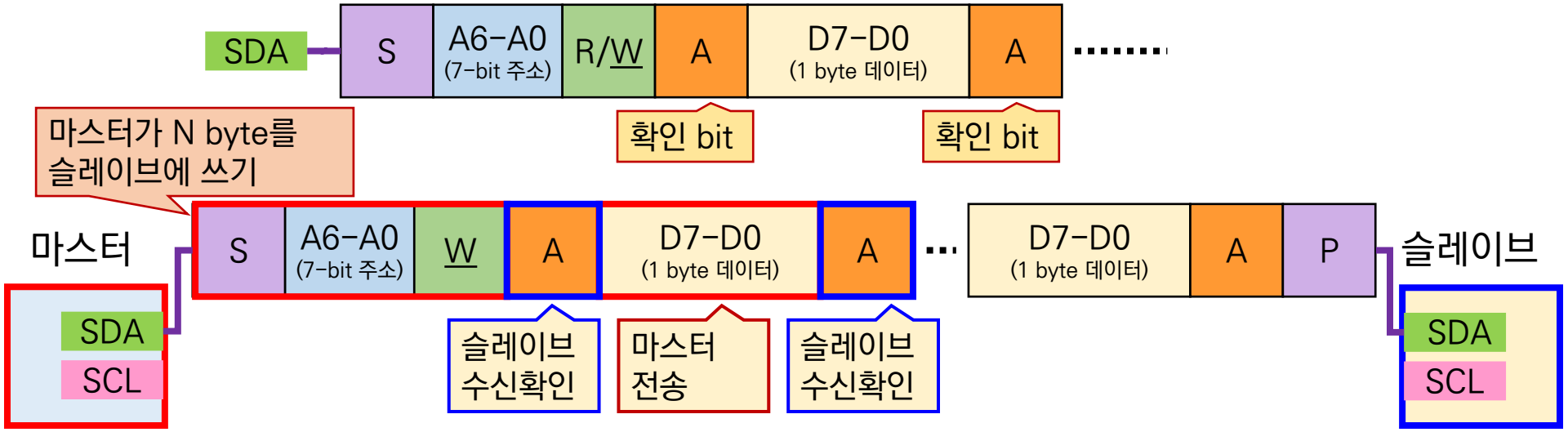



[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 데이터 수신 여부 확인 (ACK/NACK)

- 수신 측은 데이터를 잘 받았음을 송신 측에 알리기 위해 1 bit를 전송
  - ✓ 잘 받았을 때 '오류 없음 (acknowledgement)'을 의미하는 ACK (LOW) 전송
  - ✓ 수신 과정에서 문제가 있었을 때 '오류 있음 (negative acknowledgement)'을 의미하는 NACK (HIGH) 전송
- 주소 (1 byte)와 데이터 (1 byte) 전송 때 모두 수신측은 확인 bit 전송

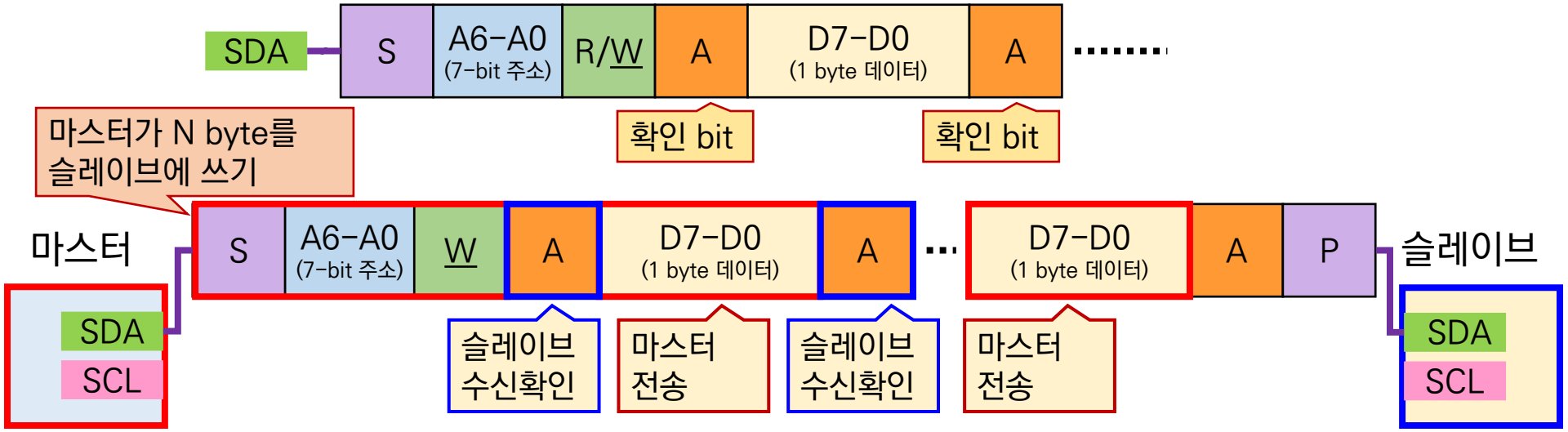



[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 데이터 수신 여부 확인 (ACK/NACK)

- 수신 측은 데이터를 잘 받았음을 송신 측에 알리기 위해 1 bit를 전송
  - ✓ 잘 받았을 때 '오류 없음 (acknowledgement)'을 의미하는 ACK (LOW) 전송
  - ✓ 수신 과정에서 문제가 있었을 때 '오류 있음 (negative acknowledgement)'을 의미하는 NACK (HIGH) 전송
- 주소 (1 byte)와 데이터 (1 byte) 전송 때 모두 수신측은 확인 bit 전송

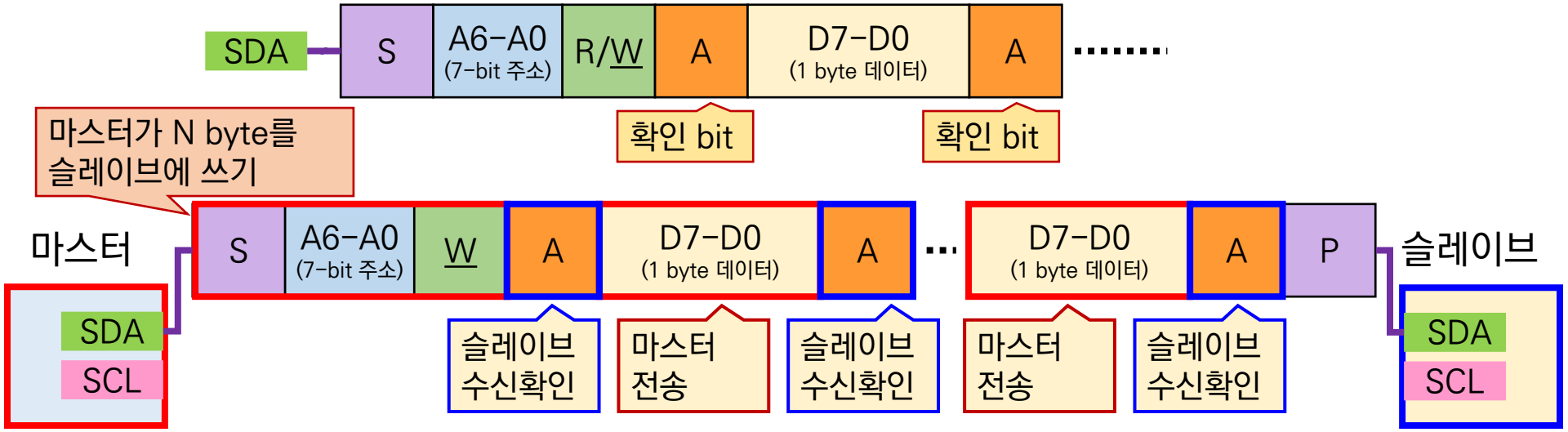



[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 데이터 수신 여부 확인 (ACK/NACK)

- 수신 측은 데이터를 잘 받았음을 송신 측에 알리기 위해 1 bit를 전송
  - ✓ 잘 받았을 때 '오류 없음 (acknowledgement)'을 의미하는 ACK (LOW) 전송
  - ✓ 수신 과정에서 문제가 있었을 때 '오류 있음 (negative acknowledgement)'을 의미하는 NACK (HIGH) 전송
- 주소 (1 byte)와 데이터 (1 byte) 전송 때 모두 수신측은 확인 bit 전송

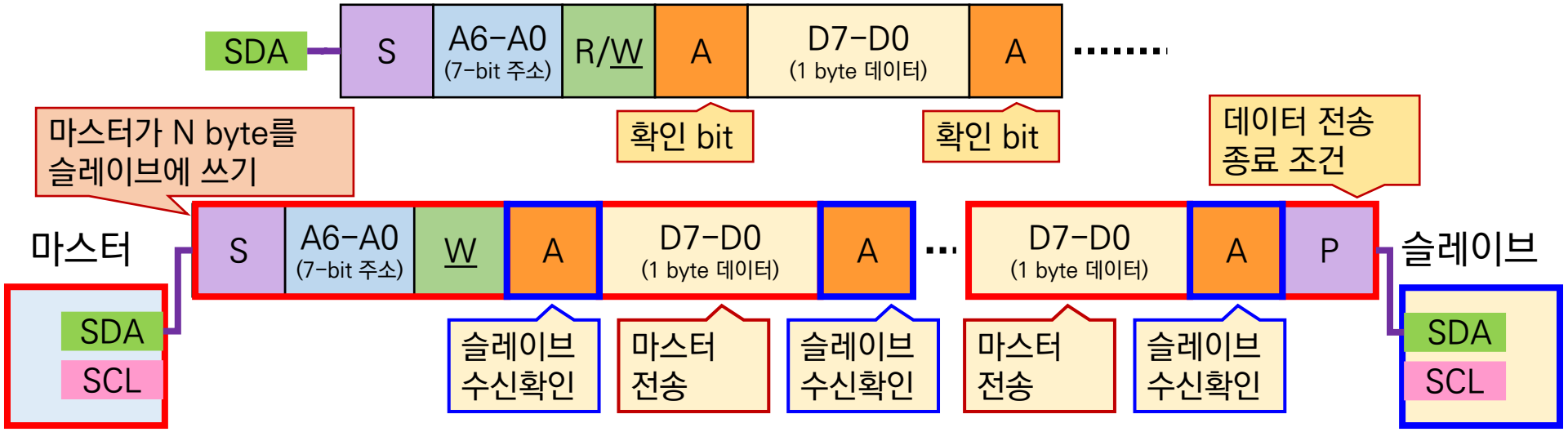



[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 데이터 수신 여부 확인 (ACK/NACK)

- 수신 측은 데이터를 잘 받았음을 송신 측에 알리기 위해 1 bit를 전송
  - ✓ 잘 받았을 때 '오류 없음 (acknowledgement)'을 의미하는 ACK (LOW) 전송
  - ✓ 수신 과정에서 문제가 있었을 때 '오류 있음 (negative acknowledgement)'을 의미하는 NACK (HIGH) 전송
- 주소 (1 byte)와 데이터 (1 byte) 전송 때 모두 수신측은 확인 bit 전송

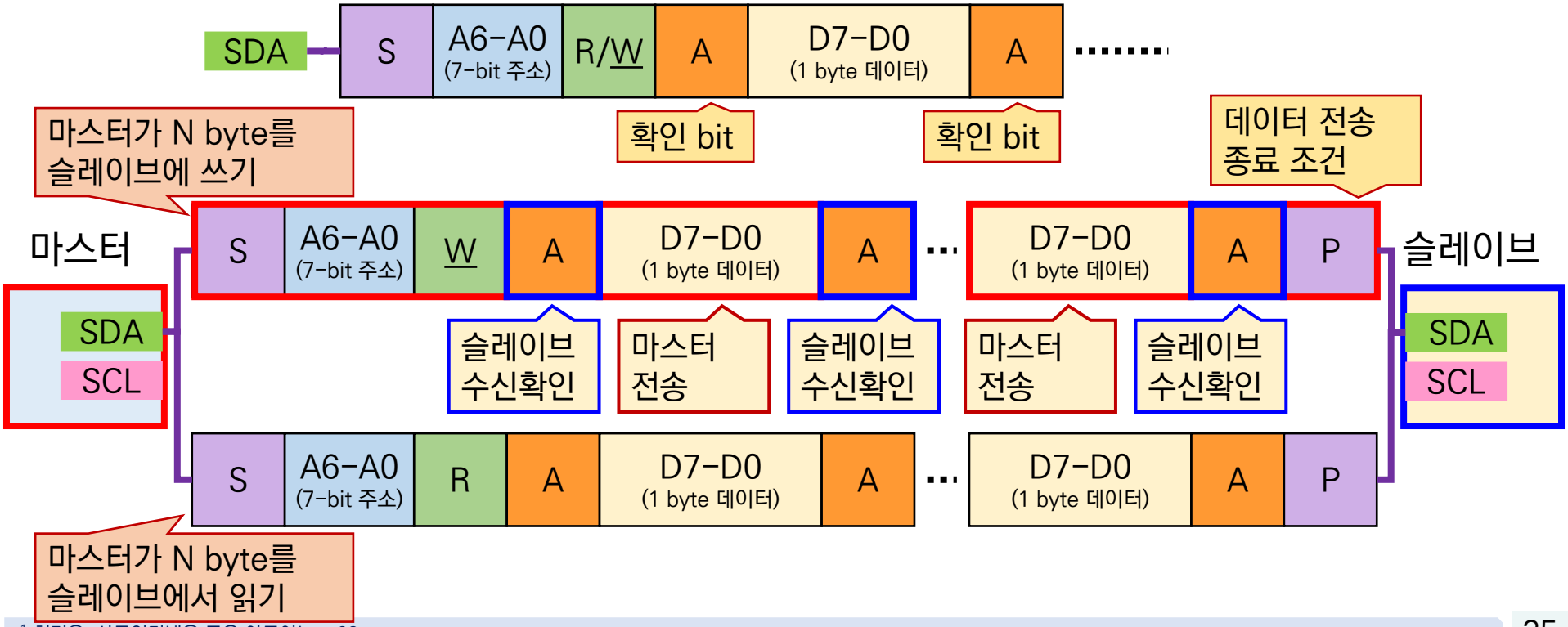



[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 데이터 수신 여부 확인 (ACK/NACK)

- 수신 측은 데이터를 잘 받았음을 송신 측에 알리기 위해 1 bit를 전송
  - ✓ 잘 받았을 때 '오류 없음 (acknowledgement)'을 의미하는 ACK (LOW) 전송
  - ✓ 수신 과정에서 문제가 있었을 때 '오류 있음 (negative acknowledgement)'을 의미하는 NACK (HIGH) 전송
- 주소 (1 byte)와 데이터 (1 byte) 전송 때 모두 수신측은 확인 bit 전송

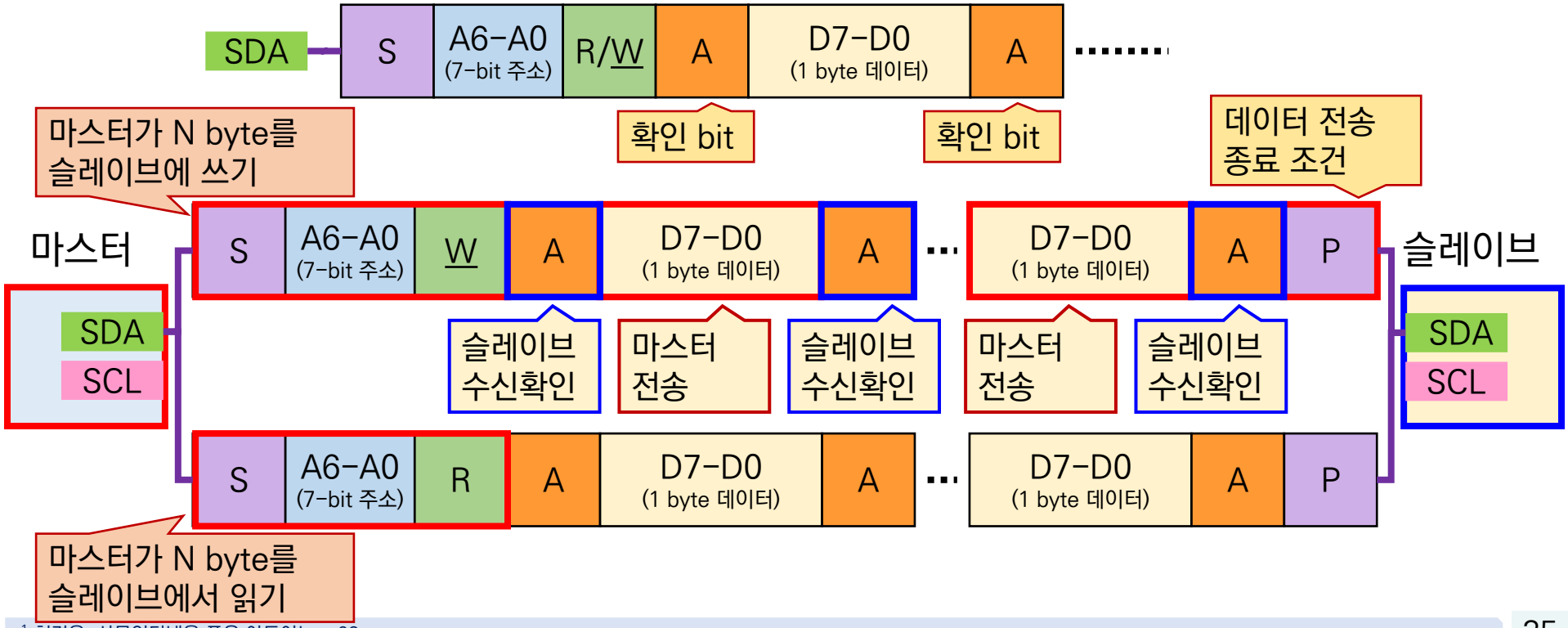



[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 데이터 수신 여부 확인 (ACK/NACK)

- 수신 측은 데이터를 잘 받았음을 송신 측에 알리기 위해 1 bit를 전송
  - ✓ 잘 받았을 때 '오류 없음 (acknowledgement)'을 의미하는 ACK (LOW) 전송
  - ✓ 수신 과정에서 문제가 있었을 때 '오류 있음 (negative acknowledgement)'을 의미하는 NACK (HIGH) 전송
- 주소 (1 byte)와 데이터 (1 byte) 전송 때 모두 수신측은 확인 bit 전송

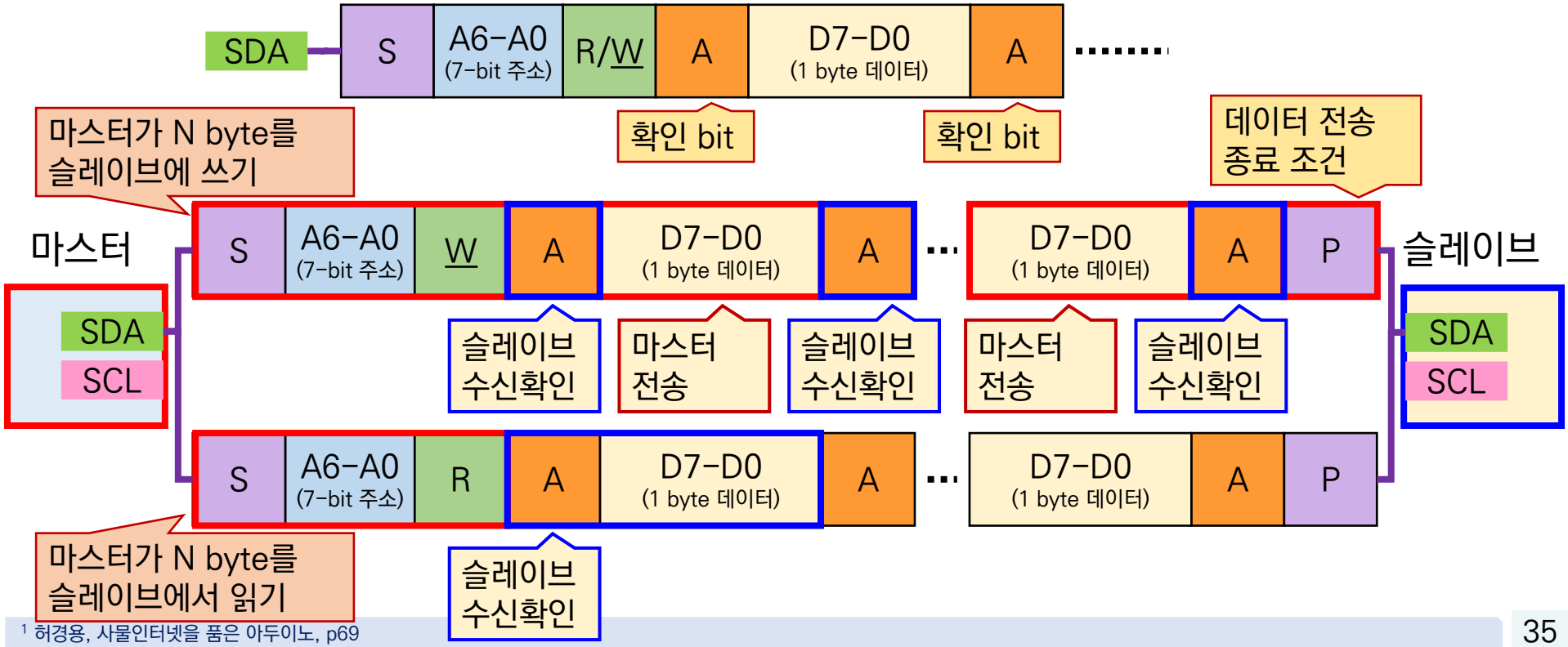



[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 데이터 수신 여부 확인 (ACK/NACK)

- 수신 측은 데이터를 잘 받았음을 송신 측에 알리기 위해 1 bit를 전송
  - ✓ 잘 받았을 때 '오류 없음 (acknowledgement)'을 의미하는 ACK (LOW) 전송
  - ✓ 수신 과정에서 문제가 있었을 때 '오류 있음 (negative acknowledgement)'을 의미하는 NACK (HIGH) 전송
- 주소 (1 byte)와 데이터 (1 byte) 전송 때 모두 수신측은 확인 bit 전송

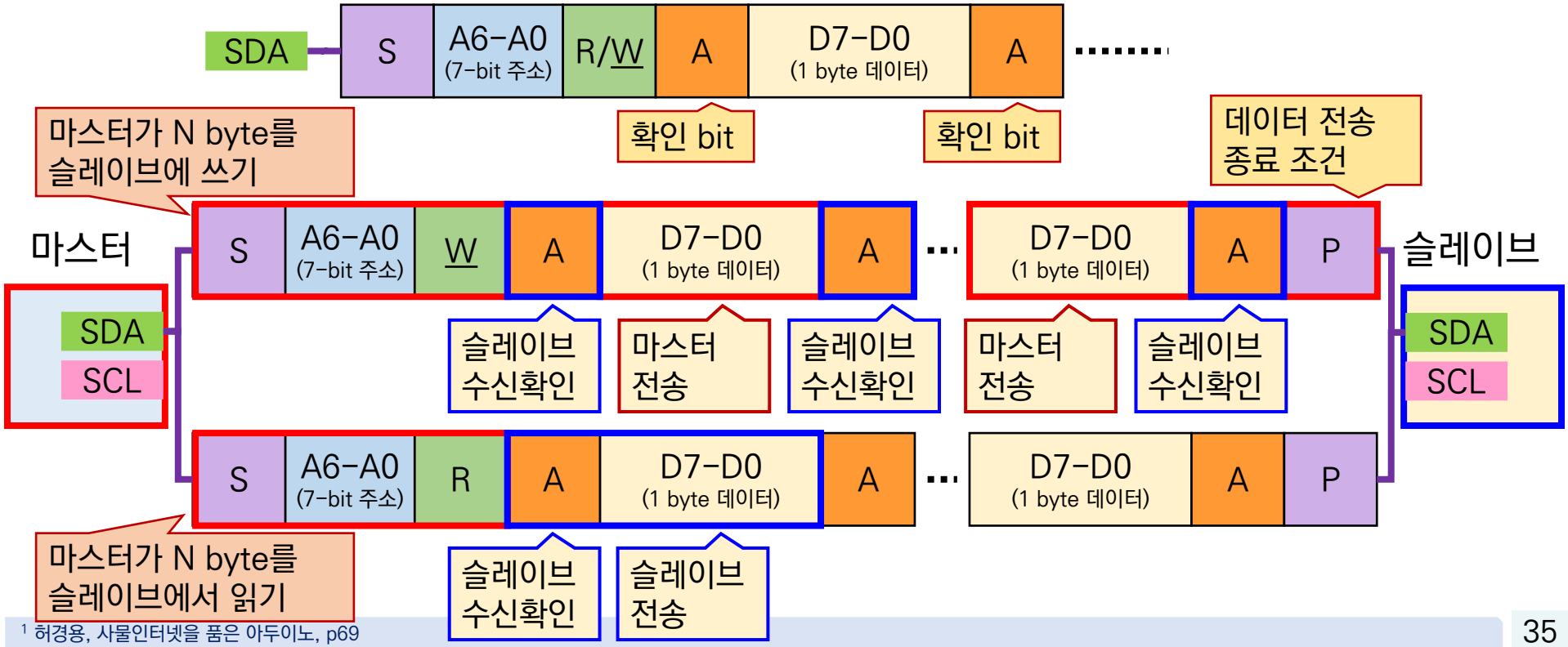



[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 데이터 수신 여부 확인 (ACK/NACK)

- 수신 측은 데이터를 잘 받았음을 송신 측에 알리기 위해 1 bit를 전송
  - ✓ 잘 받았을 때 '오류 없음 (acknowledgement)'을 의미하는 ACK (LOW) 전송
  - ✓ 수신 과정에서 문제가 있었을 때 '오류 있음 (negative acknowledgement)'을 의미하는 NACK (HIGH) 전송
- 주소 (1 byte)와 데이터 (1 byte) 전송 때 모두 수신측은 확인 bit 전송

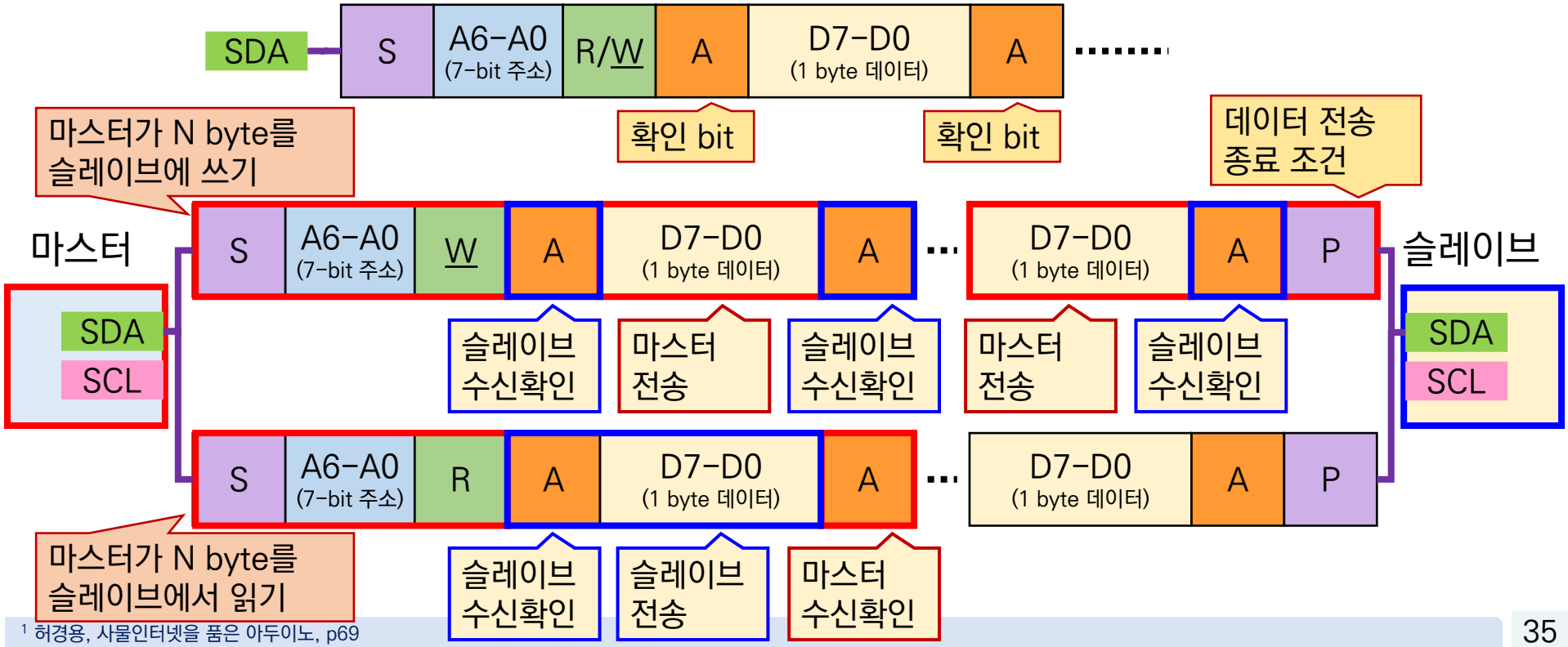



[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 데이터 수신 여부 확인 (ACK/NACK)

- 수신 측은 데이터를 잘 받았음을 송신 측에 알리기 위해 1 bit를 전송
  - ✓ 잘 받았을 때 '오류 없음 (acknowledgement)'을 의미하는 ACK (LOW) 전송
  - ✓ 수신 과정에서 문제가 있었을 때 '오류 있음 (negative acknowledgement)'을 의미하는 NACK (HIGH) 전송
- 주소 (1 byte)와 데이터 (1 byte) 전송 때 모두 수신측은 확인 bit 전송

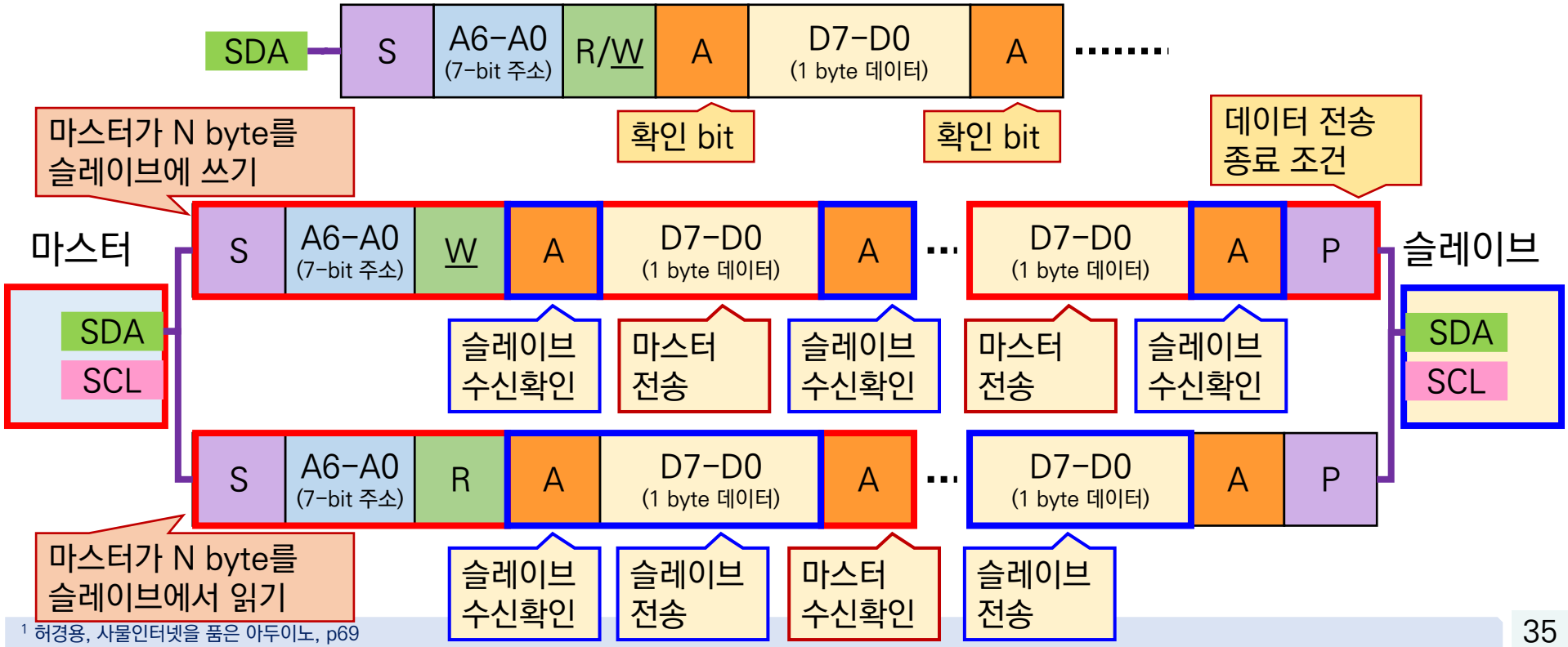



[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 데이터 수신 여부 확인 (ACK/NACK)

- 수신 측은 데이터를 잘 받았음을 송신 측에 알리기 위해 1 bit를 전송
  - ✓ 잘 받았을 때 '오류 없음 (acknowledgement)'을 의미하는 ACK (LOW) 전송
  - ✓ 수신 과정에서 문제가 있었을 때 '오류 있음 (negative acknowledgement)'을 의미하는 NACK (HIGH) 전송
- 주소 (1 byte)와 데이터 (1 byte) 전송 때 모두 수신측은 확인 bit 전송

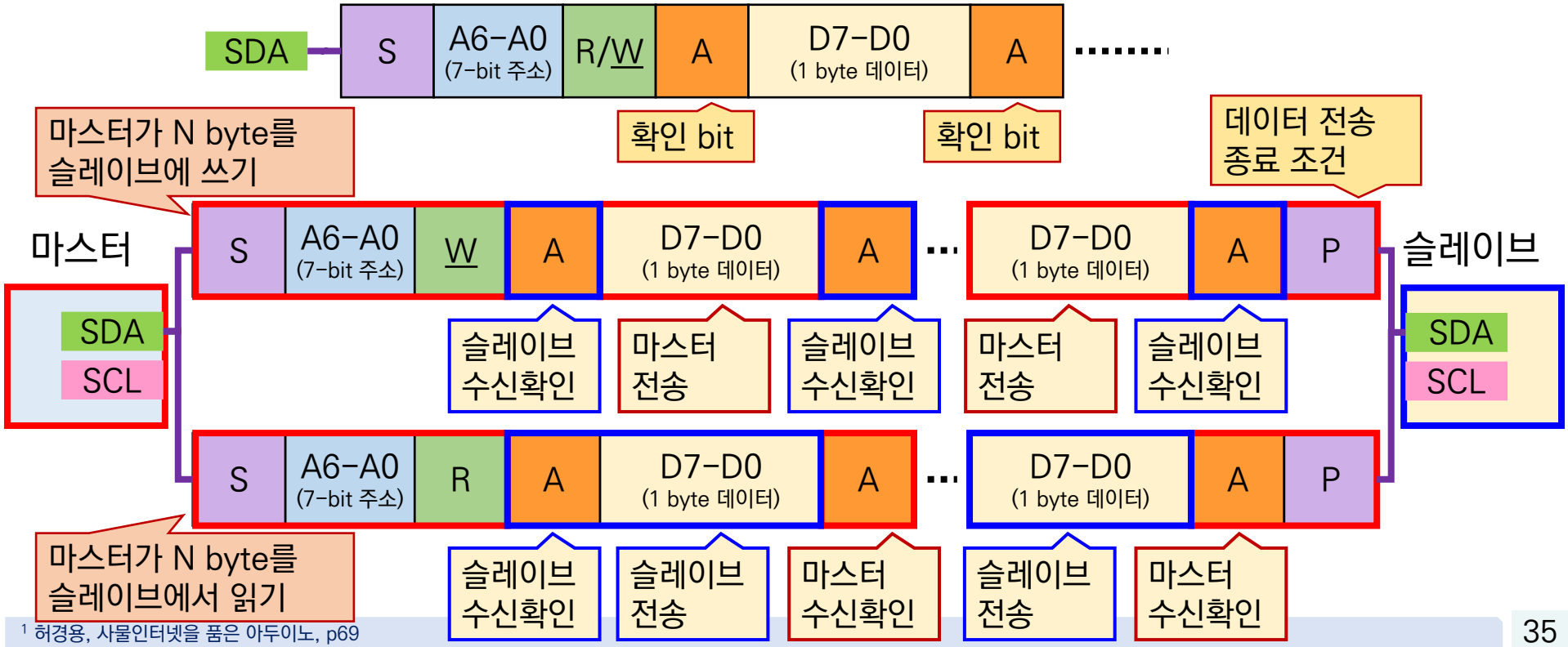



[1] 허경용, 사물인터넷을 품은 아두이노, p69

# I²C 데이터 전송 방식[1]

## ■ 데이터 수신 여부 확인 (ACK/NACK)

- 수신 측은 데이터를 잘 받았음을 송신 측에 알리기 위해 1 bit를 전송
  - ✓ 잘 받았을 때 '오류 없음 (acknowledgement)'을 의미하는 ACK (LOW) 전송
  - ✓ 수신 과정에서 문제가 있었을 때 '오류 있음 (negative acknowledgement)'을 의미하는 NACK (HIGH) 전송
- 주소 (1 byte)와 데이터 (1 byte) 전송 때 모두 수신측은 확인 bit 전송

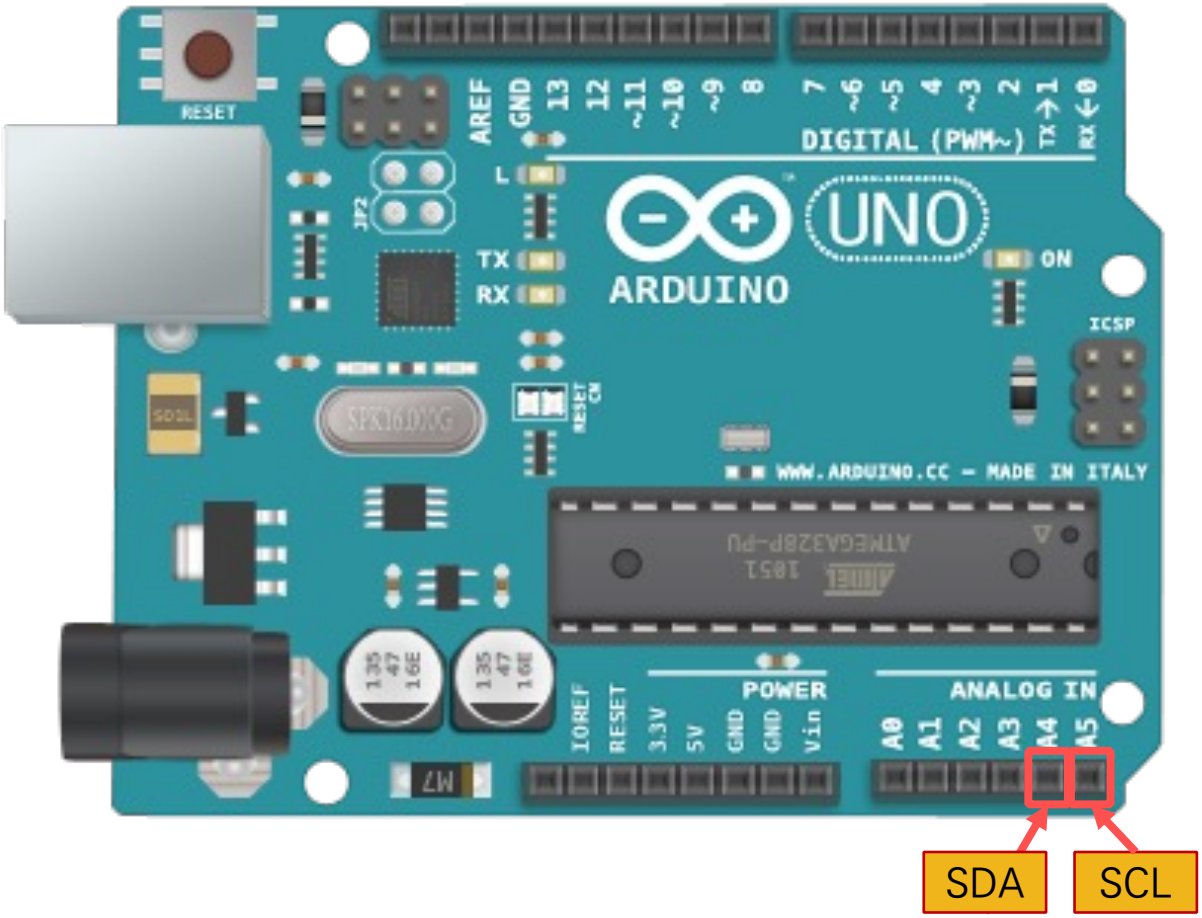



[1] 허경용, 사물인터넷을 품은 아두이노, p69

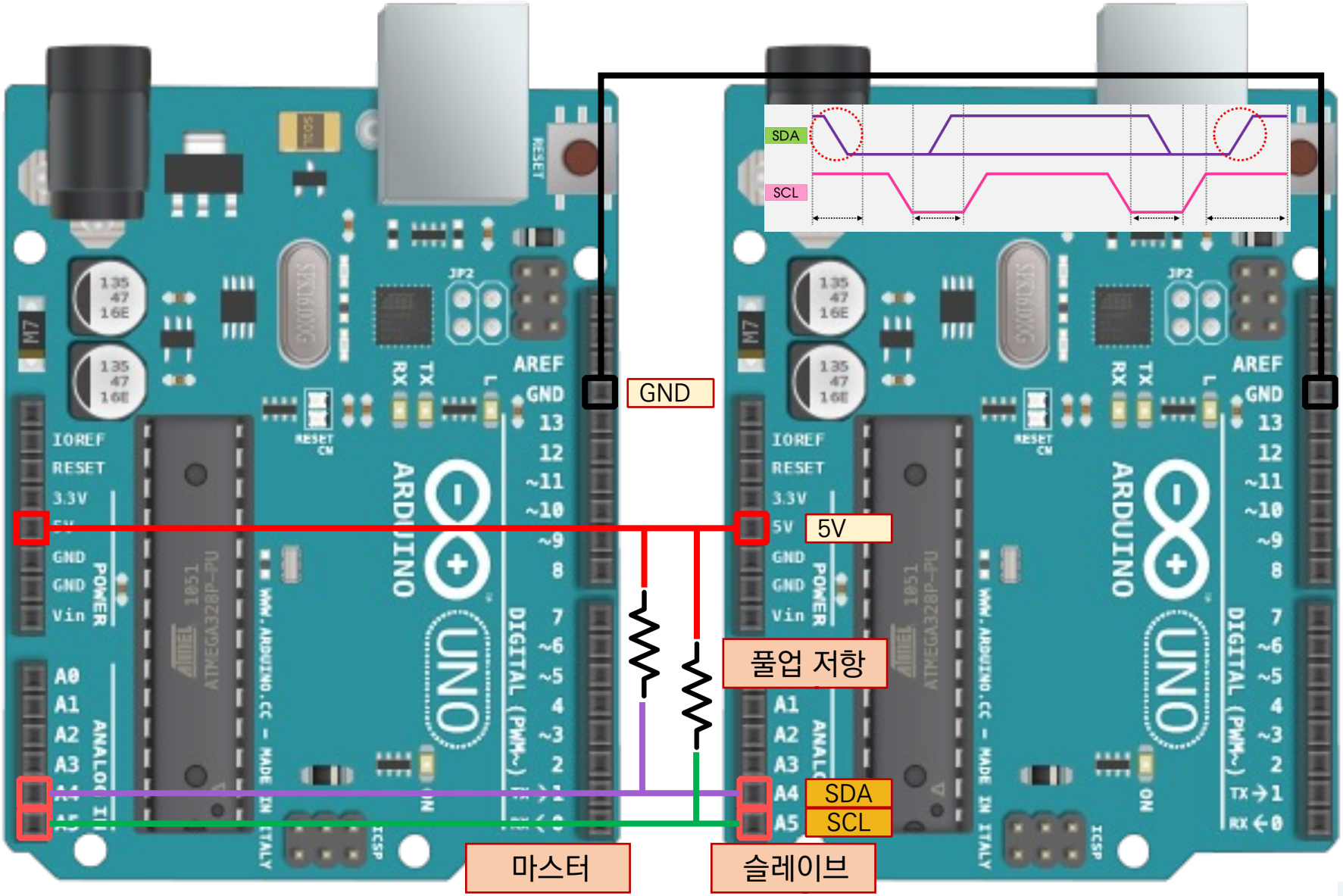
# 아두이노 $I^2C$ 연결

SCL, SDA, GND

# 아두이노 우노 I²C 예약 핀 번호

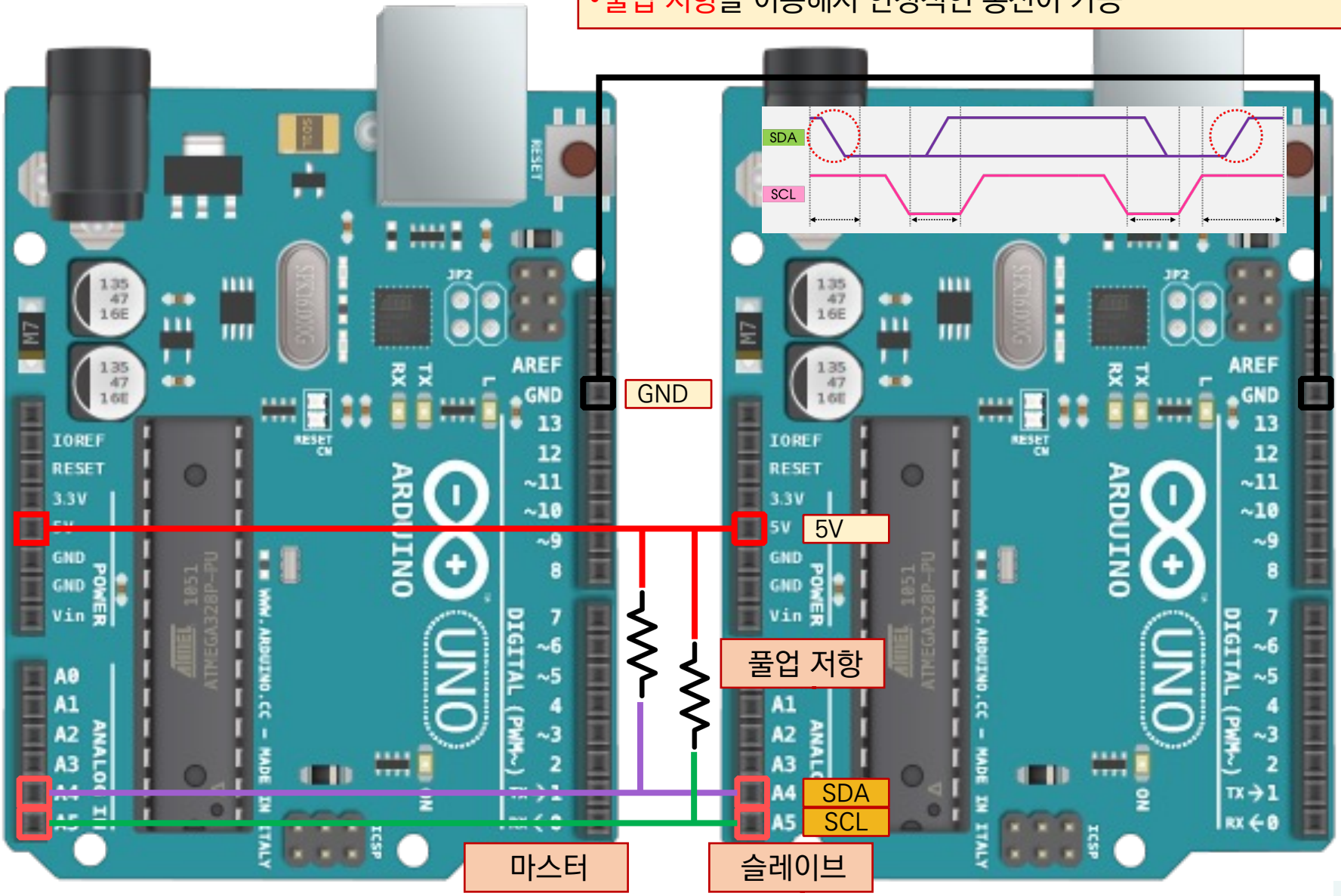


# 아두이노 우노 I²C 예약 핀 번호



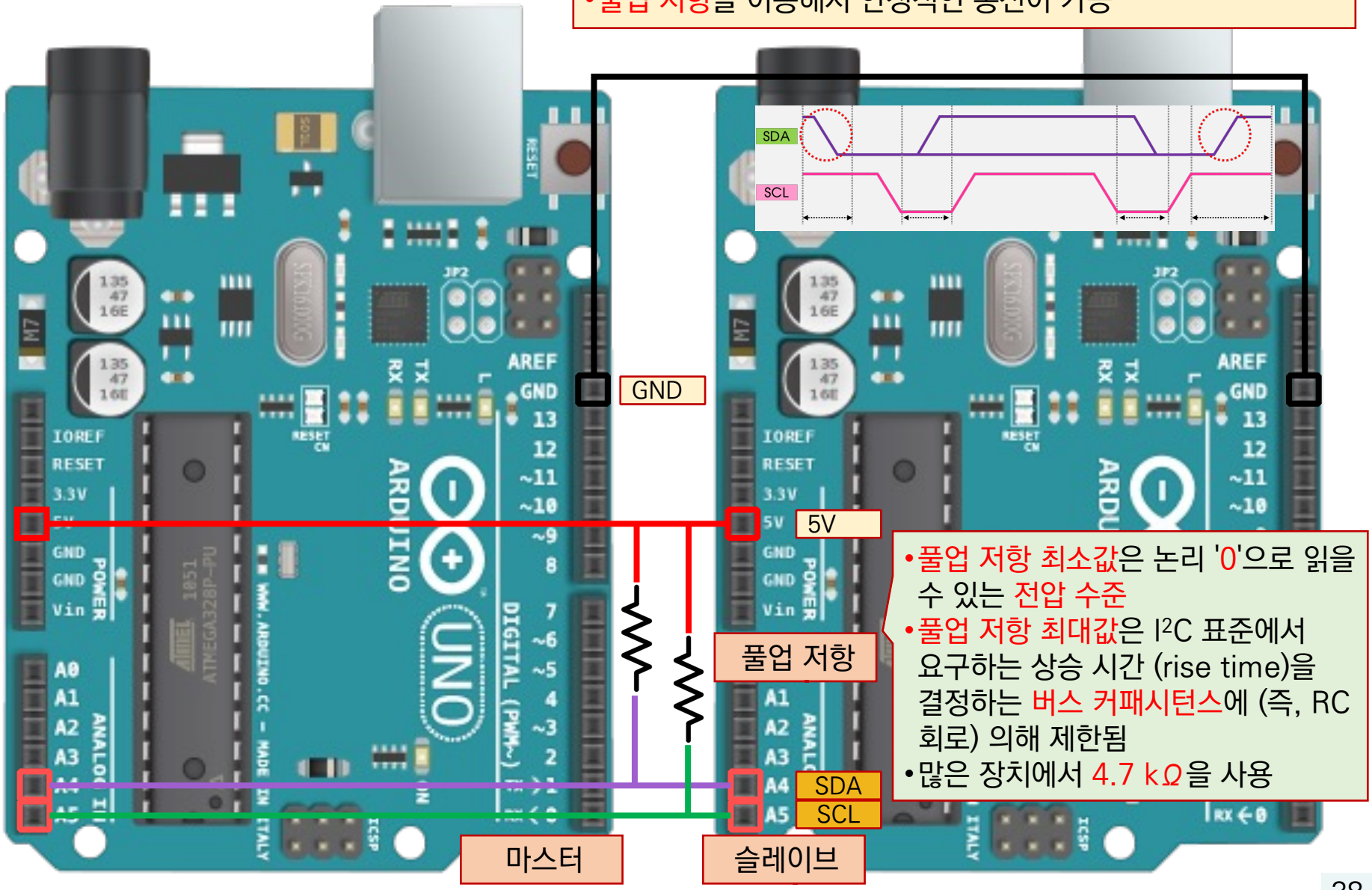
# 아두이노 우노 I²C 예약 핀 번호

- SCL과 SDA 은 사용되지 않는 경우 HIGH를 유지
- I²C는 출력단의 내부가 '오픈 드레인' 구조 (미완성 상태) 이므로 다른 회로와 연결되지 않은 경우 플로팅 상태
- 풀업 저항을 이용해서 안정적인 통신이 가능



# 아두이노 우노 I²C 예약 핀 번호

- SCL과 SDA 은 사용되지 않는 경우 HIGH를 유지
- I²C는 출력단의 내부가 '오픈 드레인' 구조 (미완성 상태) 이므로 다른 회로와 연결되지 않은 경우 플로팅 상태
- 풀업 저항을 이용해서 안정적인 통신이 가능




- 풀업 저항 최소값은 논리 '0'으로 읽을 수 있는 전압 수준
- 풀업 저항 최대값은 I²C 표준에서 요구하는 상승 시간 (rise time)을 결정하는 버스 커패시턴스에 (즉, RC 회로) 의해 제한됨
- 많은 장치에서 4.7 kΩ을 사용

# 아두이노 $I^2C$ 통신

Wire.h

# I²C 라이브러리[1]

## ■ I²C 통신을 위한 라이브러리 제공: `Wire.h`

- I²C 기본 설정 함수와 데이터 전송 함수
- I²C 오브젝트는 `TwoWire`[1]에서 한 개만 생성 가능, 전역객체 `Wire`

## ■ 주요 멤버

- `begin()`: I²C 통신 초기화
  - ✓ 인수로 주소가 있으면 슬레이브, 없으면 마스터로 I²C 버스에 참여함을 의미
- `requestFrom()`: 마스터가 특정 슬레이브에게 지정한 양의 데이터를 요청
- `beginTransmission()`: 특정 슬레이브에게 데이터 전송 시작
  - ✓ 실제 전송은 `write()`, `endTransmission()` 호출 후 일어남
- `endTransmission()`: 큐에 기록된 데이터를 슬레이브에게 전송하며 종료
  - ✓ 반환값: 0 (전송 성공), 1 (전송 데이터 버퍼 초과), 2 (주소 전송 후 NACK 수신), 3 (데이터 전송 후 NACK 수신), 4 (기타 TWI 오류)
- `write()`: (마스터 또는 슬레이브에게) 전송할 데이터를 큐에 기록

[1] https://www.arduino.cc/en/Reference/Wire
[2] I²C 사용 과정에서 필립스와 법적 충돌을 피하기 위해 많은 제조 회사가 자기만의 이름을 사용하나 기본 내용은 동일, 예, Atmel은 TWI (Two Wire Interface)를 사용

# I²C 라이브러리[1]

## ■ I²C 통신을 위한 라이브러리 제공: Wire.h

- I²C 기본 설정 함수와 데이터 전송 함수
- I²C 오브젝트는 TwoWire[1]에서 한 개만 생성 가능, 전역객체 Wire

## ■ 주요 멤버

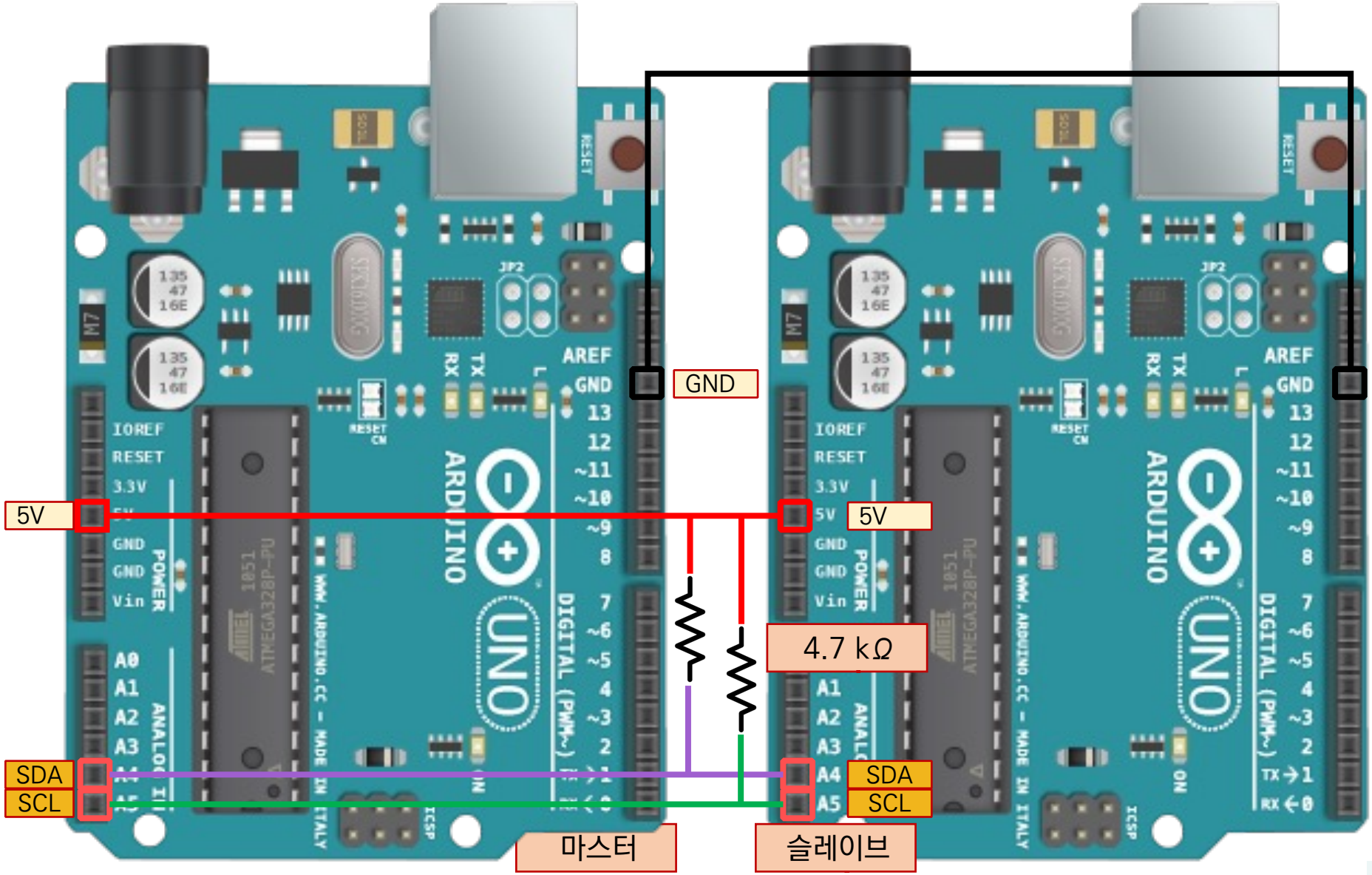
- available(): read() 로 읽어 들일 수 있는 유효한 바이트 수 반환
  - ✓ 마스터가 requestFrom()을 통해 슬레이브에게 데이터를 요청 한 후, 슬레이브로부터 실제 도창학 데이터가 있는지 검사하는 데 사용
  - ✓ 슬레이브가 onReceive() 내에서 사용
- read(): (마스터 또는 슬레이브가) 전송한 데이터를 읽어 들이는데 사용
  - ✓ 마스터가 request() 호출에 의해 슬레이브가 전송한 데이터를 읽어서 반환
  - ✓ 마스터가 전송한 데이터를 슬레이브가 읽어 들일 때도 사용
- onRequest(): 마스터로부터 데이터 요청을 받았을 때 호출되는 핸들러 등록
- onReceive(): 마스터로부터 데이터를 수신했을 때 호출되는 핸들러를 등록
  - ✓ 핸들러는 인수로 마스터로부터 수신하는 데이터 바이트 수를 가짐
  - ✓ 예, void myHandler(int numBytes)

[1] https://www.arduino.cc/en/Reference/Wire
[2] I²C 이름 사용 과정에서 필립스와 법적 충돌을 피하기 위해 많은 제조 회사가 자기만의 이름을 사용, 기술 내용은 동일, 예 Atmel은 TWI (Two Wire Interface) 사용

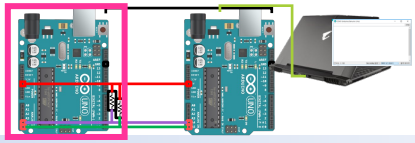
# $I^2C$ 통신 예제

1:1 단방향 문자열 송신

# I²C 통신 예제 1:1 단방향 문자열 송신 (마스터-슬레이브 연결)



# I²C 통신 예제 1:1 단방향 문자열 송신 (마스터 스케치)

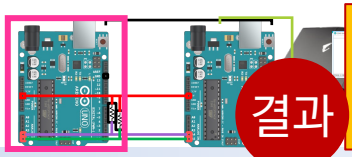


Practice

```
#include <Wire.h>

#define sv  8

byte i = 0;

void setup() {
  Wire.begin();
}

void loop() {
  const byte *p = "Slave!";

  Wire.beginTransmission(sv);


  Wire.write("Hello, ");
  Wire.write(p, strlen(p));
  Wire.write(i++);
  Wire.write('\n');


  Wire.endTransmission();
  delay(1000);
}
```

[1] https://www.arduino.cc/en/Reference/WireBegin
[2] https://www.arduino.cc/en/Reference/WireBeginTransmission
[3] https://www.arduino.cc/en/Reference/WireWrite
[4] https://www.arduino.cc/en/Reference/WireEndTransmission

# I²C 통신 예제 1:1 단방향 문자열 송신 (마스터 스케치)



결과

```
Hello, Slave! 23
Hello, Slave! 24
…
```

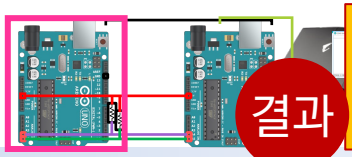
```
#include <Wire.h>

#define sv  8

byte i = 0;

void setup() {
  Wire.begin();
}

void loop() {
  const byte *p = "Slave!";

  Wire.beginTransmission(sv);


  Wire.write("Hello, ");
  Wire.write(p, strlen(p));
  Wire.write(i++);
  Wire.write('\n');


  Wire.endTransmission();
  delay(1000);
}
```

- I²C 라이브러리 Wire.h 포함하여 전역 객체 Wire를 사용
- SPI.h와 달리 Wire.h는 마스터와 슬레이브 모두 사용 가능

- I²C 통신에 참여할 슬레이브 주소 sv를 마스터에서 정의
- 슬레이브에서도 자신의 주소를 알기 위해 sv 정의 필요

[1] https://www.arduino.cc/en/Reference/WireBegin
[2] https://www.arduino.cc/en/Reference/WireBeginTransmission
[3] https://www.arduino.cc/en/Reference/WireWrite
[4] https://www.arduino.cc/en/Reference/WireEndTransmission

# I²C 통신 예제 1:1 단방향 문자열 송신 (마스터 스케치)



결과

```
Hello, Slave! 23
Hello, Slave! 24
…
```

```
#include <Wire.h>

#define sv  8

byte i = 0;

void setup() {
  Wire.begin();
}

void loop() {
  const byte *p = "Slave!";

  Wire.beginTransmission(sv);


  Wire.write("Hello, ");
  Wire.write(p, strlen(p));
  Wire.write(i++);
  Wire.write('\n');


  Wire.endTransmission();
  delay(1000);
}
```

- I²C 라이브러리 Wire.h 포함하여 전역 객체 Wire를 사용
- SPI.h와 달리 Wire.h는 마스터와 슬레이브 모두 사용 가능

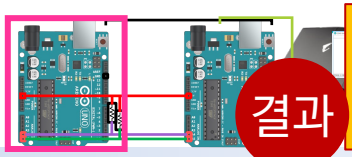
- I²C 통신에 참여할 슬레이브 주소 sv를 마스터에서 정의
- 슬레이브에서도 자신의 주소를 알기 위해 sv 정의 필요

- begin()[1] 는 I²C 통신을 초기화
  인수가 없으면 마스터로서 I²C 통신에 참여를 의미
  7-bit 주소 (<128)가 인수로 주어지면 슬레이브로 참여 의미

- beginTransmission()[2] 는 I²C 통신에 참여하고 있는 슬레이브에게 데이터 전송을 시작함을 의미
- 인수는 데이터를 보낼 슬레이브의 주소

[1] https://www.arduino.cc/en/Reference/WireBegin
[2] https://www.arduino.cc/en/Reference/WireBeginTransmission
[3] https://www.arduino.cc/en/Reference/WireWrite
[4] https://www.arduino.cc/en/Reference/WireEndTransmission

# I²C 통신 예제 1:1 단방향 문자열 송신 (마스터 스케치)



결과

```
Hello, Slave! 23
Hello, Slave! 24
…
```

```
#include <Wire.h>

#define sv  8

byte i = 0;

void setup() {
  Wire.begin();
}

void loop() {
  const byte *p = "Slave!";

  Wire.beginTransmission(sv);


  Wire.write("Hello, ");
  Wire.write(p, strlen(p));
  Wire.write(i++);
  Wire.write('\n');


  Wire.endTransmission();
  delay(1000);
}
```

- I²C 라이브러리 Wire.h 포함하여 전역 객체 Wire를 사용
- SPI.h와 달리 Wire.h는 마스터와 슬레이브 모두 사용 가능

- I²C 통신에 참여할 슬레이브 주소 sv를 마스터에서 정의
- 슬레이브에서도 자신의 주소를 알기 위해 sv 정의 필요

- begin()[1] 는 I²C 통신을 초기화
  인수가 없으면 마스터로서 I²C 통신에 참여를 의미
  7-bit 주소 (<128)가 인수로 주어지면 슬레이브로 참여 의미

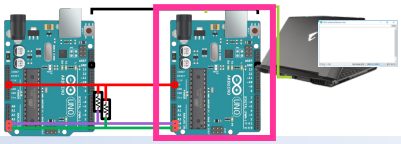
- beginTransmission()[2] 는 I²C 통신에 참여하고 있는 슬레이브에게 데이터 전송을 시작함을 의미
- 인수는 데이터를 보낼 슬레이브의 주소

- write()[3] 는 마스터가 슬레이브로 전송할 데이터를 큐에 넣는 작업을 수행하며 세 가지 형태가 있음 (즉, 오버로드)
  write(문자열 상수)  // 예, write("Hello, ");
  write(byte 배열, 길이) // 예, write(p, strlen(p));
  write(1-byte 값)  // 예, write('\n');
- write()는 마스터의 요청에 따라 슬레이브에서 마스터로 데이터를 전송할 때도 사용 (다음 예제 참조)

- endTransmission()[4] 는 beginTransmission()으로 시작된 슬레이브에 대한 데이터 전송을 종료하며 동시에,
- write()로 큐에 넣어 놓은 데이터를 슬레이브에게 전송

[1] https://www.arduino.cc/en/Reference/WireBegin
[2] https://www.arduino.cc/en/Reference/WireBeginTransmission
[3] https://www.arduino.cc/en/Reference/WireWrite
[4] https://www.arduino.cc/en/Reference/WireEndTransmission

# I²C 통신 예제 1:1 단방향 문자열 송신 (슬레이브 스케치)

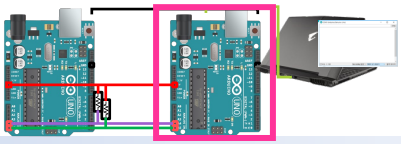


Practice

```
#include <Wire.h>
#define sv  8

byte i = 0;

void setup() {
  Wire.begin(sv);
  Wire.onReceive(receiveEvent);
  Serial.begin(9600);
}

void loop() {
  delay(100);
}

void receiveEvent(int howMany) {
  while (Wire.available() > 2) {
    char c = Wire.read();
    Serial.print(c);
  }
  int x = Wire.read();
  Serial.print(x);
  char c = Wire.read();
  Serial.print(c);
}
```

[1] https://www.arduino.cc/en/Reference/WireOnReceive
[2] https://www.arduino.cc/en/Reference/WireAvailable
[3] https://www.arduino.cc/en/Reference/WireRead

# I²C 통신 예제 1:1 단방향 문자열 송신 (슬레이브 스케치)



Practice

- 슬레이브가 참여할 I²C 통신에서 자신의 주소 sv 정의
- 마스터에서도 참여하는 슬레이브 주소 sv 를 알고 있음

- begin() 의 인수로 sv (자신에게 부여된 슬레이브 주소)를 사용하여 I²C 통신에 참여 시작
- onReceive()[1] 는 마스터가 슬레이브에게 데이터를 전송할 때 (즉 endTransmission() 실행) 대응 핸들러 함수를 등록
- 핸들러 함수는 슬레이브가 데이터를 받았을 때 호출됨
- 핸들러 함수 인수로 마스터로부터 받은 바이트 수가 제공
  예, void receiveEvent(int numBytes)

```
#include <Wire.h>
#define sv  8

byte i = 0;

void setup() {
  Wire.begin(sv);
  Wire.onReceive(receiveEvent);
  Serial.begin(9600);
}

void loop() {
  delay(100);
}

void receiveEvent(int howMany) {
  while (Wire.available() > 2) {
    char c = Wire.read();
    Serial.print(c);
  }
  int x = Wire.read();
  Serial.print(x);
  char c = Wire.read();
  Serial.print(c);
}
```

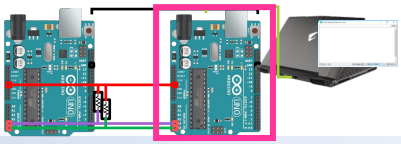
결과

```
Hello, Slave! 23
Hello, Slave! 24
…
```

- howMany에는 마스터로부터 받은 바이트 수가 저장

[1] https://www.arduino.cc/en/Reference/WireOnReceive
[2] https://www.arduino.cc/en/Reference/WireAvailable
[3] https://www.arduino.cc/en/Reference/WireRead

# I²C 통신 예제 1:1 단방향 문자열 송신 (슬레이브 스케치)



Practice

- 슬레이브가 참여할 I²C 통신에서 자신의 주소 sv 정의
- 마스터에서도 참여하는 슬레이브 주소 sv 를 알고 있음

- begin() 의 인수로 sv (자신에게 부여된 슬레이브 주소)를 사용하여 I²C 통신에 참여 시작
- onReceive()[1] 는 마스터가 슬레이브에게 데이터를 전송할 때 (즉 endTransmission() 실행) 대응 핸들러 함수를 등록
- 핸들러 함수는 슬레이브가 데이터를 받았을 때 호출됨
- 핸들러 함수 인수로 마스터로부터 받은 바이트 수가 제공
  예, void receiveEvent(int numBytes)

- available()[2] 는 read()를 사용해 읽어 들일 수 있는 바이트 개수를 반환하며 마스터와 슬레이브에서 모두 사용
- 마스터: requestFrom()을 호출 한 후 사용되어야 함
- 슬레이브: onReceive() 핸들러 안에서 사용되어야 함

```
#include <Wire.h>
#define sv  8

byte i = 0;

void setup() {
  Wire.begin(sv);
  Wire.onReceive(receiveEvent);
  Serial.begin(9600);
}

void loop() {
  delay(100);
}

void receiveEvent(int howMany) {
  while (Wire.available() > 2) {
    char c = Wire.read();
    Serial.print(c);
  }
  int x = Wire.read();
  Serial.print(x);
  char c = Wire.read();
  Serial.print(c);
}
```

결과

```
Hello, Slave! 23
Hello, Slave! 24
…
```

- howMany에는 마스터로부터 받은 바이트 수가 저장

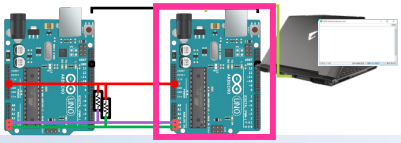
- i 와 '\n'를 남기고 처리

마스터

```
Wire.write("Hello, ");
Wire.write(p, strlen(p));
Wire.write(i++);
Wire.write('\n');
```

[1] https://www.arduino.cc/en/Reference/WireOnReceive
[2] https://www.arduino.cc/en/Reference/WireAvailable
[3] https://www.arduino.cc/en/Reference/WireRead

# I²C 통신 예제 1:1 단방향 문자열 송신 (슬레이브 스케치)



Practice

- 슬레이브가 참여할 I²C 통신에서 자신의 주소 sv 정의
- 마스터에서도 참여하는 슬레이브 주소 sv 를 알고 있음

- begin() 의 인수로 sv (자신에게 부여된 슬레이브 주소)를 사용하여 I²C 통신에 참여 시작
- onReceive()[1] 는 마스터가 슬레이브에게 데이터를 전송할 때 (즉 endTransmission() 실행) 대응 핸들러 함수를 등록
- 핸들러 함수는 슬레이브가 데이터를 받았을 때 호출됨
- 핸들러 함수 인수로 마스터로부터 받은 바이트 수가 제공
  예, void receiveEvent(int numBytes)

- available()[2] 는 read()를 사용해 읽어 들일 수 있는 바이트 개수를 반환하며 마스터와 슬레이브에서 모두 사용
- 마스터: requestFrom()을 호출 한 후 사용되어야 함
- 슬레이브: onReceive() 핸들러 안에서 사용되어야 함

- read()[3] 는 I²C 통신을 통해 전송한 데이터를 읽는 함수로서 마스터와 슬레이브에서 모두 사용
- 마스터: requestFrom()을 호출 한 후 슬레이브가 마스터에게 전송한 데이터 1 byte를 읽어 들이는데 사용
- 슬레이브: (onReceive() 핸들러 안에서) 마스터가 슬레이브에게 전송한 데이터 1 byte를 읽어 들이는데 사용

```
#include <Wire.h>
#define sv  8

byte i = 0;

void setup() {
  Wire.begin(sv);
  Wire.onReceive(receiveEvent);
  Serial.begin(9600);
}

void loop() {
  delay(100);
}

void receiveEvent(int howMany) {
  while (Wire.available() > 2) {
    char c = Wire.read();
    Serial.print(c);
  }
  int x = Wire.read();
  Serial.print(x);
  char c = Wire.read();
  Serial.print(c);
}
```

결과

```
Hello, Slave! 23
Hello, Slave! 24
…
```

- howMany에는 마스터로부터 받은 바이트 수가 저장

- i 와 '\n'를 남기고 처리

마스터

```
Wire.write("Hello, ");
Wire.write(p, strlen(p));
Wire.write(i++);
Wire.write('\n');
```

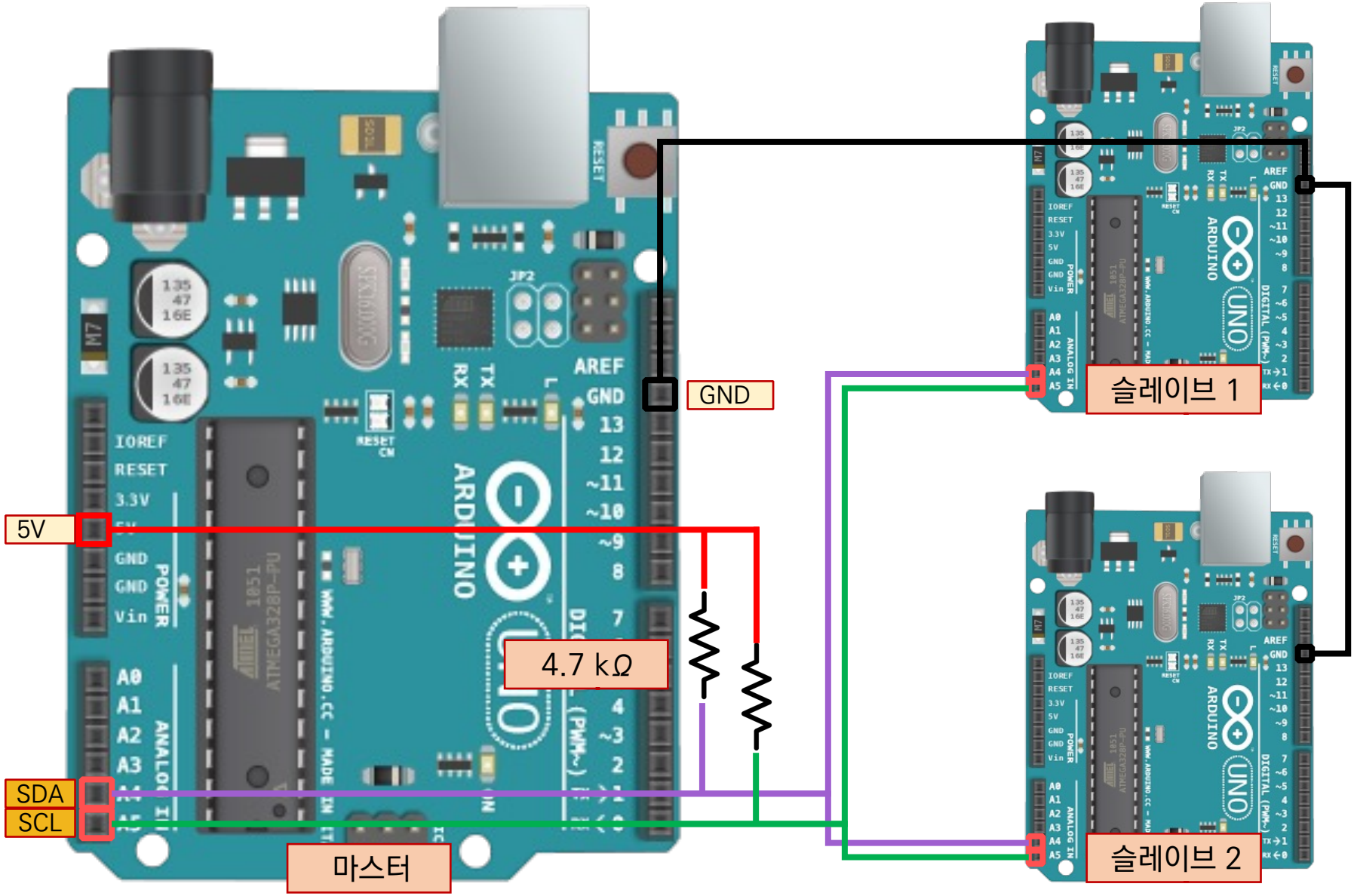
- 마스터에서 전송하는 데이터 마지막에 숫자 i와 '\n'이 있음을 알고 있기 때문에 가능한 수신 방법

[1] https://www.arduino.cc/en/Reference/WireOnReceive
[2] https://www.arduino.cc/en/Reference/WireAvailable
[3] https://www.arduino.cc/en/Reference/WireRead

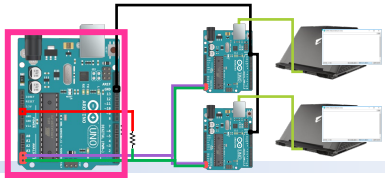
# $I^2C$ 통신 예제

1:2 단방향 문자열 송신

# I²C 통신 예제 1:2 단방향 문자열 송신 (마스터-슬레이브 연결)



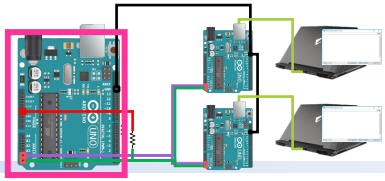
# I²C 통신 예제 1:2 단방향 문자열 송신 (마스터 스케치)



Practice

```
#include <Wire.h>

#define sv1  8
#define sv2  9

byte i = 0;

void setup() {
  Wire.begin();
}

void loop() {

  Wire.beginTransmission(sv1);
  Wire.write("Hello, S1!");
  Wire.write(i++);
  Wire.write('\n');
  Wire.endTransmission();


  Wire.beginTransmission(sv2);
  Wire.write("Hello, S2!");
  Wire.write(i++);
  Wire.write('\n');
  Wire.endTransmission();

  delay(1000);
}
```

# I²C 통신 예제 1:2 단방향 문자열 송신 (마스터 스케치)



Practice

```
#include <Wire.h>

#define sv1  8
#define sv2  9

byte i = 0;

void setup() {
  Wire.begin();
}

void loop() {

  Wire.beginTransmission(sv1);
  Wire.write("Hello, S1!");
  Wire.write(i++);
  Wire.write('\n');
  Wire.endTransmission();


  Wire.beginTransmission(sv2);
  Wire.write("Hello, S2!");
  Wire.write(i++);
  Wire.write('\n');
  Wire.endTransmission();

  delay(1000);
}
```

- I²C 통신에 참여할 슬레이브 주소 sv1, sv2 를 마스터에서 정의하고, 동일한 주소가 슬레이브 스케치에서도 정의 필요

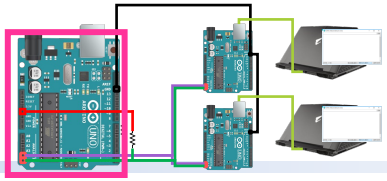
결과

```
Hello, S1! 23
Hello, S1! 25
…
```

결과

```
Hello, S2! 24
Hello, S2! 26
…
```

# I²C 통신 예제 1:2 단방향 문자열 송신 (마스터 스케치)



Practice

```
#include <Wire.h>

#define sv1  8
#define sv2  9

byte i = 0;

void setup() {
  Wire.begin();
}

void loop() {

  Wire.beginTransmission(sv1);
  Wire.write("Hello, S1!");
  Wire.write(i++);
  Wire.write('\n');
  Wire.endTransmission();


  Wire.beginTransmission(sv2);
  Wire.write("Hello, S2!");
  Wire.write(i++);
  Wire.write('\n');
  Wire.endTransmission();

  delay(1000);
}
```

- I²C 통신에 참여할 슬레이브 주소 sv1, sv2 를 마스터에서 정의하고, 동일한 주소가 슬레이브 스케치에서도 정의 필요

결과

```
Hello, S1! 23
Hello, S1! 25
…
```

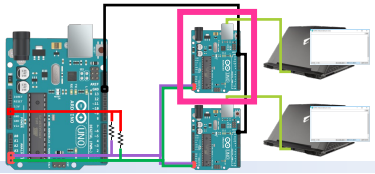
결과

```
Hello, S2! 24
Hello, S2! 26
…
```

- beginTransmission() 에 인수로 첫 번째 슬레이브 주소 sv1을 주면서 I²C 데이터 전송을 시작함을 의미

- beginTransmission() 에 인수로 두 번째 슬레이브 주소 sv2을 주면서 I²C 데이터 전송을 시작함을 의미

# I²C 통신 예제 1:2 단방향 문자열 송신 (슬레이브 1 스케치)

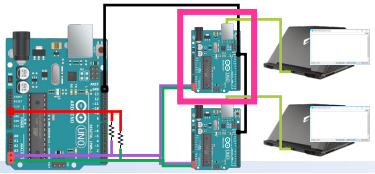


Practice

```
#include <Wire.h>
#define sv1  8

byte i = 0;

void setup() {
  Wire.begin(sv1);
  Wire.onReceive(receiveEvent);
  Serial.begin(9600);
}

void loop() {
  delay(100);
}

void receiveEvent(int howMany) {
  while (Wire.available() > 2) {
    char c = Wire.read();
    Serial.print(c);
  }
  int x = Wire.read();
  Serial.print(x);
  char c = Wire.read();
  Serial.print(c);
}
```

- howMany에는 마스터로부터 받은 바이트 수가 저장
- i 와 '\n'를 남기고 처리
- 마스터에서 전송하는 데이터 마지막에 숫자 i와 '\n'이 있음을 알고 있기 때문에 가능한 수신 방법

# I²C 통신 예제 1:2 단방향 문자열 송신 (슬레이브 1 스케치)



Practice

- 슬레이브가 참여할 I²C 통신에서 자신의 주소 sv1 정의
- 마스터에서도 참여하는 슬레이브 주소 sv1 를 알고 있음

결과

```
Hello, S1! 23
Hello, S1! 25
…
```

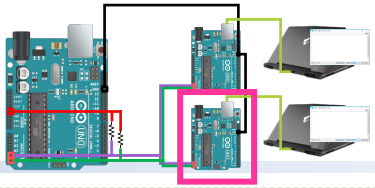
결과

```
Hello, S2! 24
Hello, S2! 26
…
```

```
#include <Wire.h>
#define sv1  8

byte i = 0;

void setup() {
  Wire.begin(sv1);
  Wire.onReceive(receiveEvent);
  Serial.begin(9600);
}

void loop() {
  delay(100);
}

void receiveEvent(int howMany) {
  while (Wire.available() > 2) {
    char c = Wire.read();
    Serial.print(c);
  }
  int x = Wire.read();
  Serial.print(x);
  char c = Wire.read();
  Serial.print(c);
}
```

- howMany에는 마스터로부터 받은 바이트 수가 저장
- i 와 '\n'를 남기고 처리
- 마스터에서 전송하는 데이터 마지막에 숫자 i와 '\n'이 있음을 알고 있기 때문에 가능한 수신 방법

# I²C 통신 예제 1:2 단방향 문자열 송신 (슬레이브 2 스케치)

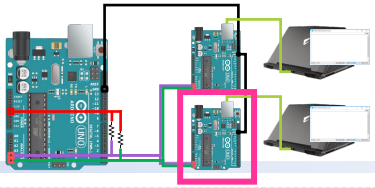


Practice

```
#include <Wire.h>
#define sv2  9

byte i = 0;

void setup() {
  Wire.begin(sv2);
  Wire.onReceive(receiveEvent);
  Serial.begin(9600);
}

void loop() {
  delay(100);
}

void receiveEvent(int howMany) {
  while (Wire.available() > 2) {
    char c = Wire.read();
    Serial.print(c);
  }
  int x = Wire.read();
  Serial.print(x);
  char c = Wire.read();
  Serial.print(c);
}
```

- howMany에는 마스터로부터 받은 바이트 수가 저장
- i 와 '\n'를 남기고 처리
- 마스터에서 전송하는 데이터 마지막에 숫자 i와 '\n'이 있음을 알고 있기 때문에 가능한 수신 방법

# I²C 통신 예제 1:2 단방향 문자열 송신 (슬레이브 2 스케치)



Practice

- 슬레이브가 참여할 I²C 통신에서 자신의 주소 sv2 정의
- 마스터에서도 참여하는 슬레이브 주소 sv2 를 알고 있음

결과

```
Hello, S1! 23
Hello, S1! 25
…
```

결과

```
Hello, S2! 24
Hello, S2! 26
…
```

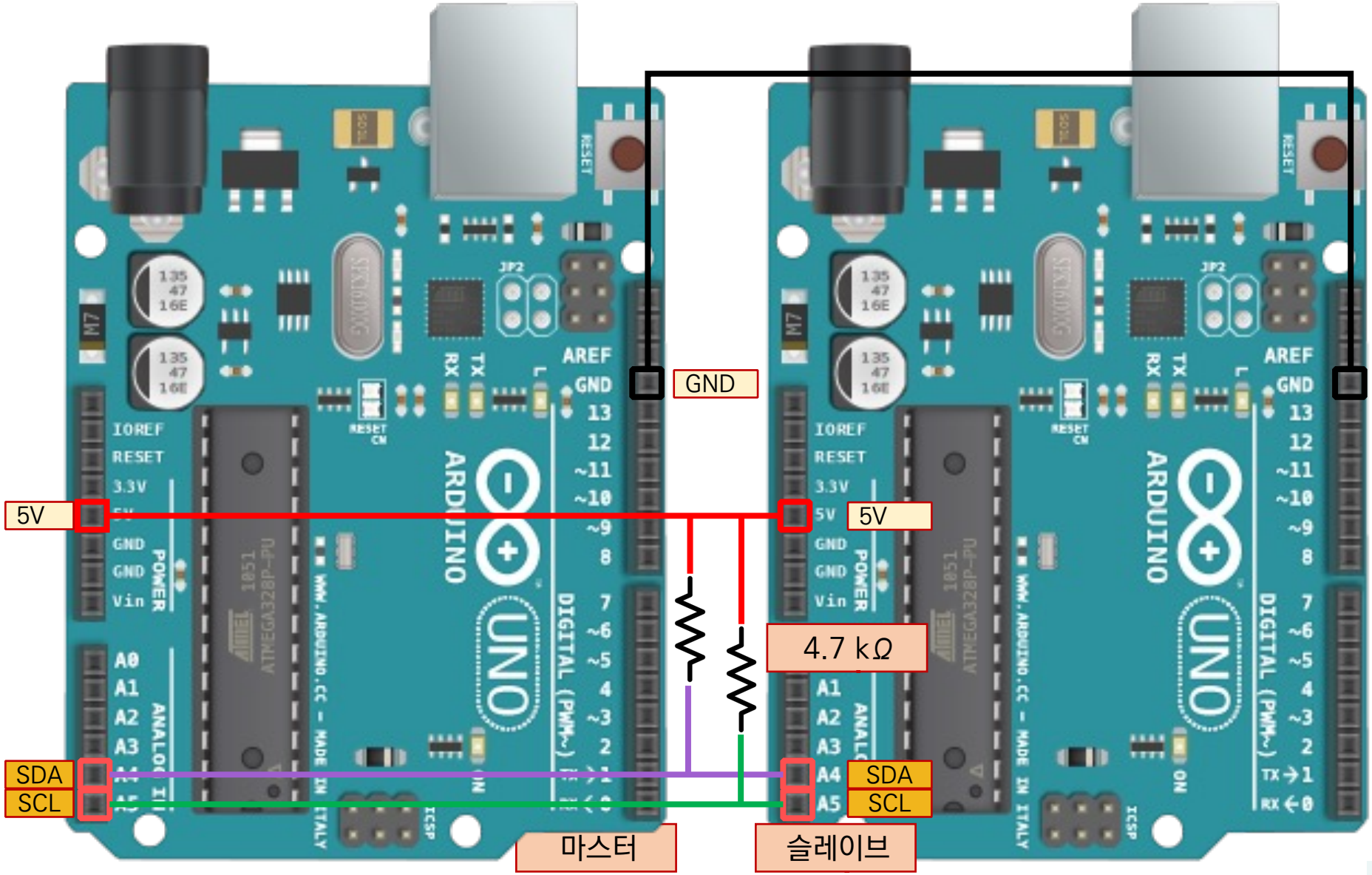
```
#include <Wire.h>
#define sv2  9

byte i = 0;

void setup() {
  Wire.begin(sv2);
  Wire.onReceive(receiveEvent);
  Serial.begin(9600);
}

void loop() {
  delay(100);
}

void receiveEvent(int howMany) {
  while (Wire.available() > 2) {
    char c = Wire.read();
    Serial.print(c);
  }
  int x = Wire.read();
  Serial.print(x);
  char c = Wire.read();
  Serial.print(c);
}
```

- howMany에는 마스터로부터 받은 바이트 수가 저장
- i 와 '\n'를 남기고 처리
- 마스터에서 전송하는 데이터 마지막에 숫자 i와 '\n'이 있음을 알고 있기 때문에 가능한 수신 방법

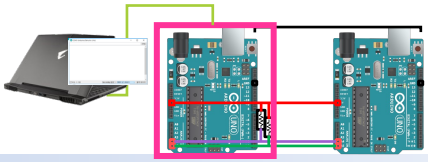
# $I^2C$ 통신 예제

1:1 바이트 데이터 요청 (센싱)

# I²C 통신 예제 1:1 바이트 데이터 요청 (마스터-슬레이브 연결)



# I²C 통신 예제 1:1 바이트 데이터 요청 (마스터 스케치)

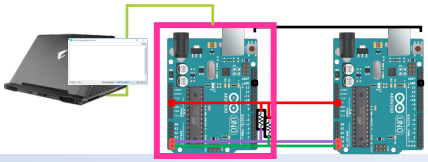


Practice

```
#include <Wire.h>

#define sv  8

void setup() {
  Wire.begin();
  Serial.begin(9600);
}

void loop() {

  Wire.requestFrom(sv, 1);


  while (Wire.available()) {
    Serial.println("Sensed data:");


    byte c = Wire.read();


    Serial.println(c);
  }


  delay(1000);
}
```

[1] https://www.arduino.cc/en/Reference/WireRequestFrom
[2] https://www.arduino.cc/en/Reference/WireAvailable
[3] https://www.arduino.cc/en/Reference/WireRead

# I²C 통신 예제 1:1 바이트 데이터 요청 (마스터 스케치)



```
Sensed data:
39
Sensed data:
56
…
```

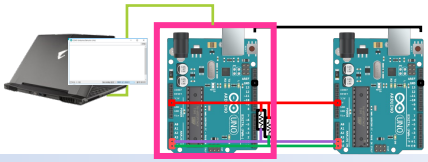
결과

```
#include <Wire.h>

#define sv  8

void setup() {
  Wire.begin();
  Serial.begin(9600);
}

void loop() {

  Wire.requestFrom(sv, 1);


  while (Wire.available()) {
    Serial.println("Sensed data:");


    byte c = Wire.read();


    Serial.println(c);
  }


  delay(1000);
}
```

- I²C 통신에 참여할 슬레이브 주소 sv를 마스터에서 정의
- 슬레이브에서도 자신의 주소를 알기 위해 sv 정의 필요

[1] https://www.arduino.cc/en/Reference/WireRequestFrom
[2] https://www.arduino.cc/en/Reference/WireAvailable
[3] https://www.arduino.cc/en/Reference/WireRead

# I²C 통신 예제 1:1 바이트 데이터 요청 (마스터 스케치)



```
Sensed data:
39
Sensed data:
56
…
```

결과

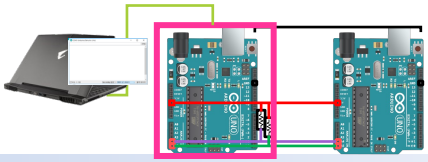
```
#include <Wire.h>

#define sv  8

void setup() {
  Wire.begin();
  Serial.begin(9600);
}

void loop() {

  Wire.requestFrom(sv, 1);


  while (Wire.available()) {
    Serial.println("Sensed data:");


    byte c = Wire.read();


    Serial.println(c);
  }


  delay(1000);
}
```

- I²C 통신에 참여할 슬레이브 주소 sv를 마스터에서 정의
- 슬레이브에서도 자신의 주소를 알기 위해 sv 정의 필요

- requestFrom()[1] 은 마스터가 슬레이브에게 데이터 전송을 요구할 때 사용
  첫 번째 인수는 데이터 전송을 요구할 슬레이브 주소
  두 번째 인수는 슬레이브로부터 전송 받을 바이트 수

[1] https://www.arduino.cc/en/Reference/WireRequestFrom
[2] https://www.arduino.cc/en/Reference/WireAvailable
[3] https://www.arduino.cc/en/Reference/WireRead

# I²C 통신 예제 1:1 바이트 데이터 요청 (마스터 스케치)



```
Sensed data:
39
Sensed data:
56
…
```

결과

```
#include <Wire.h>

#define sv  8

void setup() {
  Wire.begin();
  Serial.begin(9600);
}

void loop() {

  Wire.requestFrom(sv, 1);


  while (Wire.available()) {
    Serial.println("Sensed data:");


    byte c = Wire.read();


    Serial.println(c);
  }


  delay(1000);
}
```

- I²C 통신에 참여할 슬레이브 주소 sv를 마스터에서 정의
- 슬레이브에서도 자신의 주소를 알기 위해 sv 정의 필요

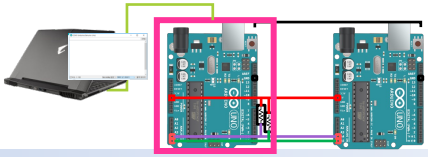
- requestFrom()[1] 은 마스터가 슬레이브에게 데이터 전송을 요구할 때 사용
  첫 번째 인수는 데이터 전송을 요구할 슬레이브 주소
  두 번째 인수는 슬레이브로부터 전송 받을 바이트 수

- available()[2] 는 read()를 사용해 읽어 들일 수 있는 바이트 개수를 반환하며 마스터와 슬레이브에서 모두 사용
- 마스터: requestFrom()을 호출 한 후 사용되어야 함
- 슬레이브: onReceive() 핸들러 안에서 사용되어야 함

- read()[3] 는 I²C 통신을 통해 전송한 데이터를 읽는 함수로서 마스터와 슬레이브에서 모두 사용
- 마스터: requestFrom()을 호출 한 후 슬레이브가 마스터에게 전송한 데이터 1 byte를 읽어 들이는데 사용
- 슬레이브: (onReceive() 핸들러 안에서) 마스터가 슬레이브에게 전송한 데이터 1 byte를 읽어 들이는데 사용

[1] https://www.arduino.cc/en/Reference/WireRequestFrom
[2] https://www.arduino.cc/en/Reference/WireAvailable
[3] https://www.arduino.cc/en/Reference/WireRead

# I²C 통신 예제 1:1 바이트 데이터 요청 (슬레이브 스케치)



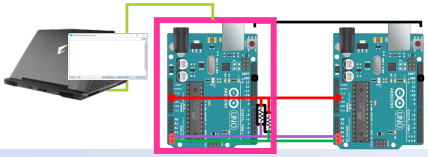
Practice

```
#include <Wire.h>
#define sv  8

byte data = 0;

void setup() {
  Wire.begin(sv);
  Wire.onRequest(requestEvent);
}


void loop() {
  data = (byte)random(0, 100);

  delay(100);
}


void requestEvent() {
  Wire.write(data);
}
```

• random() 함수로 센서 값을 모사

[1] https://www.arduino.cc/en/Reference/WireOnRequest
[2] https://www.arduino.cc/en/Reference/WireWrite

# I²C 통신 예제 1:1 바이트 데이터 요청 (슬레이브 스케치)



```
Sensed data:
39
Sensed data:
56
…
```

결과

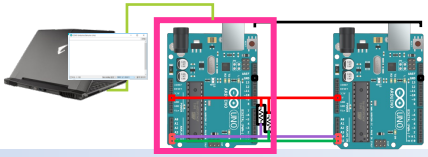
- 슬레이브가 참여할 I²C 통신에서 자신의 주소 sv 정의
- 마스터에서도 참여하는 슬레이브 주소 sv 를 알고 있음

```
#include <Wire.h>
#define sv  8

byte data = 0;

void setup() {
  Wire.begin(sv);
  Wire.onRequest(requestEvent);
}


void loop() {
  data = (byte)random(0, 100);

  delay(100);
}


void requestEvent() {
  Wire.write(data);
}
```

- random() 함수로 센서 값을 모사

[1] https://www.arduino.cc/en/Reference/WireOnRequest
[2] https://www.arduino.cc/en/Reference/WireWrite

# I²C 통신 예제 1:1 바이트 데이터 요청 (슬레이브 스케치)



```
Sensed data:
39
Sensed data:
56
…
```

결과

- 슬레이브가 참여할 I²C 통신에서 자신의 주소 sv 정의
- 마스터에서도 참여하는 슬레이브 주소 sv 를 알고 있음

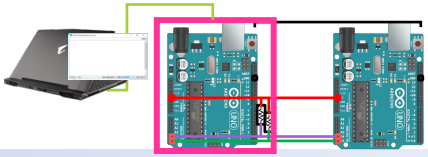
- onRequest()[1] 는 마스터가 슬레이브에게 데이터 전송을 '요청'할 때 (즉 requestFrom() 실행) 대응 핸들러 함수를 등록
- 핸들러 함수는 슬레이브가 데이터 '요청'을 받았을 때 호출됨
- onReceive() 핸들러와는 달리 인수와 반환값으로 아무 것도 가지지 않음
  예, void requestEvent()

```
#include <Wire.h>
#define sv  8

byte data = 0;

void setup() {
  Wire.begin(sv);
  Wire.onRequest(requestEvent);
}


void loop() {
  data = (byte)random(0, 100);

  delay(100);
}


void requestEvent() {
  Wire.write(data);
}
```

- random() 함수로 센서 값을 모사

[1] https://www.arduino.cc/en/Reference/WireOnRequest
[2] https://www.arduino.cc/en/Reference/WireWrite

# I²C 통신 예제 1:1 바이트 데이터 요청 (슬레이브 스케치)



```
Sensed data:
39
Sensed data:
56
…
```

결과

- 슬레이브가 참여할 I²C 통신에서 자신의 주소 sv 정의
- 마스터에서도 참여하는 슬레이브 주소 sv 를 알고 있음

- onRequest()[1] 는 마스터가 슬레이브에게 데이터 전송을 '요청'할 때 (즉 requestFrom() 실행) 대응 핸들러 함수를 등록
- 핸들러 함수는 슬레이브가 데이터 '요청'을 받았을 때 호출됨
- onReceive() 핸들러와는 달리 인수와 반환값으로 아무 것도 가지지 않음
  예, void requestEvent()

- write()[2] 는 마스터의 요청에 따라 슬레이브에서 마스터로 데이터를 전송할 때 사용하며 세 가지 형태가 있음 (즉, 오버로드)
  write(문자열 상수) // 예, write("Hello, ");
  write(byte 배열, 길이) // 예, write(p, strlen(p));
  write(1-byte 값) // 예, write('\n');
- write() 는 마스터가 슬레이브로 전송할 데이터를 큐에 넣는 작업을 수행할 때도 사용 (앞 예제 참조)

```
#include <Wire.h>
#define sv  8

byte data = 0;

void setup() {
  Wire.begin(sv);
  Wire.onRequest(requestEvent);
}


void loop() {
  data = (byte)random(0, 100);

  delay(100);
}


void requestEvent() {
  Wire.write(data);
}
```

- random() 함수로 센서 값을 모사

[1] https://www.arduino.cc/en/Reference/WireOnRequest
[2] https://www.arduino.cc/en/Reference/WireWrite

# SPI vs. $I^2C$

# Compare SPI and $I^2C$ SPI와 I2C 비교

| SPI | $I^2C$ |
| --- | --- |
| SCLK, MOSI, MISO, SS | SCL, SDA |

# Compare SPI and $I^2C$ SPI와 I2C 비교

| SPI | $I^2C$ |
| --- | --- |
| SCLK, MOSI, MISO, SS | SCL, SDA |
| $I^2C$ 보다 빠른 속도 | SPI 보다 느린 속도 |

# Compare SPI and I²C SPI와 I2C 비교

| SPI | I²C |
| --- | --- |
| SCLK, MOSI, MISO, SS | SCL, SDA |
| I²C 보다 빠른 속도 | SPI 보다 느린 속도 |
| I²C 보다 적은 전력 소비 | SPI 보다 많은 전력 소비 |

# Compare SPI and $I^2C$ SPI와 I2C 비교

| SPI | $I^2C$ |
| --- | --- |
| SCLK, MOSI, MISO, SS | SCL, SDA |
| $I^2C$ 보다 빠른 속도 | SPI 보다 느린 속도 |
| $I^2C$ 보다 적은 전력 소비 | SPI 보다 많은 전력 소비 |
| 4개의 연결선 | 2개의 연결선 |

# Compare SPI and I²C SPI와 I2C 비교

| SPI | I²C |
| --- | --- |
| SCLK, MOSI, MISO, SS | SCL, SDA |
| I²C 보다 빠른 속도 | SPI 보다 느린 속도 |
| I²C 보다 적은 전력 소비 | SPI 보다 많은 전력 소비 |
| 4개의 연결선 | 2개의 연결선 |
| 장치가 증가할 때 연결선 SS 증가 | 장치가 증가할 때 추가 연결 없음 |

# Compare SPI and I²C SPI와 I2C 비교

| SPI | I²C |
| --- | --- |
| SCLK, MOSI, MISO, SS | SCL, SDA |
| I²C 보다 빠른 속도 | SPI 보다 느린 속도 |
| I²C 보다 적은 전력 소비 | SPI 보다 많은 전력 소비 |
| 4개의 연결선 | 2개의 연결선 |
| 장치가 증가할 때 연결선 SS 증가 | 장치가 증가할 때 추가 연결 없음 |
| 수신 확인 기능 (Ack) 없음 | 수신 확인 기능 (Ack) 있음 |

# Compare SPI and I²C SPI와 I2C 비교

| SPI | I²C |
| --- | --- |
| SCLK, MOSI, MISO, SS | SCL, SDA |
| I²C 보다 빠른 속도 | SPI 보다 느린 속도 |
| I²C 보다 적은 전력 소비 | SPI 보다 많은 전력 소비 |
| 4개의 연결선 | 2개의 연결선 |
| 장치가 증가할 때 연결선 SS 증가 | 장치가 증가할 때 추가 연결 없음 |
| 수신 확인 기능 (Ack) 없음 | 수신 확인 기능 (Ack) 있음 |
| 아주 짧은 거리 (예, PCB) | SPI 보다 긴 거리 |

# Compare SPI and I²C SPI와 I2C 비교

| SPI | I²C |
|---|---|
| SCLK, MOSI, MISO, SS | SCL, SDA |
| I²C 보다 빠른 속도 | SPI 보다 느린 속도 |
| I²C 보다 적은 전력 소비 | SPI 보다 많은 전력 소비 |
| 4개의 연결선 | 2개의 연결선 |
| 장치가 증가할 때 연결선 SS 증가 | 장치가 증가할 때 추가 연결 없음 |
| 수신 확인 기능 (Ack) 없음 | 수신 확인 기능 (Ack) 있음 |
| 아주 짧은 거리 (예, PCB) | SPI 보다 긴 거리 |
| 오직 하나의 마스터 지원 | 다수의 마스터 지원 |

# Compare SPI and I²C SPI와 I2C 비교

| SPI | I²C |
|---|---|
| SCLK, MOSI, MISO, SS | SCL, SDA |
| I²C 보다 빠른 속도 | SPI 보다 느린 속도 |
| I²C 보다 적은 전력 소비 | SPI 보다 많은 전력 소비 |
| 4개의 연결선 | 2개의 연결선 |
| 장치가 증가할 때 연결선 SS 증가 | 장치가 증가할 때 추가 연결 없음 |
| 수신 확인 기능 (Ack) 없음 | 수신 확인 기능 (Ack) 있음 |
| 아주 짧은 거리 (예, PCB) | SPI 보다 긴 거리 |
| 오직 하나의 마스터 지원 | 다수의 마스터 지원 |
| I²C 보다 노이즈에 약함 | SPI 보다 노이즈에 강함 |

# Compare SPI and I²C SPI와 I2C 비교

| SPI | I²C |
|---|---|
| SCLK, MOSI, MISO, SS | SCL, SDA |
| I²C 보다 빠른 속도 | SPI 보다 느린 속도 |
| I²C 보다 적은 전력 소비 | SPI 보다 많은 전력 소비 |
| 4개의 연결선 | 2개의 연결선 |
| 장치가 증가할 때 연결선 SS 증가 | 장치가 증가할 때 추가 연결 없음 |
| 수신 확인 기능 (Ack) 없음 | 수신 확인 기능 (Ack) 있음 |
| 아주 짧은 거리 (예, PCB) | SPI 보다 긴 거리 |
| 오직 하나의 마스터 지원 | 다수의 마스터 지원 |
| I²C 보다 노이즈에 약함 | SPI 보다 노이즈에 강함 |
| I²C 보다 높은 가격에 구현 | SPI 보다 저렴한 가격에 구현 |

# $I^2C$ 통신 LCD

# I²C LCD 관련 홈페이지[1,2]

Link Link

PLAY아두이노

아두이노 텍스트 LCD 2004 IIC/I2C 모듈 A56

아두이노 LCD Arduino LCD2004 IIC 12C A56

| | |
|---|---|
| 소비자가 | 견적문의요망 |
| 마일리지 | 0 |
| 제조사 | LKembedded |
| 상품코드 | P008559929 |
| 배송기간 | 입금 확인 후 2일~3일 |
| 재고위치 | 국내 재고 |
| 구매수량 | |

견적

보여진 사진은 제품을 대표적으로 묘사하는 사진입니다.
정확한 규격은 제품의 데이터시트에서 확인하셔야 합니다

제품상세정보 | 상품 Q&A | 배송/교환/환불

제품정보

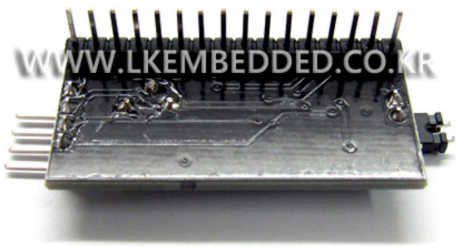
아두이노 텍스트 LCD 2004 IIC/I2C 모듈 A56 - 아두이노 LCD Arduino LCD2C

**제품명**

-텍스트 LCD 2004 IIC/I2C 모듈(블루백라이트내장 및 인터페이스보드내장)

**제품사진**

PLAY아두이노




**제품소개**

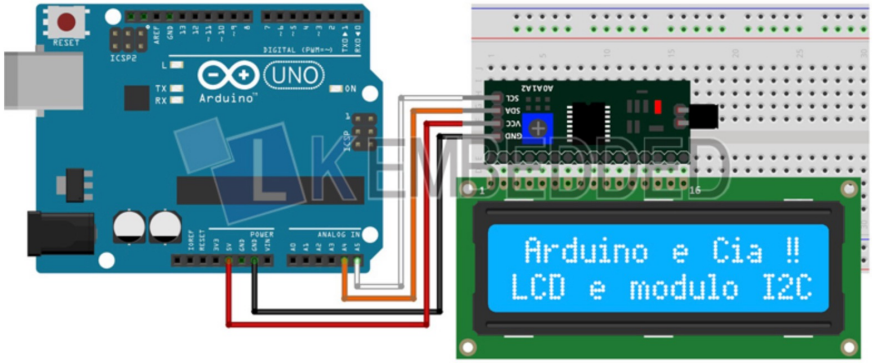
-LCD IIC/I2C Interface Adapter Plate 보드
-기존 16핀 LCD모듈을 사용하려면 많은 핀을 사용해야하지만 LCD I2C 인터페이스 변경 후 2핀으로만 작동가능
-가변저항을 통하여 LCD 밝기 조절 가능
-LCD1602, 1604, 2002, 2004 등 사용가능
-아두이노 스케치 프로그램 버전 1.6.4 이상에서는 동작하지 않을 수 있음

**제품특징**

-동작전압 5V
-통신방법 IIC/I2C

**제품자료**

-회로도




[1] https://www.icbanq.com/shop/product_detail.asp?prod_code=P008559929
[2] http://www.lkembedded.co.kr/goods/goods_view.php?goodsNo=15454

# I²C LCD 아두이노 라이브러리 사용자-생성 라이브러리

## ■ 라이브러리 다운로드 및 추가

fdebrabander / Arduino-LiquidCrystal-I2C-library

Watch 34 | Star 239 | Fork 247

Code | Issues 11 | Pull requests 1 | Projects 0 | Wiki | Insights

Join GitHub today

GitHub is home to over 28 million developers working together to host and review code, manage projects, and build software together.

Sign up

Dismiss

Library for the LiquidCrystal LCD display connected to an Arduino board.

18 commits | 1 branch | 0 releases | 2 contributors

Branch: master | New pull request | Find file | Clone or download

joaopedrosgs and fdebrabander Update LiquidCrystal_I2C.cpp — Latest commit e3701fb on Mar 9, 2017

| | | |
|---|---|---|
| examples | Remove the call to backlight(), to demonstrate it is on by default. | 7 years ago |
| LiquidCrystal_I2C.cpp | Update LiquidCrystal_I2C.cpp | 2 years ago |
| LiquidCrystal_I2C.h | Update LiquidCrystal_I2C.h | 2 years ago |
| README.md | Explained how to install the library and include it in your project. | 7 years ago |
| keywords.txt | Initial commit. | 7 years ago |

README.md

### Installation

Create a new folder called "LiquidCrystal_I2C" under the folder named "libraries" in your Arduino sketchbook folder. Create the folder "libraries" in case it does not exist yet. Place all the files in the "LiquidCrystal_I2C" folder.

### Usage

To use the library in your own sketch, select it from *Sketch > Import Library*.

This library is based on work done by DFROBOT (www.dfrobot.com).

[1] https://github.com/fdebrabander/Arduino-LiquidCrystal-I2C-library

# I²C LCD 아두이노 라이브러리 사용자-생성 라이브러리

## ■ 라이브러리 다운로드 및 추가

- 다운로드 받은 라이브러리 파일

Arduino-LiquidCrystal-I2C-library-master.zip

Arduino-LiquidCrystal-I2C-library-master

- examples
- keywords.txt
- LiquidCrystal_I2C.cpp
- LiquidCrystal_I2C.h
- README.md

- 아두이노 라이브러리의 일반적 구성
- 헤더 파일, 소스 파일, 예제 디렉토리

라이브러리 파일 구조

C:₩Users₩Jaeseok₩Documents₩Arduino₩libraries

- Arduino
  - libraries
    - LiquidCrystal_I2C

- 라이브러리 디렉토리 (LiquidCrystal_I2C)를 스케치북 밑에 위치한 사용자-생성 라이브러리 디렉토리에 생성한 후 복사

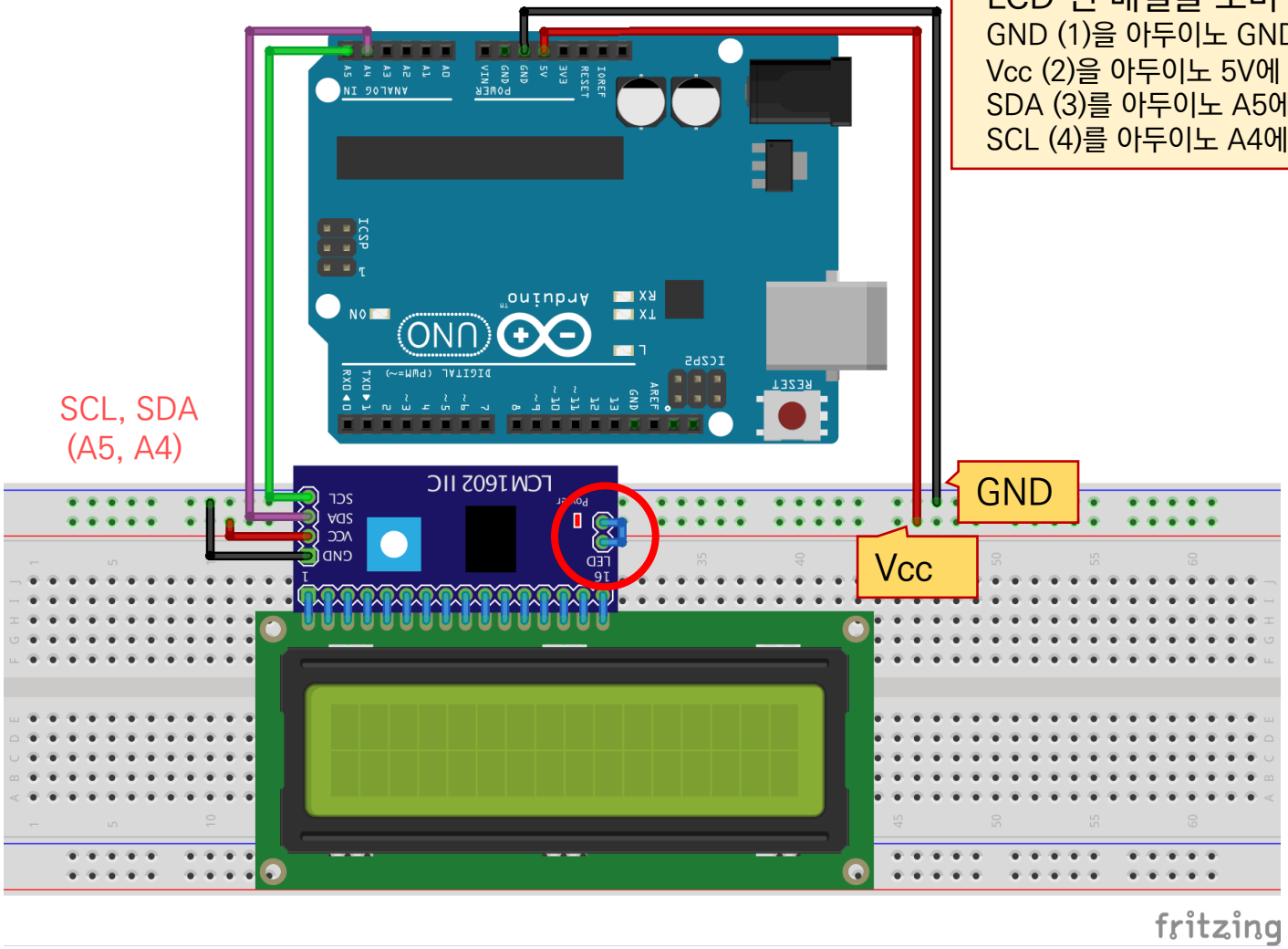
사용자-생성 라이브러리 디렉토리

[1] https://github.com/fdebrabander/Arduino-LiquidCrystal-I2C-library

# I²C LCD와 아두이노 연결

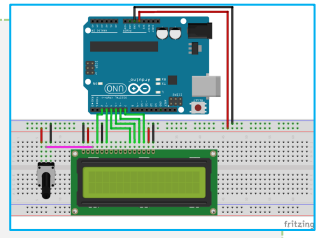
Practice

- LCD 핀 배열을 보며 (데이터시트 확인!)
  GND (1)을 아두이노 GND에 연결
  Vcc (2)을 아두이노 5V에 연결
  SDA (3)를 아두이노 A5에 연결
  SCL (4)를 아두이노 A4에 연결



# I²C LCD 스케치 OX 반복 출력

Practice



```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 20, 4);

void setup() {
  lcd.begin();
  lcd.backlight();
  lcd.print("Hello, Arduino!");
  lcd.setCursor(0, 1);
  lcd.print("Embedded Systems");
  delay(1000); lcd.clear();
}

void loop() {
  for (int j = 0; j < 4; j++) {
    for (int i = 0; i < 20; i++) {
      lcd.setCursor(i, j);
      lcd.print("X"); delay(10);
    }
  }

  for (j = 3; j >= 0; j--) {
    for (i = 19; i >= 0; i--) {
      lcd.setCursor(i, j);
      lcd.print("O"); delay(10);
    }
  }
}
```

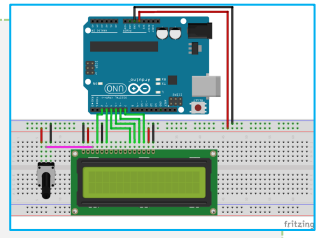
[1] https://github.com/fdebrabander/Arduino-LiquidCrystal-I2C-library

# I²C LCD 스케치 OX 반복 출력

Practice

- LiquidCrystal_I2C[1] 라이브러리 사용을 위해 LiquidCrystal_I2C.h 헤더 파일 포함
- I²C 라이브러리 사용을 위해 Wire.h 포함

- 0x27은 I²C 통신에 참여하는 LCD의 주소
- (20, 4)는 LCD의 (열, 행) 정의



```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 20, 4);

void setup() {
  lcd.begin();
  lcd.backlight();
  lcd.print("Hello, Arduino!");
  lcd.setCursor(0, 1);
  lcd.print("Embedded Systems");
  delay(1000); lcd.clear();
}

void loop() {
  for (int j = 0; j < 4; j++) {
    for (int i = 0; i < 20; i++) {
      lcd.setCursor(i, j);
      lcd.print("X"); delay(10);
    }
  }

  for (j = 3; j >= 0; j--) {
    for (i = 19; i >= 0; i--) {
      lcd.setCursor(i, j);
      lcd.print("O"); delay(10);
    }
  }
}
```

[1] https://github.com/fdebrabander/Arduino-LiquidCrystal-I2C-library

# I²C LCD 스케치 OX 반복 출력

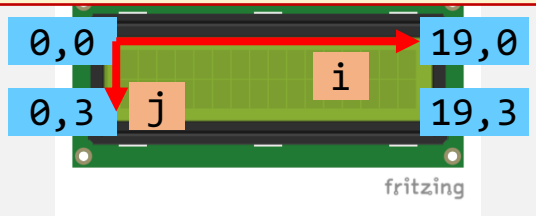
Practice

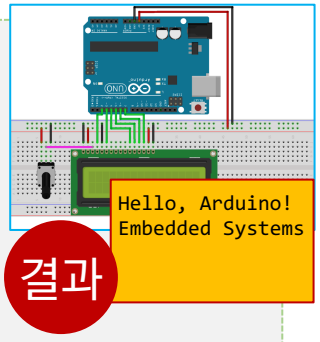
```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 20, 4);

void setup() {
  lcd.begin();
  lcd.backlight();
  lcd.print("Hello, Arduino!");
  lcd.setCursor(0, 1);
  lcd.print("Embedded Systems");
  delay(1000); lcd.clear();
}

void loop() {
  for (int j = 0; j < 4; j++) {
    for (int i = 0; i < 20; i++) {
      lcd.setCursor(i, j);
      lcd.print("X"); delay(10);
    }
  }

  for (j = 3; j >= 0; j--) {
    for (i = 19; i >= 0; i--) {
      lcd.setCursor(i, j);
      lcd.print("O"); delay(10);
    }
  }
}
```

- LiquidCrystal_I2C[1] 라이브러리 사용을 위해 LiquidCrystal_I2C.h 헤더 파일 포함
- I²C 라이브러리 사용을 위해 Wire.h 포함

- 0x27은 I²C 통신에 참여하는 LCD의 주소
- (20, 4)는 LCD의 (열, 행) 정의

- begin()은 기존 라이브러리와 달리 폭과 높이가 인수로 주어지지 않음 주의
- backlight()는 LCD 백라이트를 켜기
- print()는 LCD 스크린에 문자열을 출력, 길이가 스크린 폭을 넘을 경우 출력되지 않음
- setCursor()는 (열, 행) 위치로 커서를 이동
- clear()는 스크린을 깨끗이 지우기




결과

[1] https://github.com/fdebrabander/Arduino-LiquidCrystal-I2C-library

# I²C LCD 스케치 OX 반복 출력

Practice

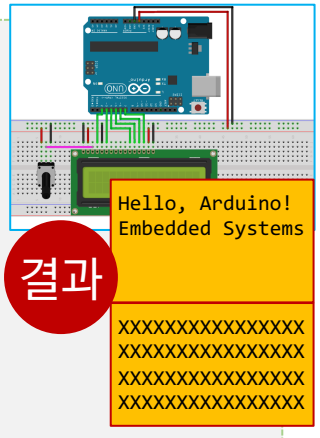
```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 20, 4);

void setup() {
  lcd.begin();
  lcd.backlight();
  lcd.print("Hello, Arduino!");
  lcd.setCursor(0, 1);
  lcd.print("Embedded Systems");
  delay(1000); lcd.clear();
}

void loop() {
  for (int j = 0; j < 4; j++) {
    for (int i = 0; i < 20; i++) {
      lcd.setCursor(i, j);
      lcd.print("X"); delay(10);
    }
  }

  for (j = 3; j >= 0; j--) {
    for (i = 19; i >= 0; i--) {
      lcd.setCursor(i, j);
      lcd.print("O"); delay(10);
    }
  }
}
```

- LiquidCrystal_I2C[1] 라이브러리 사용을 위해 LiquidCrystal_I2C.h 헤더 파일 포함
- I²C 라이브러리 사용을 위해 Wire.h 포함

- 0x27은 I²C 통신에 참여하는 LCD의 주소
- (20, 4)는 LCD의 (열, 행) 정의

- begin()은 기존 라이브러리와 달리 폭과 높이가 인수로 주어지지 않음 주의
- backlight()는 LCD 백라이트를 켜기
- print()는 LCD 스크린에 문자열을 출력, 길이가 스크린 폭을 넘을 경우 출력되지 않음
- setCursor()는 (열, 행) 위치로 커서를 이동
- clear()는 스크린을 깨끗이 지우기

- 중첩 for 문에서 행-인덱스 j가 먼저 사용됨을 유의
- 내부 for 문에서 열-인덱스 j를 이용해 문자 'X' 출력
- 20*4개의 'X'를 (0, 0)…(19, 3) 위치까지 10 ms 간격으로 하나씩 출력




[1] https://github.com/fdebrabander/Arduino-LiquidCrystal-I2C-library

# I²C LCD 스케치 OX 반복 출력

Practice

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 20, 4);

void setup() {
  lcd.begin();
  lcd.backlight();
  lcd.print("Hello, Arduino!");
  lcd.setCursor(0, 1);
  lcd.print("Embedded Systems");
  delay(1000); lcd.clear();
}

void loop() {
  for (int j = 0; j < 4; j++) {
    for (int i = 0; i < 20; i++) {
      lcd.setCursor(i, j);
      lcd.print("X"); delay(10);
    }
  }

  for (j = 3; j >= 0; j--) {
    for (i = 19; i >= 0; i--) {
      lcd.setCursor(i, j);
      lcd.print("O"); delay(10);
    }
  }
}
```

- LiquidCrystal_I2C[1] 라이브러리 사용을 위해 LiquidCrystal_I2C.h 헤더 파일 포함
- I²C 라이브러리 사용을 위해 Wire.h 포함

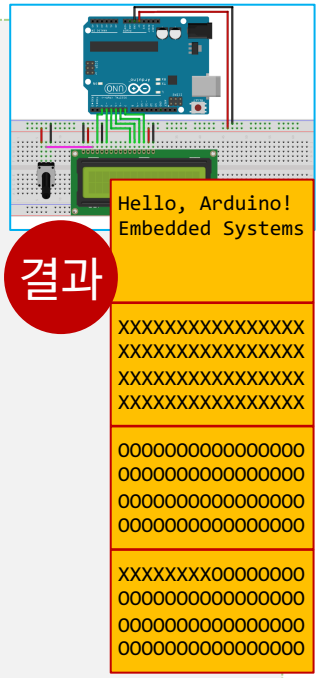
- 0x27은 I²C 통신에 참여하는 LCD의 주소
- (20, 4)는 LCD의 (열, 행) 정의

- begin()은 기존 라이브러리와 달리 폭과 높이가 인수로 주어지지 않음 주의
- backlight()는 LCD 백라이트를 켜기
- print()는 LCD 스크린에 문자열을 출력, 길이가 스크린 폭을 넘을 경우 출력되지 않음
- setCursor()는 (열, 행) 위치로 커서를 이동
- clear()는 스크린을 깨끗이 지우기

- 중첩 for 문에서 행-인덱스 j가 먼저 사용됨을 유의
- 내부 for 문에서 열-인덱스 j를 이용해 문자 'X' 출력
- 20*4개의 'X'를 (0, 0)…(19, 3) 위치까지 10 ms 간격으로 하나씩 출력

- 중첩 for 문에서 행-인덱스 j가 먼저 사용됨을 유의
- 내부 for 문에서 열-인덱스 j를 이용해 문자 'O' 출력
- 20*4개의 'O'를 (19, 3)…(0, 0) 위치까지 10 ms 간격으로 하나씩 출력

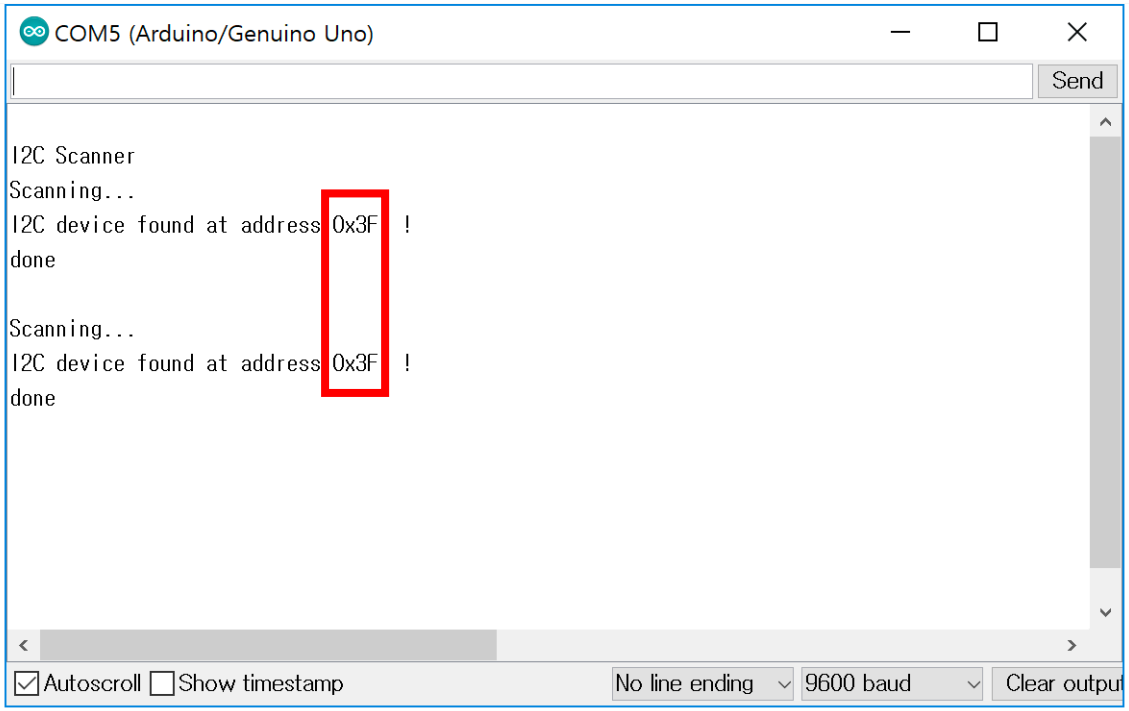
결과




[1] https://github.com/fdebrabander/Arduino-LiquidCrystal-I2C-library

# I²C 주소 검색

- I²C 장치의 7-bit 주소를 모를 경우 검색 필요
  - 'i2c_scanner[1]' 스케치를 활용하여 주소 검색
  - 사용 방법
    - ✓ I²C 장치를 아두이노에 연결
    - ✓ 'i2c_scanner' 스케치를 아두이노에 업로드
    - ✓ 시리얼 모니터를 통해 주소값 확인




[1] https://playground.arduino.cc/Main/I2cScanner

# Summary

- SPI hardware and spec
- SPI communication applications
- $I^2C$ hardware and spec
- $I^2C$ communication applications

# Q&A

Thank you

Questions?

Contact: eclass.sch.ac.kr
(순천향대학교 학습플랫폼 LMS)